

Robyn Documentation

Robyn Documentation

Robyn Documentation

Generated from
<https://robyn.tech/documentation/en>

Do not edit manually; run `python robyn_documentation/scrape.py` to regenerate.

API Documentation

Welcome to the Robyn API documentation. You'll find comprehensive guides and documentation to help you start working with Robyn as quickly as possible, as well as support if you get stuck.

We have divided the documentation into two parts: the Example Application and the API Docs.

Real World AppAPI Docs

Getting started

The Example Application is a simple web application that demonstrates how to use the Robyn API. It is a great place to start if you are new to Robyn.

The API Reference contains detailed information about the Robyn API. It is a great place to start if you are already familiar with Robyn and want to learn more about the API.

Example Application

Getting Started

Learn how to authenticate your API requests.

[Read more](#)

Authentication and Authorization

Understand how to use authentication and authorization.

[Read more](#)

Middleware

Read about different kinds of Middlewares and how to use them.

[Read more](#)

Monitoring and Logging

Learn how to have monitoring and logging in Robyn.

[Read more](#)

Real Time Notifications

Learn how to have real time notification in Robyn.

[Read more](#)

Deployments

Learn how to deploy your app to production and manage your deployments.

[Read more](#)

OpenAPI Documentation

Learn how OpenAPI docs are generate for your applications.

[Read more](#)

Api Docs

Installation

Start using Robyn in your project.

[Read more](#)

Getting Started

Start with creating basic routes in Robyn.

[Read more](#)

The Request Object

Learn about the Request Object in Robyn.

[Read more](#)

The Robyn Env file

Learn about the Robyn variables

[Read more](#)

Middleware, Events and Websockets

Learn about Middlewares, Events and Websockets in Robyn.

[Read more](#)

Authentication

Learn about Authentication in Robyn.

[Read more](#)

Const Requests and Multi Core Scaling

Learn about Const Requests and Multi Core Scaling in Robyn.

[Read more](#)

CORS

CORS

[Read more](#)

Templating

Learn about Templating in Robyn.

[Read more](#)

Redirection

Learn how to redirect requests to different endpoints.

[Read more](#)

File Uploads

Learn how to upload and download files to your server using Robyn.

[Read more](#)

Form Data and Multi Part Form Data

Learn how to handle form data.

[Read more](#)

Websockets

Learn how to use Websockets in Robyn.

[Read more](#)

Server-Sent Events

Learn how to implement Server-Sent Events for real-time communication.

[Read more](#)

Exceptions

Learn how to handle exceptions in Robyn.

[Read more](#)

Scaling the Application

Learn how to scaled Robyn across multiple cores.

[Read more](#)

Advanced Features

Learn about advanced features in Robyn.

[Read more](#)

Multiprocess Execution

Learn about the behaviour or variables during multithreading

[Read more](#)

Direct Rust Usage

Learn about directly using Rust in Robyn.

[Read more](#)

GraphQL Support

Learn about GraphQL Support in Robyn.

[Read more](#)

OpenAPI Documentation

Learn how to generate OpenAPI docs for your applications.

[Read more](#)

Dependency Injection

Learn about Dependency Injection in Robyn.

[Read more](#)

Once upon a time in the city of Gotham, there was a powerful superhero named Robyn. Robyn had a unique set of abilities that allowed it to fetch information from the far corners of the internet. It could send requests and receive responses at lightning speed, and its prowess was admired by developers everywhere.

One day, Batman approached Robyn for help with building a web application. Batman had heard about Robyn's powerful features and

wanted to harness them to create a remarkable application. Batman was looking for an ally and in Robyn, he found the best one!

Installing Robyn

Robyn is a Python library that you can install using `pip` OR `conda`

installation

`pipconda`

```
pip install robyn
```

```
```While there are other more  
extensions of Robyn like
```

```
installation
```

`pipconda`



pip install “robyntemplating” ``It is recommended to install the base package first and then install the extensions as needed.

## **What's next?**

Now, we can start using Robyn to build our application.

- Getting Started
- 

## **Keep a track of client's IP address**

Now that the portal was up and ready, Batman realised that the Joker was using the Gotham Police Dashboard

too. So, he wanted to keep a track of the IP address of the client who was accessing his application. He used the following code to do so:

Batman scaled his application across multiple cores for better performance. He used the following command:

## **Request**

GET

/hello\_world

```
from robyn import Robyn, Request
```

```
app = Robyn(__file__)
```

```
@app.get("/")
```

```
async def h(request: Request):
 return f"hello to you,"
```

```
{request.ip_addr}"
```## [What's next?]
(/documentation/en/api_reference/advanced
next)
```

Batman wondered about how to help users explore the endpoints in his application.

Robyn showed him the OpenAPI Documentation!

```
[OpenAPI Documentation]
(/documentation/en/api_reference/openapi_
```

```
---
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
# Advanced Routing and Parameter
Injection
```

Robyn's routing system goes far beyond simple URL matching. It includes

sophisticated parameter injection, route optimization, and flexible pattern matching that makes building complex APIs effortless.

[Understanding Parameter Injection]
(/documentation/en/api_reference/advanced/parameter-injection)

Robyn automatically analyzes your function signatures and injects the appropriate request components. This eliminates boilerplate code and makes handlers cleaner.

The Injection Engine

The parameter injection system works in two phases:

1. ****Function Introspection****: Robyn analyzes your function signature at registration time
2. ****Runtime Injection****: For each request, Robyn provides the exact parameters your function needs

****Type-Based Injection****: Uses type annotations to determine what to inject

Type-Based Parameter Injection

```
from robyn import Request,  
QueryParams, Headers from  
robyn.types import PathParams,  
RequestBody, RequestMethod
```

```
@app.post("/users/:user_id/posts/:post  
_id") async def update_post( # Robyn  
automatically injects these based on  
type annotations request: Request, #  
Complete request object path_params:  
PathParams, # {"user_id": "123",  
"post_id": "456"} query_params:  
QueryParams, # ?  
draft=true&tags=python,web headers:  
Headers, # All request headers body:  
RequestBody, # Raw request body
```

```
method: RequestMethod # “POST” ):
user_id = path_params[“user_id”]
post_id = path_params[“post_id”]
is_draft = query_params.get(“draft”)
== “true” content_type =
headers.get(“content-type”)

return {
    "user_id": user_id,
    "post_id": post_id,
    "is_draft": is_draft,
    "body_size": len(body),
    "method": method
}
```

``Name-Based Injection: Uses parameter names to inject components when type annotations aren't available

Name-Based Parameter Injection

```
# Reserved parameter names that Robyn
recognizes
@app.get("/search/:category")
def search_handler(
    query_params,          # Injected by
name
    path_params,          # Injected by
name
    headers,              # Injected by
name
    request                # Injected by
name (full request object)
):
    category = path_params["category"]
    search_term = query_params.get("q",
    "")
    user_agent = headers.get("user-
agent", "")

    return {
        "category": category,
        "search": search_term,
        "user_agent": user_agent
    }
```

```### Complete List of Injectable Types

Type	Annotation	Reserved Name	Description
---	---	---	---
`Request`	`request`, `req`, `r`		Complete request object
`QueryParams`	`query_params`		URL query parameters
`Headers`	`headers`		Request headers
`PathParams`	`path_params`		URL path parameters
`RequestBody`	`body`		Raw request body
`RequestMethod`	`method`		HTTP method (GET, POST, etc.)
`RequestURL`	`url`		Request URL information
`FormData`	`form_data`		Form-encoded data
`RequestFiles`	`files`		Uploaded files
`RequestIP`	`ip_addr`		Client IP address


```
| `RequestId` | `identity` |  
Authentication identity |
```

```
## [Advanced URL Patterns]  
(/documentation/en/api_reference/advanced  
url-patterns)
```

```
### Dynamic Route Parameters
```

Robyn supports multiple types of path parameters with flexible matching patterns.

```
### Dynamic Route Patterns
```

Simple parameter

```
@app.get("/users/:id") def  
get_user(path_params): return  
{“user_id”: path_params[“id”]}
```

Multiple parameters

```
@app.get("/users/:user_id/posts/:post_id") def get_user_post(path_params):  
    return { "user_id":  
            path_params["user_id"], "post_id":  
            path_params["post_id"] }
```

Optional parameters with defaults

```
@app.get("/posts/:id/:slug?") def  
get_post(path_params): post_id =  
path_params["id"] slug =  
path_params.get("slug", f"post-  
{post_id}") return {"id": post_id,  
"slug": slug}
```

Wildcard matching

```
@app.get("/files/*filepath") def  
serve_file(path_params): filepath =  
path_params["filepath"] return  
{“serving”: filepath} ``### Catch-all  
(wildcard) routes
```

A `*name` segment is a **catch-all**: unlike `:name` (which matches a single path segment), it matches the **rest of the path**, including multiple `/`-separated levels. The captured remainder is available in `request.path_params` under that name.

This is useful for single-page-app fallbacks, API gateways/proxies, and serving nested file paths.

Catch-all routes

```
@app.get("/files/*path")
async def read_file(request):
    # GET /files/img/2024/logo.png ->
    path == "img/2024/logo.png"
    return {"path":
request.path_params["path"]}
```

```
# SPA fallback: handle any unmatched
sub-path under /app
```

```
@app.get("/app/*path")
async def spa(request):
    return {"route":
request.path_params["path"]}
```

```
` `` `### Route Constraints and Validation
```

While Robyn doesn't have built-in parameter validation, you can implement it in your handlers for type safety.

```
### Parameter Validation
```

```
import re from robyn import  
HTTPException
```

```
@app.get("/users/:user_id") def  
get_user(path_params): user_id =  
path_params["user_id"]
```

```
# Validate that user_id is numeric  
if not user_id.isdigit():  
    raise HTTPException(400, "user_id  
must be numeric")
```

```
user_id = int(user_id)  
if user_id <= 0:  
    raise HTTPException(400, "user_id  
must be positive")
```

```
return {"user_id": user_id}
```

```
@app.get("/posts/:slug") def  
get_post_by_slug(path_params): slug =
```

```
path_params["slug"]
```

```
# Validate slug format
if not re.match(r'^[a-z0-9-]+$', slug):
    raise HTTPException(400, "Invalid
slug format")
```

```
return {"slug": slug}
```

``## Route Optimization

Const Routes for Static Responses

Use `const=True` for responses that never change. These are cached in Rust memory and the handler function is never re-executed after startup.

When no middleware is registered, const routes take a fast path served entirely from the Rust layer without entering Python at all. When

middleware is registered (including global before-request and after-request handlers), const routes still serve the cached response but middleware executes normally for every request. This means const routes are always safe to use alongside middleware.

Const Route Optimization

```
# Perfect for health checks, static  
configuration
```

```
@app.get("/health", const=True)  
def health_check():  
    return {"status": "healthy",  
"version": "1.0"}
```

```
# API metadata that rarely changes
```

```
@app.get("/api/info", const=True)  
def api_info():  
    return {  
        "name": "My API",  
        "version": "2.1.0",
```

```
        "documentation": "/docs"
    }

# Static configuration endpoints
@app.get("/config/public", const=True)
def public_config():
    return {
        "max_upload_size": "10MB",
        "allowed_origins":
["https://myapp.com"]
    }
```

```### Route Priority and Ordering

Routes are matched in the order they're registered. More specific routes should be registered before general ones.

### Route Ordering Best Practices

# GOOD: Specific routes first

```
@app.get("/users/profile") def
get_current_user_profile(): return
{“profile”: “current_user”}
```

```
@app.get("/users/settings") def
get_user_settings(): return {“settings”:
“user_settings”}
```

```
@app.get("/users/:id") def
get_user(path_params): return
{“user_id”: path_params[“id”]}
```

**BAD: This would never  
be reached**

**@app.get("/users/:id")**

**# Registered first**

**@app.get("/users/profile") # Never matched!**

**```## Advanced Query Parameter Handling**

## **Query Parameter Parsing**

Robyn provides rich query parameter handling with automatic type conversion helpers.

## **Advanced Query Parameters**

```
@app.get("/search")
def search(query_params):
```

```
Basic parameter access
q = query_params.get("q", "")

Parameters with defaults
page = int(query_params.get("page",
"1"))
limit =
int(query_params.get("limit", "10"))

Boolean parameters
include_deleted =
query_params.get("include_deleted",
"false").lower() == "true"

Array parameters (?
tags=python&tags=web&tags=api)
tags =
query_params.get_list("tags") or []

Convert to dict for easier
processing
all_params = query_params.to_dict()

return {
 "query": q,
 "page": page,
```

```
 "limit": limit,
 "include_deleted":
include_deleted,
 "tags": tags,
 "all_params": all_params
 }
```

```### Complex Query String Patterns

Handle complex query patterns like filtering, sorting, and nested parameters.

Complex Query Patterns

```
@app.get("/api/products") def
get_products(query_params): #
Filtering: ?
filter[category]=electronics&filter[price_min]=100 filters = {} for key, value
in query_params.to_dict().items(): if
key.startswith("filter[") and
key.endswith("]"): filter_key =
```



```
key[7:-1] # Remove “filter[” and ”]”  
filters[filter_key] = value
```

```
# Sorting: ?sort=price&order=desc  
sort_field = query_params.get("sort",  
"created_at")  
sort_order = query_params.get("order",  
"asc")
```

```
# Pagination: ?page=2&per_page=20  
page = int(query_params.get("page",  
"1"))  
per_page =  
min(int(query_params.get("per_page",  
"10")), 100) # Cap at 100
```

```
# Field selection: ?  
fields=id,name,price  
fields = query_params.get("fields",  
"").split(",") if  
query_params.get("fields") else None
```

```
return {  
    "filters": filters,  
    "sort": {"field": sort_field,  
"order": sort_order},
```

```
    "pagination": {"page": page,  
"per_page": per_page},  
    "fields": fields  
}
```

``## SubRouters and Modular Routing

Creating SubRouters

SubRouters help organize large applications by grouping related routes with common prefixes.

A SubRouter only needs a prefix (which may be passed positionally, e.g. `SubRouter("/api/v1")`). You can also pass tags to label every route the SubRouter contributes in the OpenAPI spec. Nested SubRouters accumulate their parents' prefixes and tags.

Earlier versions required the module name as the first argument
(SubRouter(__name__, prefix="/api/v1")).
That argument is no longer needed; it is deprecated and ignored if passed.

SubRouter Organization

```
from robyn import Robyn, SubRouter

app = Robyn(__file__)

# API v1 routes
api_v1 = SubRouter(prefix="/api/v1")

@api_v1.get("/users")
def list_users():
    return {"users": []}

@api_v1.get("/users/:id")
def get_user(path_params):
    return {"user_id":
path_params["id"]}

@api_v1.post("/users")
```

```
def create_user(body):  
    return {"created": True, "data":  
body}
```

```
# Admin routes  
admin = SubRouter(prefix="/admin")
```

```
@admin.get("/dashboard")  
def admin_dashboard():  
    return {"dashboard": "admin"}
```

```
@admin.get("/users")  
def admin_users():  
    return {"admin_users": []}
```

```
# Register subrouters  
app.include_router(api_v1)  
app.include_router(admin)
```

```
# Routes are now available at:  
# /api/v1/users, /api/v1/users/:id,  
/admin/dashboard, etc.
```

```
```### SubRouter Middleware
```

Configure authentication handlers on  
SubRouters and apply authentication to

```
routes using `auth_required=True`.
```

```
SubRouter Middleware
```

```
from robyn import SubRouter from
robyn.authentication import
AuthenticationHandler
```

# Admin routes with authentication

```
admin = SubRouter(prefix="/admin")
```

```
class
```

```
AdminAuth(AuthenticationHandler):
```

```
def authenticate(self, request):
```

```
 auth_header =
```

```
 request.headers.get("authorization",
 "") if not
```

```
 auth_header.startswith("Bearer "):
```

```
 return None
```

```
 token = auth_header[7:] # Remove
 "Bearer "
```

```
 return
self.validate_admin_token(token)

def validate_admin_token(self, token):
 # Your token validation logic
 if token == "admin-secret-token":
 return {"user": "admin"} #
Return identity object
 return None
```

# **Configure the authentication handler for this SubRouter**

```
admin.configure_authentication(AdminAuth())
```



# **Routes must explicitly require authentication with auth\_required=True**

```
@admin.get("/users",
auth_required=True) def
admin_users(): return
{“admin_users”: [“user1”, “user2”]}
```

```
@admin.delete("/users/:id",
auth_required=True) def
delete_user(path_params): return
```

```
{“deleted”: path_params[“id”]} ``##
```

## Route Testing and Debugging

### Route Inspection

Debug your routes by inspecting the registered route table.

### Route Debugging

```
from robyn import Robyn

app = Robyn(__file__)

@app.get("/users/:id")
def get_user(path_params):
 return {"user": path_params["id"]}

@app.post("/users")
def create_user(body):
 return {"created": True}

Debug: Print all registered routes
if __name__ == "__main__":
```

```
 print("Registered routes:")
 for route in app.get_routes():
 print(f"{route.method}
{route.path}")
```

```
 app.start(port=8080)
```### Route Performance Monitoring
```

Add timing middleware to monitor route performance.

```
### Performance Monitoring
```

```
import time from robyn import Robyn
```

```
app = Robyn(file)
```

```
@app.before_request def
timing_middleware(request):
    request.start_time = time.time()
    return request
```

```
@app.after_request def
timing_after_middleware(request,
```

```
response): duration = time.time() -  
request.start_time  
print(f"{request.method}  
{request.url.path} - {duration:.3f}s")  
response.headers["X-Response-Time"]  
= f"{duration:.3f}s" return response
```

```
@app.get("/slow") def  
slow_endpoint(): time.sleep(0.1) #  
Simulate work return {"message":  
"slow response"} ``## Best Practices
```

1. Parameter Injection Patterns

- **Use type annotations** for better IDE support and self-documenting code
- **Inject only what you need** to keep handlers focused
- **Validate parameters early** to provide clear error messages

2. Route Organization

- **Group related routes** using SubRouters
- **Order routes from specific to general** to avoid matching issues
- **Use consistent naming conventions** for path parameters

3. Performance Optimization

- **Use const routes** for static responses
- **Minimize parameter injection** in high-traffic endpoints
- **Cache expensive computations** rather than repeating them

4. Error Handling

- **Validate path parameters** early in handlers
- **Provide meaningful error messages** for invalid input
- **Use consistent error response formats** across your API

What's Next?

Now that you've mastered advanced routing, explore other Robyn features:

- [Middleware Development Guide](#)
 - [Performance Optimization](#)
 - [WebSocket Advanced Features](#)
-

Agents

This guide demonstrates how to build AI-powered agents using Robyn's MCP (Model Context Protocol) implementation.

Overview

The agent system connects AI assistants like Claude Desktop to your development environment, providing seamless access to:

- File system operations (read, search, organize)

- Task and note management
- System monitoring and git integration
- Web content fetching and analysis
- Context-aware code analysis

Quick Start

1. Run the MCP server:

```
python examples/agents.py
```

2. Connect your AI assistant to <http://localhost:8080/mcp>

2. Start using natural language commands:

- “What files are in my projects directory?”
- “Show me my recent git commits”

- “Create a note about today’s standup meeting”
- “What processes are using the most CPU?”
- “Add a task to review the quarterly report”

Configuration

The assistant creates the following structure:

```
~/Documents/  
├─ notes/           # Markdown notes  
└─ tasks.json      # Task list  
  
~/projects/        # Development  
projects  
├─ project1/  
└─ project2/  
`` `## [Security]  
(/documentation/en/api_reference/agents#
```

- * File access restricted to home directory
- * Safe mathematical expression evaluation
- * Path validation for all file operations
- * Read-only git operations

[Available Resources]
(/documentation/en/api_reference/agents#resources)

File System

- * ``fs://{path}`` - Read files in home directory
- * ``fs://dir/{path}`` - List directory contents

Git Integration

- * ``git://repo/{repo_name}`` - Repository status and commits

System Monitoring

- * ``system://processes`` - Running processes
- * ``system://stats`` - System statistics

[Available Tools]
(/documentation/en/api_reference/agents#tools)

- * ``create_note(title, content, tags)`` - Create markdown notes
- * ``add_task(task, priority, due_date)`` - Add tasks
- * ``complete_task(task_id)`` - Mark tasks complete
- * ``search_files(query, directory)`` - Search file contents
- * ``fetch_url_content(url, max_length)`` - Download web content

[Available Prompts]
(/documentation/en/api_reference/agents#prompts)

- * ``analyze_file_structure(directory)`` - Generate project analysis
- * ``code_review_request(file_path,`

```
focus_area)` - Create code reviews
* `task_prioritization(context)` -
Organize and prioritize work
```

```
## [Dependencies]
(/documentation/en/api_reference/agents/
```

Optional enhanced functionality:

```
pip install psutil # Enhanced system
monitoring ``## Implementation
Examples
```

Development Workflow

“Analyze my projects directory and help prioritize work based on recent activity”

Project Analysis

“Review my web-app project structure and suggest improvements”

Meeting Notes

“Create a note about today’s architecture review with key decisions”

Code Search

“Find all files mentioning ‘authentication’ and summarize approaches”

Task Management

“Add high-priority task to refactor user service, due Friday”

Integration Benefits

Connecting AI assistants to your development environment enables:

- Native file system browsing
- Context-aware project conversations
- Personalized code suggestions
- Real-time task management
- Workspace-specific code reviews

Advanced Features

The MCP implementation includes:

- URI templates with parameter extraction
- Auto-generated schemas from type hints
- Async/sync operation handlers
- MCP-compliant error handling
- Type-safe parameter passing

Extend easily with custom resources, tools, and prompts for your specific

workflow.

AI Agent and Memory

Robyn includes built-in AI capabilities that allow you to create intelligent applications with conversation memory, context awareness, and pluggable agent runners. The AI module provides abstractions for memory storage and agent execution that can be easily integrated into your Robyn applications.

Installation

The AI features are included with the base Robyn installation:


```
pip install robyn  
```## [Quick Start]  
(/documentation/en/api_reference/ai#quick
start)
```

Here's a simple example of using  
Robyn's AI features:

```
from robyn import Robyn from
robyn.ai import agent, memory

app = Robyn(file)
```

# Create memory instance

```
mem =
memory(provider="inmemory",
user_id="user123")
```

# Create agent with memory

```
chat_agent = agent(runner="simple",
memory=mem)
```

```
@app.get("/chat") async def
chat_endpoint(request): query =
request.query_params.get("q", [""])[0]
if not query: return {"error": "Query
required"}
```

```
Run agent with conversation history
result = await chat_agent.run(query,
history=True)
return result
```

## ``## Memory System

The memory system provides persistent storage for conversation history and context. It supports multiple providers and offers a consistent interface for storing and retrieving conversation data.

### **Memory Providers**

#### **InMemory Provider**

The simplest provider that stores data in memory. Data is lost when the application restarts.

```
from robyn.ai import memory
```

```
Create in-memory storage
mem = memory(provider="inmemory",
user_id="user123")
```

```
Add messages
await mem.add("Hello, how are you?")
await mem.add("I'm doing great,
thanks!")
```

```
Retrieve all messages
messages = await mem.get()
```

```
Clear memory
await mem.clear()
```### Memory API
```

The Memory class provides these key methods:

- * ``add(message, metadata=None)`` - Store a message with optional metadata
- * ``get(query=None)`` - Retrieve messages, optionally filtered by query
- * ``clear()`` - Clear all stored messages for the user

```
## [Agent System]
(/documentation/en/api_reference/ai#agent
system)
```

Agents provide the execution layer for AI functionality. They can use different runners and integrate with memory for context-aware responses.

Agent Runners

Simple Runner

A runner with OpenAI integration that provides intelligent responses:

```
from robyn.ai import agent
```

Create simple agent with OpenAI

```
from robyn.ai import configure
```

```
config =  
configure(openai_api_key="your-  
openai-key") simple_agent =  
agent(runner="simple", config=config)
```

Use the agent

```
result = await  
simple_agent.run("What's the  
weather like?") # Returns structured  
response with AI-generated content  
``### Agent API
```

The Agent class provides:

- `run(query, history=False, **kwargs)`
 - Execute the agent with optional history context
- Automatic memory integration when provided
- Support for custom runners and configuration

Complete Example

Here's a comprehensive example showing all features:

```
from robyn import Robyn
from robyn.ai import agent, memory

app = Robyn(__file__)

# Create memory with InMemory provider
mem = memory(
    provider="inmemory",
    user_id="guest"
)

# Create agent with memory
chat_agent = agent(runner="simple",
memory=mem)

@app.get("/")
async def home():
    return {"message": "Robyn AI Chat API"}
```

```
@app.post("/chat")
async def chat(request):
    """Chat with AI agent"""
    data = request.json()
    query = data.get("query", "")
    include_history =
data.get("history", True)

    if not query:
        return {"error": "Query is
required"}

    try:
        result = await
chat_agent.run(query,
history=include_history)
        return {
            "query": query,
            "response":
result.get("response"),
            "history_included":
include_history
        }
    except Exception as e:
        return {"error": str(e)}
```

```
@app.get("/memory")
async def get_memory():
    """Retrieve conversation history"""
    try:
        memories = await mem.get()
        return {"memories": memories,
"count": len(memories)}
    except Exception as e:
        return {"error": str(e)}
```

```
@app.delete("/memory")
async def clear_memory():
    """Clear conversation history"""
    try:
        await mem.clear()
        return {"message": "Memory
cleared"}
    except Exception as e:
        return {"error": str(e)}
```

```
@app.post("/memory")
async def add_memory(request):
    """Add message to memory"""
    data = request.json()
    message = data.get("message", "")
    metadata = data.get("metadata", {})
```

```

        if not message:
            return {"error": "Message is
required"}

        try:
            await mem.add(message,
metadata)
            return {"message": "Added to
memory"}
        except Exception as e:
            return {"error": str(e)}

if __name__ == "__main__":
    app.start(host="127.0.0.1",
port=8080)
```## [Advanced Usage]
(/documentation/en/api_reference/ai#adv:
usage)

```

### ### Custom Memory Providers

You can create custom memory providers by extending the `MemoryProvider` abstract base class:

```
from robyn.ai import
MemoryProvider from typing import
Dict, List, Any, Optional
```

```
class
CustomMemoryProvider(MemoryPro
vider): async def store(self, user_id:
str, data: Dict[str, Any]) -> None: #
Implement custom storage logic pass
```

```
async def retrieve(self, user_id: str,
query: Optional[str] = None) ->
List[Dict[str, Any]]:
 # Implement custom retrieval logic
 return []
```

```
async def clear(self, user_id: str) ->
None:
 # Implement custom clearing logic
 pass
```

# Use custom provider

```
from robyn.ai import Memory
custom_mem =
Memory(provider=CustomMemoryPr
ovider(), user_id="user123") ``###
Custom Agent Runners
```

Similarly, you can create custom agent runners:

```
from robyn.ai import AgentRunner
from typing import Dict, Any

class CustomAgentRunner(AgentRunner):
 async def run(self, query: str,
**kwargs) -> Dict[str, Any]:
 # Implement custom agent logic
```

```
 return {
 "response": f"Custom
response to: {query}",
 "processed": True
 }
```

```
Use custom runner
from robyn.ai import Agent
custom_agent =
Agent(runner=CustomAgentRunner())
```## [Best Practices]
(/documentation/en/api_reference/ai#best
practices)
```

1. ****User Isolation****: Always use unique user IDs to isolate memory between different users
2. ****Error Handling****: Wrap AI operations in try-catch blocks as external services may fail
3. ****Memory Management****: Regularly clear or archive old memories to prevent unbounded growth
4. ****Configuration****: Store sensitive configuration (API keys, etc.) in environment variables

5. ****Testing****: Use the simple runner for development and testing before deploying complex agents

[Troubleshooting]
(/documentation/en/api_reference/ai#trou

Common Issues

****ImportError for openai****: Install the required package:

pip install openai ``**Memory not persisting**: Note that the in-memory provider loses data when the application restarts. Consider implementing a custom persistent provider for production use.

Agent timeouts: Complex operations may take time. Consider implementing timeout handling in your endpoints.

Memory growing too large:
Implement periodic cleanup or use
providers with built-in retention
policies.

Architecture Deep Dive

Robyn's architecture is unique in the Python web framework landscape. It combines Python's expressiveness with Rust's performance through a carefully designed hybrid system. This deep dive explains how Robyn works under the hood and why it's so fast.

The Hybrid Python-Rust Design

Two-Layer Architecture

Robyn operates on two distinct but interconnected layers:

1. **Python Layer:** Provides the developer-facing API, routing configuration, and business logic
2. **Rust Layer:** Handles HTTP parsing, request routing, response generation, and I/O operations

The Python layer is where you write your application code. It handles:

- Route definitions and decorators
- Request parameter injection
- Middleware configuration

- Business logic execution
- Response formatting

Python Layer Example

```
from robyn import Robyn, Request
```

```
app = Robyn(__file__)
```

```
@app.get("/users/:id")
```

```
async def get_user(request: Request,  
user_id: str):
```

```
    # Business logic runs in Python
```

```
    user = await
```

```
fetch_user_from_db(user_id)
```

```
    return {"user": user.to_dict()}
```

```
```The Rust layer handles all  
performance-critical operations:
```

- \* HTTP request parsing
- \* URL routing and matching
- \* WebSocket connections
- \* Static file serving
- \* Response serialization

```
Rust Layer (Internal)
```

```
// This happens internally in Robyn's
Rust core impl HttpRouter { pub fn
route_request(&self, method: &str,
path: &str) -> Option { // High-
performance routing using matchit
crate
self.router.at(path).ok().map(|matche
d| { RouteInfo { handler:
matched.value.clone(), params:
matched.params.clone(), } }) } } ``###
PyO3 Bridge: Connecting Python and
Rust
```

The magic happens through PyO3,  
which enables seamless  
communication between Python and  
Rust:

**Function Registration:** When you define a route handler in Python, Robyn registers it with the Rust runtime through PyO3 bindings.

## Function Registration Flow

```
Python side - route registration
@app.get("/api/data")
def handler(request):
 return {"data": "example"}
```

```
Internally, this creates a
FunctionInfo object
that's passed to the Rust runtime
```**Request Execution Flow**:
```

When a request arrives, the Rust layer routes it and then calls back into Python to execute your handler.

Request Execution

1. HTTP Request arrives → Rust HTTP parser

2. Route matching → Rust router (matchit crate)
 3. Parameter extraction → Rust
 4. Handler execution → Python (via PyO3)
 5. Response processing → Rust
 6. HTTP Response → Client ``##
- Performance Optimizations

Fast Path for Static Responses

Robyn includes a “const routes” optimization for static responses:

When you mark a route as `const`, Robyn caches the response in Rust memory and never re-executes the Python handler. If no middleware is registered, const routes are served entirely from the Rust layer without entering Python at all. When

middleware is present, the cached response is still used but before-request and after-request middleware execute normally.

Const Routes

```
# This response is cached in Rust
memory
@app.get("/health", const=True)
def health_check():
    return {"status": "healthy"}
```

```
# Without middleware: served directly
from Rust, bypassing Python entirely
# With middleware: cached response is
used, but middleware still runs
```### Zero-Copy Request Handling
```

The Rust layer uses zero-copy techniques wherever possible:

- \* Request bodies are parsed once and shared between layers
- \* String data is referenced rather than



copied

- \* Response buffers are reused across requests

### ### Async Runtime Integration

Robyn integrates with Python's asyncio while maintaining Rust's async runtime:

**\*\*Sync Handlers\*\***: Executed in a thread pool to avoid blocking the async runtime

### ### Sync Handler Execution

```
@app.get("/sync") def
sync_handler(request): # Runs in
thread pool time.sleep(1) # Won't
block other requests return "Done"
``Async Handlers: Executed directly
in the async runtime for maximum
performance
```

# Async Handler Execution

```
@app.get("/async")
async def async_handler(request):
 # Runs in main async runtime
 await asyncio.sleep(1)
 return "Done"
```  
## [Advanced Parameter Injection]  
(/documentation/en/api_reference/architecture/parameter-injection)
```

One of Robyn's most sophisticated features is its parameter injection system:

Type-Based Injection

Robyn analyzes your function signatures and automatically injects the appropriate request components based on type annotations.

Type-Based Injection



```
from robyn import Request,  
QueryParams, Headers from  
robyn.types import PathParams,  
RequestBody
```

```
@app.post("/complex/:id") async def  
complex_handler( request: Request, #  
Full request object query_params:  
QueryParams, # ?param=value  
headers: Headers, # Request headers  
path_params: PathParams, # :id from  
URL body: RequestBody # Request  
body ): return { "id":  
path_params["id"], "query":  
query_params.to_dict(), "user_agent":  
headers.get("user-agent"),  
"body_length": len(body) } ```###  
Name-Based Injection
```

You can also use reserved parameter names without type annotations.

Name-Based Injection

```
@app.get("/simple/:id")
def simple_handler(query_params,
path_params, headers):
    # Parameters injected based on
names
    return {
        "id": path_params["id"],
        "search": query_params.get("q",
""),
        "auth":
headers.get("authorization")
    }
```
```

## [Memory Management]

(/documentation/en/api\_reference/archite  
management)

## ### Python Object Lifecycle

Robyn carefully manages Python objects across the Python-Rust boundary:

1. **\*\*Request Objects\*\***: Created once per request and reused
2. **\*\*Response Objects\*\***: Efficiently

serialized and passed to Rust

3. **\*\*Handler References\*\***: Stored in Rust and called via PyO3

### ### Rust Memory Safety

The Rust layer benefits from Rust's ownership system:

- \* No memory leaks from HTTP parsing
- \* Safe concurrent access to shared data
- \* Automatic cleanup of connection resources

## [Scaling Architecture]  
([/documentation/en/api\\_reference/architecture](/documentation/en/api_reference/architecture))

### ### Multi-Process Mode

Robyn can spawn multiple processes to utilize all CPU cores.

### ### Multi-Process Scaling

# **Spawn 4 worker processes**

```
python app.py --processes 4 --workers
2
```

# **Each process runs independently with shared-nothing architecture**

```
```### Multi-Worker Mode
```

Within each process, multiple worker threads handle requests concurrently.

Worker Thread Model

```
# Workers share the same Python  
interpreter  
# but handle requests concurrently
```

```
app.start(port=8080, workers=4)
```## [WebSocket Architecture]
(/documentation/en/api_reference/architecture/
architecture)
```

### ### Persistent Connections

Robyn's WebSocket implementation maintains persistent connections in the Rust layer while allowing Python handlers to process messages:

**\*\*Connection Management\*\***: Handled entirely in Rust for efficiency

**\*\*Message Processing\*\***: Python handlers process individual messages

**\*\*Broadcasting\*\***: Rust-based message distribution for high throughput

### ### WebSocket Flow

```
from robyn import
WebSocketDisconnect
```



```
@app.websocket("/chat") async def
websocket_handler(websocket): try:
while True: message = await
websocket.receive_text() response =
process_chat_message(message) await
websocket.send_text(response) except
WebSocketDisconnect: pass ``## Why
This Architecture Works
```

1. **Best of Both Worlds:** Python's productivity with Rust's performance
2. **Gradual Optimization:** Hot paths can be moved to Rust incrementally
3. **Memory Efficiency:** Minimal copying between layers
4. **Async Integration:** Seamless integration with Python's async/await

## 5. **Safety:** Rust's memory safety prevents common server vulnerabilities

This architecture allows Robyn to achieve performance comparable to pure Rust web frameworks while maintaining the ease of development that Python developers expect.

## What's Next?

Now that you understand Robyn's architecture, explore how to leverage its advanced features:

- [Advanced Routing and Parameter Injection](#)
  - [Performance Optimization Guide](#)
  - [WebSocket Deep Dive](#)
-

After Creating a basic version of the app, Batman wanted to restrict the access to the Gotham Police Department. So, he enquired about the Authentication functionalities in Robyn.

## **Authentication**

As Batman found out, Robyn provides an easy way to add an authentication middleware to your application. You can then specify `auth_required=True` in your routes to make them accessible only to authenticated users.

### **Request**

GET

# /hello\_world

```
@app.get("/auth", auth_required=True)
async def auth(request: Request):
 # This route method will only be
 # executed if the user is authenticated
 # Otherwise, a 401 response will be
 # returned
 return "Hello, world"
```

````To add an authentication middleware, you can use the `configure_authentication`` method. This method requires an `AuthenticationHandler`` object as an argument. This object specifies how to authenticate a user, and uses a `TokenGetter`` object to retrieve the token from the request. Robyn does currently provide a `BearerGetter`` class that gets the token from the `Authorization`` header, using the `Bearer`` scheme. Here is an example of a basic authentication handler:

```
### Request
```

GET

/hello_world

```
class
```

```
BasicAuthHandler(AuthenticationHan  
dler): def authenticate(self, request:  
Request) -> Optional[Identity]: token =  
self.token_getter.get_token(request) if  
token == "valid": return  
Identity(claims={}) return None
```

```
app.configure_authentication(BasicAu  
thHandler(token_getter=BearerGetter(  
)))`The authenticate method should  
return anIdentityobject if the user is  
authenticated, orNoneotherwise. The  
Identity object can contain any data  
you want, and will be accessible in the  
route methods using  
therequest.identity` attribute.
```

Note: that this authentication system is basically only using a before request middleware under the hood. This means you can overlook it and create your own authentication system using middlewares if you want to. However, Robyn still provides this easy to implement solution that should suit most use cases.

What's next?

Now, that Batman has learned about authentication, he wanted to know about certain optimization techniques that he could use to make his application faster. He found out about the following features

- Const Requests and Multi Core Scaling
-

Cheatsheet

A quick, copy-paste reference for the most common Robyn tasks. Every snippet targets the current release. For the full story on any topic, follow the links in the navigation sidebar.

Install and run a minimal app

app.py

```
# pip install robyn
from robyn import Robyn
```

```
app = Robyn(__file__)
```

```
@app.get("/")
```

```
def index():  
    return "Hello, world"  
  
app.start(host="0.0.0.0", port=8080)  #  
host defaults to 127.0.0.1  
```### Routes for every HTTP method
```

```
@app.get("/items") def list_items(): ...
```

```
@app.post("/items") def create_item():
```

```
...
```

```
@app.put("/items/:id") def
replace_item(): ...
```

```
@app.patch("/items/:id") def
update_item(): ...
```

```
@app.delete("/items/:id") def
delete_item(): ...
```



**also available:**

**@app.head,**

**@app.options,**

**@app.connect,**

**@app.trace**

````### Path parameters (single,  
optional, catch-all)`

```
@app.get("/users/:id")  
def get_user(request):  
    return {"id":  
request.path_params["id"]}
```

```
@app.get("/posts/:id/:slug?")
# optional trailing segment
def get_post(request):
    return
request.path_params.get("slug", "")
```

```
@app.get("/files/*path")
# catch-all: matches the rest of the
path
def read_file(request):
    return {"path":
request.path_params["path"]}    # e.g.
"img/2024/logo.png"
```### Query parameters
```

```
@app.get("/search") def
search(request): q =
request.query_params.get("q", "") #
last value, with a default tags =
request.query_params.get_all("tags")
every value -> list[str] return {"q": q,
"tags": tags} ```### Read request data
```

```

@app.post("/inspect")
def inspect(request):
 data = request.json()
 # parsed JSON body -> dict / list
 raw = request.body
 # raw body (str | bytes)
 form = request.form_data
 # dict[str, str]
 files = request.files
 # dict[str, bytes]
 content_type =
request.headers.get("Content-Type")
 return {
 "method": request.method,
 "path": request.url.path,
 "ip": request.ip_addr,
 }
````### Return responses

```

```

from robyn import Response,
status_codes

```

```

@app.get("/text") def text(): return
"plain text"

```

```
@app.get("/json") def json_body():  
    return {"key": "value"} # dict / list is  
    serialized to JSON automatically
```

```
@app.get("/custom") def custom():  
    return Response(  
        status_code=status_codes.HTTP_201_C  
        REATED, headers={"Content-Type":  
        "text/plain"}, body="created", #  
        description= is also accepted (legacy  
        alias) ) ``### Redirect
```

```
from robyn import Response
```

```
@app.get("/old")  
def old():  
    return Response(status_code=307,  
        headers={"Location": "/new"}, body="")  
````### Set a cookie
```

```
from robyn import Response, Headers
```

```
@app.get("/login") def login():
response = Response(status_code=200,
headers=Headers({}), body="ok")
response.set_cookie(key="session",
value="abc123", max_age=3600,
path="/", http_only=True,
secure=True, same_site="Strict",)
return response ``### Middleware
(before / after request)
```

```
@app.before_request()
global; pass a path for a per-route
hook
def add_trace(request):
 request.headers.set("x-trace", "1")
 return request
```

```
@app.after_request("/")
per-route; may take (request,
response)
def stamp(request, response):
 response.headers.set("x-served",
"robyn")
```

```
 return response
```### Authentication
```

```
from robyn import Request from
robyn.authentication import
AuthenticationHandler, BearerGetter,
Identity
```

```
class Auth(AuthenticationHandler):
def authenticate(self, request:
Request) -> Identity | None: token =
self.token_getter.get_token(request) #
reads “Bearer ” if token == “valid”:
return Identity(claims={“user”:
“bruce”}) return None
```

```
app.configure_authentication(Auth(to
ken_getter=BearerGetter()))
```

```
@app.get(“/me”, auth_required=True)
def me(request): return
```

request.identity.claims # populated
once authenticated ```### SubRouters

```
from robyn import SubRouter
```

```
api = SubRouter(prefix="/api/v1")
```

```
@api.get("/users")  
def list_users():  
    return {"users": []}
```

```
app.include_router(api)  
# routes mounted at /api/v1/users  
```### WebSockets
```

```
@app.websocket("/ws") async def
ws(websocket): while True: message =
await websocket.receive_text() await
websocket.send_text(f"echo:
{message}") # also: send_json(obj)
await websocket.broadcast("to
everyone") # every client on /ws
```

```
@ws.on_connect def
on_connect(websocket): return
“Welcome!”
```

```
@ws.on_close def
on_close(websocket): return
“Goodbye” ``### Streaming and
Server-Sent Events
```

```
from robyn import StreamingResponse,
SSEResponse, SSEMessage
```

```
@app.get("/sse")
def sse(request):
 def events():
 for i in range(3):
 yield SSEMessage(f"tick
{i}", event="tick", id=str(i))
 return SSEResponse(events())
text/event-stream
```

```
@app.get("/stream")
def stream(request):
 def chunks():
 yield b"chunk-1"
```



```
 yield b"chunk-2"
 return StreamingResponse(chunks(),
media_type="application/octet-stream")
```### Static files
```

```
from robyn import serve_file,
serve_html
```

```
app.serve_directory( route="/static",
directory_path="./assets",
index_file="index.html",
show_files_listing=False, )
```

```
@app.get("/download") def
download(): return
serve_file("./report.pdf",
file_name="report.pdf") # as an
attachment
```

```
@app.get("/page") def page(): return
serve_html("./templates/index.html")
```### Templating (Jinja2)
```

```
from robyn.templating import
JinjaTemplate

template = JinjaTemplate("./templates")

@app.get("/hello")
def hello():
 return
 template.render_template(template_name='
name="Bruce")
```### Raise an HTTP error
```

```
from robyn.exceptions import
HTTPException # note:
robyn.exceptions, not robyn
```

```
@app.get("/users/:id") def
get_user(request): if not
request.path_params["id"].isdigit():
raise HTTPException(400, "id must be
numeric") # (status_code, detail)
return {"id":
request.path_params["id"]} ```###
```

Const requests (computed once,
cached in Rust)

```
@app.get("/health", const=True)
def health():
    return {"status": "healthy"}
```### Dependency injection
```

```
app.inject_global(DB="global-db") #
available to every route
app.inject(CACHE="router-cache") #
available to this router's routes
```

```
@app.get("/data") def data(request,
global_dependencies,
router_dependencies): return { "db":
global_dependencies["DB"], "cache":
router_dependencies["CACHE"], }
```### CORS
```

```
from robyn import Robyn, ALLOW_CORS
```

```
app = Robyn(__file__)
```

```
ALLOW_CORS(app, origins=
["http://localhost:3000"])    # or
origins="*"
```### OpenAPI / Swagger
```

```
@app.get("/users",
openapi_name="List Users",
openapi_tags=["Users"]) def
list_users(): return {"users": []}
```

**Swagger UI is served at  
/docs and the spec at  
/openapi.json**

```
```### Lifecycle events
```

```
@app.startup_handler  
async def on_startup():  
    print("starting up")
```

```
@app.shutdown_handler  
def on_shutdown():  
    print("shutting down")
```

```
```### Scaling and CLI flags
```

```
python app.py --processes 4 --workers
2 # spread across cores / threads
```

```
python app.py --fast # auto-tune
processes, workers and log level
python app.py --dev # auto-reload on
change (single process) python app.py
--log-level WARNING # DEBUG / INFO /
WARNING / ERROR ``## What's next?
```

- Getting Started — the guided introduction
  - The Request Object and Response Objects
  - Advanced Routing — path params, wildcards and ordering
- 

After authentication, Batman was worried about the website traffic during the rush hours. He was worried about the server crashing when Joker would try to break all the criminals from the Arkham asylum one more time. So, Robyn told him

about the `Const Requests` feature and the multi-core scaling potential.

## Const Requests

Robyn told Batman that you can pre-compute the response for each route. This will compute the response even before execution. This will improve the response time bypassing the need to access the router.

### Request

GET

/hello\_world

```
@app.get("/", const=True)
async def h():
 return "Hello, world"
```

```
` `` ## [Multi-core scaling]
(/documentation/en/api_reference/const_1
core-scaling)
```

Robyn told Batman that he can use the `--workers`` flag to scale the application to multiple cores. This will create multiple instances of the application and will distribute the load among them. This will improve the performance of the application.

```
Request
```

```
python3 app.py --workers=N --
process=M`The authenticate method
should return anIdentityobject if the
user is authenticated,
orNoneotherwise. The Identity object
can contain any data you want, and will
be accessible in the route methods
using therquest.identity` attribute.
```



**Note: that this authentication system is basically only using a before request middleware under the hood. This means you can overlook it and create your own authentication system using middlewares if you want to. However, Robyn still provides this easy to implement solution that should suit most use cases.**

## **What's next?**

After making the application faster, Batman was happy and wanted to make a request from his Frontend Dashboard.

But he was faced with CORS issues! He asked Robyn about how to solve

this issue. Robyn told him about the following features

- CORS
- 

## CORS

Batman was annoyed on getting a CORS error whenever he tried to access the API.

## Scaling the Application

You can allow CORS for your application by adding the following code:

# Request

GET

/hello\_world

```
from robyn import Robyn, ALLOW_CORS

app = Robyn(__file__)
ALLOW_CORS(app, origins =
["http://localhost:<PORT>/"])
```## [What's next?]
(/documentation/en/api_reference/cors#wl
next)
```

After fixing the CORS issues. Batman was satisfied but he wanted to learn about ways to have small frontend pages in the server itself.

Robyn told him about templates and how he can use them to render HTML pages.

```
* [Templating]
(/documentation/en/api_reference/templat
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
## [Dependency Injection]
(/documentation/en/api_reference/depende
injection)
```

Batman wanted to learn about dependency injection in Robyn. Robyn introduced him to the concept of dependency injection and how it can be used in Robyn.

Robyn has two types of dependency injection:
One is for the application level and the other is for the router level.

```
### Application Level Dependency
Injection
```

Application level dependency injection is used to inject dependencies into the

application. These dependencies are available to all the requests.

```
### Request
```

```
GET
```

```
/hello\_world
```

```
from robyn import Robyn,  
ALLOW_CORS
```

```
app = Robyn(file)  
GLOBAL_DEPENDENCY = "GLOBAL  
DEPENDENCY"
```

```
app.inject_global(GLOBAL_DEPENDE  
NCY=GLOBAL_DEPENDENCY)
```

```
@app.get("/sync/global_di") def  
sync_global_di(request,  
global_dependencies): return  
global_dependencies["GLOBAL_DEPE
```

NDENCY”] ``### Router Level Dependency Injection

Router level dependency injection is used to inject dependencies into the router. These dependencies are available to all the requests of that router.

Request

GET

/hello_world

```
from robyn import Robyn, ALLOW_CORS,  
Request
```

```
app = Robyn(__file__)  
ROUTER_DEPENDENCY = "ROUTER  
DEPENDENCY"
```

```
app.inject(ROUTER_DEPENDENCY=ROUTER_DEPI
```

```
    @app.get("/sync/global_di")
    def sync_global_di(r: Request,
router_dependencies):
    return
router_dependencies["ROUTER_DEPENDENCY"]
```Note: `router_dependencies`,
`global_dependencies` are reserved
parameters and must be named as
such. The order of the parameters does
not matter among them. However, the
`router_dependencies` and
`global_dependencies` must only come
after the `request` parameter.
```

### ### WebSocket Dependency Injection

WebSockets support the same dependency injection system as HTTP routes. The `global\_dependencies` and `router\_dependencies` parameters work in the main handler, `on\_connect`, and `on\_close` callbacks.

### ### WebSocket DI

WebSocket

/chat

```
from robyn import Robyn import
logging
```

```
app = Robyn(file)
```

```
app.inject_global(logger=logging.getLo
gger(name))
app.inject(cache=RedisCache())
```

```
@app.websocket("/chat") async def
chat(websocket,
global_dependencies=None,
router_dependencies=None): logger =
global_dependencies.get("logger")
cache =
router_dependencies.get("cache")
```



```
logger.info(f"New connection:
{websocket.id}")
```

```
 while True:
 message = await
websocket.receive_text()
 cache.set(f"ws_{websocket.id}",
message)
 await websocket.broadcast(f"User
{websocket.id}: {message}")
```

```
@chat.on_connect async def
on_connect(websocket,
global_dependencies=None): logger =
global_dependencies.get("logger")
logger.info(f"Client connected:
{websocket.id}") return "Connected"
``## What's next?
```

Batman, being the familiar with the dark side wanted to know about Exceptions!

Robyn introduced him to the concept of exceptions and how he can use them to handle errors in his application.

- Exceptions
- 

## **Custom Exception Handler**

Batman learned how to create custom error handlers for different exception types in his application. He wrote the following code to handle exceptions and return a custom error response:

**Request**

GET

/hello\_world

```
@app.exception
def handle_exception(error: Exception):
 return Response(status_code=500,
description=f"error msg: {error}",
headers={})
```## [What's next?]
(/documentation/en/api_reference/except:
next)
```

Now, Batman wanted to scale his application across multiple cores. Robyn led him to Scaling.

```
* [Scaling]
(/documentation/en/api_reference/scaling)
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
uploads -->
```

```
## [File Uploads]
(/documentation/en/api_reference/file-
uploads#file-uploads)
```

Batman learned how to handle file uploads using Robyn. He created an endpoint to handle file uploads using the following code:

```
## [Sending a File without MultiPart  
Form Data]  
(/documentation/en/api_reference/file-  
uploads#sending-a-file-without-  
multipart-form-data)
```

Batman scaled his application across multiple cores for better performance. He used the following command:

```
### Request
```

```
GET
```

```
/hello\_world
```

```
@app.post("/upload") async def  
upload(): body = request.body file =  
bytearray(body)
```

```
# write whatever filename with  
open('test.txt', 'wb') as f: f.write(file)
```

```
return {'message': 'success'} ``##
```

Sending a File with MultiPart Form Data

Batman scaled his application across multiple cores for better performance. He used the following command:

Request

GET

/hello_world

```
@app.post("/sync/multipart-file")  
def sync_multipart_file(request:  
Request):  
    files = request.files  
    file_names = files.keys()
```

```
        return {"file_names":  
list(file_names)}  
```## [File Downloads]  
(/documentation/en/api_reference/file-
uploads#file-downloads)
```

Batman now wanted to allow users to download files from his application. He created an endpoint to handle file downloads using the following code:

```
Serving Simple HTML Files
```

Batman scaled his application across multiple cores for better performance. He used the following command:

```
Request
```

```
GET
```

```
/hello_world
```

```
from robyn import Robyn, Request,
serve_html
```

```
app = Robyn(file)
```

```
@app.get("/") async def h(request:
Request): return
serve_html("./index.html")
```

```
app.start(port=8080) ``### Serving
simple HTML strings
```

Speaking of HTML files, Batman wanted to serve simple HTML strings. He was suggested to use the following code:

## **Request**

GET

/hello\_world

```
from robyn import Robyn, Request, html
```

```
app = Robyn(__file__)
```

```
@app.get("/")
async def h(request: Request):
 html_string = "<h1>Hello
World</h1>"
 return html(html_string)

app.start(port=8080)
```### Serving Other Files
```

Now, that Batman was able to serve HTML files, he wanted to serve other files like CSS, JS, and images. He was suggested to use the following code:

```
### Request
```

```
GET
```

```
/hello\_world
```

```
from robyn import Robyn, serve_file,
Request
```



```
app = Robyn(file)
```

```
@app.get("/") async def h(request:  
Request): return  
serve_file("./index.html",  
file_name="index.html") # file_name  
is optional
```

```
app.start(port=8080) ``### Serving  
Directories
```

After serving other files, Batman wanted to serve directories, e.g. to serve a React build directory or just a simple HTML/CSS/JS directory. He was suggested to use the following code:

Request

GET

/hello_world

```

from robyn import Robyn, serve_file,
Request

app = Robyn(__file__)

app.serve_directory(
    route="/test_dir",

    directory_path=os.path.join(current_file,
    "build"),
    index_file="index.html",
)

app.start(port=8080)
```
[What's next?]
(/documentation/en/api_reference/file-
uploads#whats-next)

```

Now, Batman was ready to learn about the advanced features of Robyn. He wanted to find a way to handle form data

```

* [Form Data]
(/documentation/en/api_reference/form_data)

```

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
[Form Data and Multi Part Form Data]
(/documentation/en/api_reference/form_data_and_multi-part-form-data)
```

Batman learned how to handle file uploads using Robyn. Now, he wanted to handle the form data.

```
[Handling Multi Part Form Data]
(/documentation/en/api_reference/form_data_multi-part-form-data)
```

Batman uploaded some multipart form data and wanted to handle it using the following code:

```
Request
```

```
GET
```

```
/hello_world
```

```
@app.post("/upload") async def
upload(request: Request): form_data =
request.form_data
```

```
return form_data ``## What's next?
```

Now, Batman was ready to learn about the advanced features of Robyn. He wanted to find a way to get realtime updates in his dashboard.

- WebSockets

---

- Add performance optimizations
- Pydantic Integration
- Implement Auto Const Requests
- Add ORM support, especially Prisma integration

- Improve Plugin Ecosystem
- Better Documentation
- Improve the Websockets
- Template Support
- GraphQL integration with Strawberry
- Invest more time in the community around Robyn.

## **Next Steps**

- Advanced Features
- 

## **Building Your First Robyn Application**

Robyn is the fastest Python web framework, combining Python's

simplicity with Rust's performance. Whether you're building APIs, web services, or full-stack applications, Robyn makes it easy to get started and scale.

## Quick Start

Install Robyn and create your first application in minutes:

```
pip install robyn
```## [Understanding Handler Types]  
(/documentation/en/api_reference/getting  
handler-types)
```

Robyn supports both synchronous and asynchronous request handlers, allowing you to choose the best approach for your use case:

* **Synchronous handlers**: Perfect for CPU-bound operations, simple logic, or when you don't need to wait for

external resources

****Asynchronous handlers****: Ideal for I/O-bound operations like database calls, HTTP requests, file operations, or any task that involves waiting

****Synchronous handlers**** are perfect for simple operations, calculations, or when you don't need to wait for external resources:

Request

GET

/hello_world

from robyn import Robyn, Request

app = Robyn(**file**)

@app.get("/") def h(request: Request):
return "Hello, world"

```
app.start(port=8080, host="0.0.0.0") #
host is optional, defaults to 127.0.0.1
``Asynchronous handlers are ideal
for database operations, HTTP
requests, file I/O, or any operation
that involves waiting:
```

Request

GET

/hello_world

```
from robyn import Request
```

```
@app.get("/")
async def h(request: Request) -> str:
    return "Hello, world"
```### Complete Example: User
Management API
```

Here's a comprehensive example that demonstrates both sync and async handlers, proper error handling, and



real-world patterns:

### Complete API Example

API

/users

```
from robyn import Robyn, Request
import asyncio import json import
time from typing import Dict, Any
```

```
app = Robyn(file)
```

# **In-memory storage for demo (use a real database in production)**

```
users: Dict[str, Dict[str, Any]] = { "1":
{ "id": "1", "name": "Alice", "email":
"alice@example.com", "created_at":
"2024-01-01"}, "2": { "id": "2", "name":
"Bob", "email": "bob@example.com",
"created_at": "2024-01-02"} }
```

# **Const route for health checks (cached in Rust for max performance)**

```
@app.get("/health", const=True) def
health_check(): return {"status":
"healthy", "version": "1.0.0"}
```

# Async handler for database-like operations

```
@app.get("/users/:id") async def
get_user(path_params): user_id =
path_params["id"]

Simulate async database lookup
await asyncio.sleep(0.01) # Simulate
DB query time

if user_id in users:
 return {"success": True, "user":
users[user_id]}
else:
```

```
 return {"success": False, "error":
"User not found"}, 404
```

# Get all users with pagination

```
@app.get("/users") async def
list_users(query_params): page =
int(query_params.get("page", "1"))
limit = int(query_params.get("limit",
"10"))
```

```
Simulate async operation
await asyncio.sleep(0.01)
```

```
user_list = list(users.values())
start = (page - 1) * limit
end = start + limit
```

```
return {
```

```
 "users": user_list[start:end],
 "total": len(user_list),
 "page": page,
 "limit": limit
}
```

# Create new user

```
@app.post("/users") async def
create_user(body): try: data =
json.loads(body)

 # Validate required fields
 if not data.get("name") or not
data.get("email"):
 return {"success": False,
"error": "Name and email are
required"}, 400

 # Generate new ID
 new_id = str(len(users) + 1)
 new_user = {
 "id": new_id,
 "name": data["name"],
 "email": data["email"],
```



```
 "created_at":
time.strftime("%Y-%m-%d")
 }

 # Simulate async database save
 await asyncio.sleep(0.02)
 users[new_id] = new_user

 return {"success": True, "user":
new_user}, 201

except json.JSONDecodeError:
 return {"success": False, "error":
"Invalid JSON"}, 400
```

# Sync handler for CPU-intensive operations

```
@app.post("/calculate") def
calculate_fibonacci(body): try: data =
json.loads(body) n =
data.get("number", 10)
```

```
 if n < 0 or n > 35: # Prevent
excessive computation
 return {"error": "Number must
be between 0 and 35"}, 400
```

```
 # CPU-intensive calculation (runs
in thread pool)
 def fib(x):
 if x <= 1:
 return x
```

```

 return fib(x-1) + fib(x-2)

 start_time = time.time()
 result = fib(n)
 calc_time = time.time() -
start_time

 return {
 "input": n,
 "result": result,
 "calculation_time": f"
{calc_time:.4f}s"
 }

except json.JSONDecodeError:
 return {"error": "Invalid JSON"},
400

```

if **name** == “**main**”:

app.start(port=8080) ``### Testing  
Your API

Once your server is running, you can test these endpoints using curl or any

HTTP client:

## API Testing

CURL

Testing

```
Test health endpoint
```

```
curl http://localhost:8080/health
```

```
Get a user
```

```
curl http://localhost:8080/users/1
```

```
List users with pagination
```

```
curl "http://localhost:8080/users?
page=1&limit=5"
```

```
Create a new user
```

```
curl -X POST
```

```
http://localhost:8080/users \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"name": "Charlie", "email":
```

```
"charlie@example.com"}'
```

```
Calculate fibonacci
curl -X POST
http://localhost:8080/calculate \
 -H "Content-Type: application/json" \
 -d '{"number": 20}'
```## [Running Your Application]
(/documentation/en/api_reference/getting
your-application)
```

Robyn applications can be run in several ways, each optimized for different scenarios. Here are the most common approaches:

****Direct execution**** - Run your application file directly with various optimization flags:

- * `--dev``: Development mode with auto-reload
- * `--fast``: Optimized settings for production
- * `--processes N``: Scale across multiple CPU cores
- * `--workers N``: Multiple workers per process

Request

GET

/hello_world

usage: app.py [-h] [--processes
PROCESSES] [--workers WORKERS] [--
log-level LOG_LEVEL] [--create] [--
docs] [--open-browser] [--version]

Robyn, a fast async web framework
with a rust runtime.

options: -h, --help show this help
message and exit --processes
PROCESSES Choose the number of
processes. [Default: 1] --workers
WORKERS Choose the number of
workers. [Default: 1] --dev
Development mode. It restarts the

server based on file changes. `–log-level LOG_LEVEL` Set the log level name `–create` Create a new project template. `–docs` Open the Robyn documentation. `–open-browser` Open the browser on successful start. `–version` Show the Robyn version. `–compile-rust-path COMPILE_RUST_PATH` Compile rust files in the given path. `–create-rust-file CREATE_RUST_FILE` Create a rust file with the given name. `–disable-openapi` Disable the OpenAPI documentation. `–fast` Fast mode. It sets the optimal values for processes, workers and log level. However, you can override them. ``### Common Run Configurations

Development Mode: Best for local development with automatic

reloading when files change.

Development

DEV

```
# Basic development mode  
python app.py --dev
```

```
# Development with custom port  
python app.py --dev --port 3000
```

```
# Development with debug logging  
python app.py --dev --log-level DEBUG  
```**Production Mode**`: Optimized for  
performance with multiple processes and
workers.
```

```
Production
```

### PROD



# **Fast mode (automatic optimization)**

```
python app.py -fast
```

# Custom scaling configuration

```
python app.py --processes 4 --workers
2
```

# Production with specific log level

```
python app.py --fast --log-level INFO
```

``**Module execution** - Use Robyn's CLI module for additional features and consistent behavior across environments:

**Request**

GET

/hello\_world

```
usage: python -m robyn app.py [-h] [--processes PROCESSES] [--workers WORKERS] [--dev] [--log-level LOG_LEVEL] [--create] [--docs] [--open-browser] [--version]
```

Robyn, a fast async web framework with a rust runtime.

options:

-h, --help show this help message and exit

--processes PROCESSES Choose the number of processes. [Default: 1]

--workers WORKERS Choose the number of workers. [Default: 1]

--dev Development mode. It restarts the server based on file changes.

--log-level LOG\_LEVEL Set the log level name

--create Create a new project template.

`--docs` Open the Robyn documentation.

`--open-browser` Open the browser on successful start.

`--version` Show the Robyn version.

`--compile-rust-path COMPILER_RUST_PATH` Compile rust files in the given path.

`--create-rust-file CREATE_RUST_FILE` Create a rust file with the given name.

`--disable-openapi` Disable the OpenAPI documentation.

`--fast` Fast mode. It sets the optimal values for processes, workers and log level. However, you can override them.

````## [Handling Different HTTP Methods]`  
`(/documentation/en/api_reference/getting-different-http-methods)`

Robyn supports all standard HTTP methods. Here's how to create a complete RESTful API with proper request handling:

****Complete REST API Example****: Here's a practical example showing all HTTP methods for a blog post API:

REST API

CRUD

/posts

from robyn import Robyn, Request

app = Robyn(**file**)

In-memory storage for demo

```
posts = { "1": {"id": "1", "title": "First  
Post", "content": "Hello World"}, "2":  
{"id": "2", "title": "Second Post",  
"content": "Learning Robyn"} }
```

GET - Retrieve all posts

```
@app.get("/posts") def  
get_posts(query_params): limit =  
int(query_params.get("limit", "10"))  
posts_list = list(posts.values())[:limit]  
return {"posts": posts_list, "total":  
len(posts)}
```


GET - Retrieve specific post

```
@app.get("/posts/:id") def  
get_post(path_params): post_id =  
path_params["id"] if post_id in posts:  
return {"post": posts[post_id]} return  
{"error": "Post not found"}, 404
```

POST - Create new post

```
@app.post("/posts") def  
create_post(request: Request): data =  
request.json() post_id = str(len(posts) +  
1) new_post = { "id": post_id, "title":  
data.get("title", ""), "content":  
data.get("content", "") } posts[post_id]  
= new_post return {"message": "Post  
created", "post": new_post}, 201
```

PUT - Update entire post

```
@app.put("/posts/:id") def
update_post(request: Request,
path_params): post_id =
path_params["id"] if post_id not in
posts: return {"error": "Post not
found"}, 404

data = request.json()
posts[post_id] = {
    "id": post_id,
    "title": data.get("title", ""),
    "content": data.get("content", "")
}
```

```
return {"message": "Post updated",  
        "post": posts[post_id]}
```

PATCH - Partial update

```
@app.patch("/posts/:id") def
patch_post(request: Request,
path_params): post_id =
path_params["id"] if post_id not in
posts: return {"error": "Post not
found"}, 404

data = request.json()
post = posts[post_id]

# Update only provided fields
if "title" in data:
    post["title"] = data["title"]
if "content" in data:
    post["content"] = data["content"]
```

```
return {"message": "Post updated",  
        "post": post}
```

DELETE - Remove post

```
@app.delete("/posts/:id") def  
delete_post(path_params): post_id =  
path_params["id"] if post_id not in  
posts: return {"error": "Post not  
found"}, 404
```

```
deleted_post = posts.pop(post_id)  
return {"message": "Post deleted",  
"post": deleted_post}
```

``## Working with JSON and Response
Formats

Robyn automatically handles JSON
serialization, but also provides

flexible response formatting options for different use cases.

Automatic JSON Handling: Robyn automatically converts Python dictionaries and lists to JSON responses with the correct Content-Type headers.

JSON Responses

JSON

/api

```
from robyn import Robyn, Request
from datetime import datetime
```

```
app = Robyn(__file__)
```

```
# Simple JSON response - automatic
serialization
@app.get("/api/status")
```



```
def get_status():
    return {
        "status": "active",
        "timestamp":
datetime.now().isoformat(),
        "version": "1.0.0"
    }
```

Complex nested JSON

```
@app.get("/api/user/:id")
def get_user_profile(path_params):
    user_id = path_params["id"]
    return {
        "user": {
            "id": user_id,
            "profile": {
                "name": "John Doe",
                "email":
"john@example.com",
                "preferences": {
                    "theme": "dark",
                    "notifications":
True
                }
            },
            "activity": [
```

```
        {"action": "login",
"timestamp": "2024-01-15T10:30:00Z"},
        {"action": "view_post",
"timestamp": "2024-01-15T10:35:00Z"}
    ]
}
}
```

List/array responses

@app.get("/api/posts")

def get_posts():

```
    return [
        {"id": 1, "title": "First
Post", "published": True},
        {"id": 2, "title": "Draft
Post", "published": False},
        {"id": 3, "title": "Latest
Post", "published": True}
    ]
```

Custom status codes with JSON

@app.post("/api/posts")

def create_post(request: Request):

try:

data = request.json()

Validate required fields

```

        if not data.get("title"):
            return {"error": "Title is
required"}, 400

        # Success response
        return {
            "message": "Post created
successfully",
            "post": {
                "id": 123,
                "title": data["title"],
                "created_at":
datetime.now().isoformat()
            }
        }, 201
    except ValueError:
        return {"error": "Invalid JSON
format"}, 400
```## [Parameter Injection and Route
Handling]
(/documentation/en/api_reference/getting
injection-and-route-handling)

```

Robyn provides powerful parameter injection that automatically extracts and injects request components into

your handler functions. This eliminates boilerplate code and makes handlers cleaner and more focused.

**\*\*Path Parameters\*\***: Extract dynamic segments from URLs using colon syntax (``:param``)

**\*\*Type-Safe Injection\*\***: Use type annotations for automatic parameter injection with IDE support

### Path Parameters

Type-based injection   Name-based injection

GET

`/users/:id`

```
from robyn import Request from
robyn.types import PathParams
```

```
@app.get("/users/:id/posts/:post_id")
async def get_user_post(request:
Request, path_params: PathParams):
 user_id = path_params["id"] post_id =
 path_params["post_id"]

 # Validate parameters
 if not user_id.isdigit():
 return {"error": "Invalid user
ID"}, 400

 return {
 "user_id": int(user_id),
 "post_id": post_id,
 "url": request.url.path
 }
```

**``Query Parameters:** Access URL  
query strings with automatic parsing  
and type conversion helpers

# Query Parameters

Advanced query handlingSimple  
query access

GET

/search?q=python&page=1

```
from robyn import Request
from robyn.robyn import QueryParams

@app.get("/search")
async def search_products(request:
Request, query_params: QueryParams):
 # Get search parameters with
defaults
 query = query_params.get("q", "")
 page = int(query_params.get("page",
"1"))
 limit =
min(int(query_params.get("limit",
"10")), 100) # Cap at 100

 # Boolean parameter
```

```
 include_sold =
query_params.get("include_sold",
"false").lower() == "true"

 # Array parameters (?
tags=python&tags=web)
 tags =
query_params.get_list("tags") or []

 # Build response
 return {
 "search_query": query,
 "pagination": {"page": page,
"limit": limit},
 "filters": {"include_sold":
include_sold, "tags": tags},
 "total_params":
len(query_params.to_dict())
 }
```

Any request param can be used in the handler function either using type annotations or using the reserved names.

**\*\*Do note that the type annotations will take precedence over the reserved**

```
names.**
```

Robyn showed Batman example syntaxes of accessing the request params:

```
Request
```

```
GET
```

```
/split_request_params
```

```
from robyn.robyn import
QueryParams, Headers from
robyn.types import PathParams,
RequestMethod, RequestBody,
RequestURL
```

```
@app.get("/untyped/query_params")
def untyped_basic(query_params):
 return query_params.to_dict()
```

```
@app.get("/typed/query_params") def
typed_basic(query_data:
```



```
QueryParams): return
query_data.to_dict()
```

```
@app.get("/untyped/path_params/:id")
def
untyped_path_params(query_params:
PathParams): return query_params #
contains the path params since the
type annotations takes precedence
over the reserved names
```

```
@app.post("/typed_untyped/combined
") def typed_untyped_combined(
query_params, method_data:
RequestMethod, body_data:
RequestBody, url: RequestURL,
headers_item: Headers,): return {
"body": body_data, "query_params":
query_params.to_dict(), "method":
method_data, "url": url.path,
"headers": headers_item.get("server"),
```

```
} ``Type
```

```
Aliases: Request, QueryParams, Headers, PathParams, RequestBody, RequestMethod, RequestURL, FormData, RequestFiles, RequestIP, RequestIdentity`
```

Reserved Names: r, req, request, query\_params, headers, path\_params, body, method, url, ip\_addr, identity, form\_data, files

As Batman continued to develop his web application with Robyn, he explored more features and implemented them using code samples.

# **Customizing Response** **Formats and Headers**

After understanding the dynamic nature of Robyn, Batman, now wanted the ability to customize response formats and headers. Robyn showed him how to do this using dictionaries and Robyn's Response object.

## **Using Dictionaries**

Batman learned to customize response formats by returning dictionaries or using Robyn's Response object. He could also set status codes and headers for each response. For example, Batman

created a response with a dictionary like this:

## Request

GET

/hello\_world

```
from robyn import Request
```

```
@app.post("/dictionary")
async def dictionary(request: Request):
 return {
 "status_code": 200,
 "description": "This is a
regular response",
 "type": "text",
 "headers": {"Header":
"header_value"},
 }
```### Using the Response object
```

To use the Response object, he wrote:

```
### Request
```

```
GET
```

```
/hello\_world
```

```
from robyn.robyn import Response,  
Request
```

```
@app.get("/response") async def  
response(request: Request): return  
Response(status_code=200,  
headers=Headers({}),  
description="OK") ``### Returning a  
Binary Output
```

Batman then wanted to return a binary output from his application. He could do this by setting the type of the response to “binary” and returning a bytes object. For example, he wrote:

Request

GET

/hello_world

```
from robyn import Request, Response
```

```
@app.get("/binary_output_response_sync")
def
binary_output_response_sync(request:
Request):
    return Response(
        status_code=200,
        headers={"Content-Type":
"application/octet-stream"},
        description="OK",
    )
```

```
@app.get("/binary_output_async")
async def binary_output_async(request:
Request):
    return b"OK"
```

```

@app.get("/binary_output_response_async'
async def
binary_output_response_async(request:
Request):
    return Response(
        status_code=200,
        headers={"Content-Type":
"application/octet-stream"},
        description="OK",
    )
```## [Response Headers]
(/documentation/en/api_reference/getting
headers)

```

Batman, being the world's greatest detective, spotted the `headers` field in the `Response` object. He, naturally wanted to know more about it. Robyn explained that he could use the `headers` field to set response headers. For example, he could set the `Content-Type` header to `application/json` by writing:

```

Local Response Headers

```

Either, by using the `headers` field in the `Response` object:

```
Request
```

```
GET
```

```
/hello_world
```

```
from robyn import Request
```

```
@app.get("/") def
binary_output_response_sync(request
: Request): return Response(
status_code=200, headers={"Content-
Type": "application/octet-stream"},
description="OK",) ``### Global
Response Headers
```

Or setting the Headers globally *per* router.



# Request

GET

/hello\_world

```
app.add_response_header("content-type",
"application/json")
```

````add\_response\_header`` appends the header to the list of headers, while ``set\_response\_header`` replaces the header if it exists.

Request

GET

/hello_world

app.set_response_header("content-type", "application/json") ``To prevent the headers from getting applied to certain endpoints, you can use

the `exclude_response_headers_for` function.

Request

GET

/hello_world

```
app.exclude_response_headers_for(["/log:
/signup"])
```### Cookies
```

Robyn provides a complete cookie API following RFC 6265. Set cookies using the `set_cookie` method on the Response object.

### Request

GET

/hello\\_world



```
from robyn import Request, Response,
Headers
```

```
@app.get("/") def set_session(request:
Request): response = Response(200,
Headers({}), "Welcome!")
response.set_cookie(key="session",
value="abc123") return response
``#### Cookie Attributes
```

You can set additional cookie attributes for security and control:

- **path**: Cookie path (default: "/")
- **domain**: Cookie domain
- **max\_age**: Cookie lifetime in seconds
- **secure**: Only send over HTTPS
- **http\_only**: Not accessible via JavaScript
- **same\_site**: CSRF protection ("Strict", "Lax", or "None" - case

insensitive)

## Request

GET

/secure-cookie

```
from robyn import Request, Response,
Headers
```

```
@app.get("/login")
def login(request: Request):
 response = Response(200,
Headers({}), "Logged in")
 response.set_cookie(
 key="auth_token",
 value="secret123",
 path="/",
 max_age=3600, # 1 hour
 secure=True, # HTTPS
only
 http_only=True, # No
JavaScript access
 same_site="Strict", # CSRF
```

```
 protection
)
 return response
```#### Deleting Cookies
```

To delete a cookie from the browser, use the `delete` method on the cookies collection. This sets `max\_age=0` which tells the browser to remove the cookie.

```
#### Request
```

```
GET
```

```
/logout
```

```
from robyn import Request, Response, Headers
```

```
@app.get("/logout") def
logout(request: Request): response =
Response(200, Headers({}), "Logged
out")
response.cookies.delete("auth_token")
```

```
return response ``#### Accessing  
Cookies
```

You can iterate over cookies or access them by name:

Request

GET

/cookies

```
from robyn import Request, Response,  
Headers
```

```
@app.get("/debug")  
def debug_cookies(request: Request):  
    response = Response(200,  
Headers({}), "Cookies set")  
    response.set_cookie("a", "1")  
    response.set_cookie("b", "2")  
  
    # Get all cookie names  
    names = response.cookies.keys()
```

```
# Iterate over cookies
for name in response.cookies:
    print(f"Cookie: {name}")

# Check if cookie exists
if "a" in response.cookies:
    print("Cookie 'a' exists")

return response
```## [Request Headers]
(/documentation/en/api_reference/getting_headers)
```

Batman, now wanted to know how to read request headers. Robyn explained that he could use the `request.headers` field to read request headers. For example, he could read the `Content-Type` header by writing:

```
Local Request Headers
```

Either, by using the `headers` field in the `Request` object:

```
Request
```

```
GET
```

```
/hello_world
```

```
from robyn import Request
```

```
@app.get("/") def
binary_output_response_sync(request
: Request): headers = request.headers
```

```
print("These are the request
headers:", headers) existing_header =
headers.get("existing_header")
existing_header =
headers.get("existing_header",
"default_value") existing_header =
headers["existing_header"] # This
syntax is also valid
```



```
headers.set("modified",
"modified_value")
headers["new_header"] =
"new_value" # This syntax is also
valid
```

```
print("These are the modified request
headers:", headers)
```

```
return "" ``Or by using the global
Request Headers:
```

## **Request**

GET

/hello\_world

```
app.add_request_header("server",
"robyn")
````add_request_header` appends the  
header to the list of headers, while  
`set_request_header` replaces the
```

header if it exists.

Request

GET

/hello_world

app.set_request_header("server",
"robyn") ``## Status Codes

After learning about response formats and headers, Batman learned to set status codes for his responses.

Request

GET

/hello_world

from robyn import status_codes, Request

```
@app.get("/response")
async def response(request: Request):
    return
Response(status_code=status_codes.HTTP_200_OK,
headers=Headers({}), description="OK")
```## [What's next?]
(/documentation/en/api_reference/getting_started/next)
```

Great, now Robyn, what is the `Request` Object that you keep talking about?, Batman said. "Next section", said Robyn.

```
* [The Request Object]
(/documentation/en/api_reference/request_and_response)
```

Batman was also interested to know about the architecture of Robyn. "Next section", said Robyn.

```
* [Architecture]
(/documentation/en/architecture)
```

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
support -->
```

```
[GraphQL Support [(With Strawberry
🍓)](https://strawberry.rocks/)]
(/documentation/en/api_reference/graphql:
support#graphql-support-with-
strawberry-)
```

This is in a very early stage right now. We will have a much more stable version when we have a stable API for Views and View Controllers.

```
[Step 1: Creating a virtualenv]
(/documentation/en/api_reference/graphql:
support#step-1-creating-a-virtualenv)
```

To ensure that there are isolated dependencies, we will use virtual environments.

```
Virtual Environment
```

```
python3 -m venv venv ``## Step 2:
Activate the virtualenv and install
Robyn
```

## Activating the virtualenv

```
source venv/bin/activate
```### Installing Robyn and Strawberry
```

```
pip install robyn strawberry-graphql  
``## Step 3: Coding the App
```

Code

```
from typing import List, Optional  
from robyn import Robyn, jsonify  
import json
```

```
import dataclasses  
import strawberry  
import strawberry.utils.graphql
```

```
@strawberry.type
```

```
class User:
    name: str
```

```
@strawberry.type
class Query:
    @strawberry.field
    def user(self) -> Optional[User]:
        return User(name="Hello")
```

```
schema = strawberry.Schema(Query)
```

```
app = Robyn(__file__)
```

```
@app.get("/", const=True)
async def get():
    return
strawberry.utils.graphql.get_graphql_
```

```
@app.post("/")
async def post(request):
    body = request.json()
    query = body["query"]
```

```
variables = body.get("variables",
None)
context_value = {"request": request}
root_value = body.get("root_value",
None)
operation_name =
body.get("operation_name", None)

data = await schema.execute(
    query,
    variables,
    context_value,
    root_value,
    operation_name,
)

return jsonify(
    {
        "data": (data.data),
        **({"errors": data.errors} if
data.errors else {}),
        **({"extensions":
data.extensions} if data.extensions
else {}),
    }
)
```

```
if __name__ == "__main__":  
    app.start(port=8080, host="0.0.0.0")  
    ```Let us try to decipher the usage  
line by line.
```

These statements just import the dependencies.

```
Section 1
```

```
from typing import List, Optional
```

```
from robyn import Robyn, jsonify
import json
```

```
import dataclasses import strawberry
import strawberry.utils.graphql
```Here, we are creating a baseUser type  
with a name` property.
```


We are then creating a GraphQL query that returns the User.

Section 2

```
@strawberry.type
class User:
    name: str
```

```
@strawberry.type
class Query:
    @strawberry.field
    def user(self) -> Optional[User]:
        return User(name="Hello")
```

```
schema = strawberry.Schema(Query)
```

```
```Now, we are initializing a Robyn
app. For us, to serve a GraphQL app, we
need to have a `get` route to return
the `GraphQL(id)` and then a post
route to process the `GraphQL` request.
```

```
Section 3
```

`app = Robyn(file)` ``We are populating the html page with the GraphQL IDE using `strawberry`. We are using `const=True` to precompute this population. Essentially, making it very fast and bypassing the execution overhead in this get request.

## Section 4

```
@app.get("/", const=True)
async def get():
 return
 strawberry.utils.graphql.get_graphql_
    ````Finally, we are getting params(body,
    query, variables, context\_value,
    root\_value, operation\_name) from the
    `request` object.
```

Section 5

```
@app.post("/") async def
post(request): body = request.json()
```

```
query = body["query"] variables =  
body.get("variables", None)  
context_value = {"request": request}  
root_value = body.get("root_value",  
None) operation_name =  
body.get("operation_name", None)
```

```
data = await schema.execute( query,  
variables, context_value, root_value,  
operation_name, )
```

```
return jsonify( { "data": (data.data),  
({"errors": data.errors} if  
data.errors else {}), {"extensions":  
data.extensions} if data.extensions  
else {}), } ) ``The above is the example  
for just one route. You can do the  
same for as many as you like. :)
```

What's next?

That's all folks. :D Keep an eye out for more updates on this page. We will be adding more examples and documentation as we go along.

Model Context Protocol (MCP)

 **Experimental:** MCP support is experimental and may change.

Robyn supports MCP, allowing AI applications like Claude Desktop to connect to your application's resources and tools.

Quick Start

```
from robyn import Robyn  
  
app = Robyn(__file__)
```

```
@app.mcp.resource("echo://{message}")
def echo_resource(message: str) -> str:
    return f"Resource echo: {message}"
```

```
@app.mcp.tool()
def echo_tool(message: str) -> str:
    return f"Tool echo: {message}"
```

```
@app.mcp.prompt()
def echo_prompt(message: str) -> str:
    return f"Please process: {message}"
```

```
app.start()
```## [Features]
(/documentation/en/api_reference/mcps#fe
```

- \* Auto-generated JSON schemas from function signatures
- \* URI templates with parameter extraction
- \* JSON-RPC 2.0 protocol compliance
- \* Type-aware parameter handling

```
[Decorator Reference]
(/documentation/en/api_reference/mcps#de
```

reference)

```
`@app.mcp.resource(uri, name=None,
description=None, mime_type=None)`
```

Register a resource that can be read by clients.

```
@app.mcp.resource("user://{user_id}/
profile") def user_profile(user_id: str) -
> str: return f"Profile for user
{user_id}"
`###@app.mcp.tool(name=None,
description=None,
input_schema=None)`
```

Register a tool that can be executed by AI models.

```
@app.mcp.tool()
def greet(name: str, formal: bool =
False) -> str:
 if formal:
 return f"Good day, {name}."
```

```
 return f"Hi {name}!"
```### `@app.mcp.prompt(name=None,
description=None, arguments=None)`
```

Register a prompt template for AI workflows.

```
@app.mcp.prompt() def
code_review(code: str, language: str =
“python”) -> str: return f”Please
review this {language} code: {code}”
```### Type Support
```

Supported types: str, int, float, bool,  
List, Dict

## URI Templates

Extract parameters from URIs:

```
@app.mcp.resource("user://{user_id}/post")
def get_user_post(user_id: str,
```



```
post_id: str) -> str:
 return f"Post {post_id} from user {user_id}"
```## [Client Usage]
(/documentation/en/api_reference/mcps#cli:usage)
```

Test with curl:



List resources

```
curl -X POST http://localhost:8080/mcp  
-H "Content-Type: application/json"  
-d '{"jsonrpc": "2.0", "id": 1, "method":  
"resources/list", "params": {}}' ``##
```

Integration with Claude Desktop

To connect your Robyn MCP server to Claude Desktop:

1. Start your Robyn app with MCP endpoints
2. Configure Claude Desktop to connect to
`http://localhost:8080/mcp`

3. Use the registered resources, tools, and prompts in your conversations

Error Handling

```
@app.mcp.tool()
def divide(a: float, b: float) -> str:
    if b == 0:
        raise ValueError("Division by
zero")
    return str(a / b)
```## [Testing]
(/documentation/en/api_reference/mcps#te
```

# Unit tests

```
python examples/mcp.py test-unit
```

# Live tests

`python examples/mcp.py test-live`

# All tests

```
python examples/mcp.py test-all ``##
Configuration
```

MCP runs at `/mcp` endpoint using JSON-RPC 2.0 over HTTP. No additional setup required.

## Claude Desktop Integration

1. Start your Robyn server
2. Configure Claude Desktop to connect to  
`http://localhost:8080/mcp`

### 3. Use resources, tools, and prompts in conversations

See `examples/mcp.py` for a complete example.

For more information:

<https://modelcontextprotocol.io/>

---

## **Working with Middlewares and Events**

As Batman's application grew more complex, Robyn taught him about middlewares, startup and shutdown events, and even working with WebSockets. Batman learned how to create functions that could execute before or after a request, manage the

application's life cycle, and handle real-time communication with clients using WebSockets.

## **Handling Events**

Batman discovered that he could add startup and shutdown events to manage his application's life cycle. He added the following code to define these events:

Batman was excited to learn that he could add events as functions as well as decorators.

### **Request**

GET

/hello\_world



```
async def startup_handler():
 print("Starting up")

app.startup_handler(startup_handler)

@app.shutdown_handler
def shutdown_handler():
 print("Shutting down")
```For an asynchronous request, Batman
used:
```

```
### Request
```

```
GET
```

```
/hello\_world
```

```
from robyn import Request
```

```
@app.get("/") async def h(request:
Request) -> str: return "Hello, world"
```

```
## [Handling Middlewares]
(/documentation/en/api_reference/middlew
middlewares)
```

Batman learned to use both sync and async functions for middlewares. He wrote the following code to add a middleware that would execute before and after each request.

A before request middleware is a function that executes before each request. It can modify the request object or perform any other operation before the request is processed.

An after request middleware is a function that executes after each request. It can modify the response object or perform any other operation after the request is processed.

Every before request middleware should accept a request object and return a request object. Every after-request middleware should accept a response object and return a response object on happy case scenario. After-request middlewares can also optionally accept the request object as the first parameter to access request data.

The ``before_request`` chain stops as soon as one of its middlewares returns a response object: the remaining ``before_request`` middlewares and the route handler (the main entry point) are skipped. The ``after_request`` middlewares, however, still run on that response before it is returned to the client – so use them for work that must always happen (e.g. logging, headers), even on a short-circuited request.

Request

Pythonafter_request with request access

POST

/http_requests

from robyn import Request, Response

```
@app.before_request("/") async def  
hello_before_request(request:  
Request):  
    request.headers.set("before",  
    "sync_before_request") return request
```

```
@app.after_request("/") def  
hello_after_request(response:  
Response):  
    response.headers.set("after",  
    "sync_after_request") return response  
``## Multiple middlewares on the  
same route
```

Batman sometimes needed more than one thing to happen before a request — say, logging *and* input sanitisation — and often alongside authentication. Robyn lets you stack as many `before_request` and `after_request` handlers on the same route as you

like; they run in the order they were registered.

The `before_request` chain runs top to bottom and stops early if a handler returns a `Response` — the remaining `before_request` middlewares and the route handler are then skipped. The `after_request` chain always runs over the outgoing response in registration order, whether that response came from the route handler or from an early-returning `before_request`.

Because `auth_required=True` is itself a `before_request` hook, it composes with your own middlewares: authentication runs first, then your custom filters.

Multiple middlewares

GET

/index

```
from robyn import Request, Response
```

```
@app.get("/index", auth_required=True)
```

```
# 1. auth runs first
```

```
async def index(request: Request):  
    return "Index Page"
```

```
@app.before_request("/index")
```

```
# 2. then this
```

```
async def log_request(request:  
Request):  
    print("logging:", request.url.path)  
    return request
```

```
@app.before_request("/index")
```

```
# 3. then this – all run, in order
```

```
async def sanitize(request: Request):  
    return request
```

```
@app.after_request("/index")
```

```
async def add_header(response:
Response):
    response.headers.set("x-processed",
"true")
    return response
```## [What's next?]
(/documentation/en/api_reference/middlew
next)
```

Robyn - Great, you're now familiar with the certain advanced concepts of Robyn.

Batman - "Authentication! I want to learn about authentication. I want to make sure that only the right people can access my application."

Robyn - Yes, Authentication!

```
* [Authentication]
(/documentation/en/api_reference/authenti
```

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
```

-->

```
[Multiprocess Execution]
(/documentation/en/api_reference/multiprocess
execution)
```

Batman wondered about the behaviour of variables in a Robyn multiprocessing environment.

Robyn reassured that it can indeed support them! i.e, handlers can be dispatched to multiple threads.

Any variable used in a multiprocessing environment is shared across multiple processes.

Whilst using multithreading in Robyn, the variables are not protected from multiple threads access by default.

If one needs a variable to be protected within a process, while accessing it from different threads, one can use `[`multiprocessing.Value`]`



(<https://docs.python.org/3/library/mult:>  
for achieving the required protection.

```
Request
```

```
GET
```

```
/hello_world
```

```
import threading
import time
from multiprocessing import Value
```

```
from robyn import Robyn, Request
```

```
app = Robyn(__file__)
```

```
count: Value = Value("i", 0)
```

```
def counter():
 while True:
 count.value += 1
 time.sleep(0.2)
 print(count.value, "added 1")
```

```
@app.get("/")
```

```
def index(request: Request):
 return f"{count.value}"

threading.Thread(target=counter,
daemon=True).start()

app.start()
```

``## What's next?

Batman wondered if it was possible to use Rust directly from Robyn's codebase.

Robyn showed him the path.

Using Rust Directly

---

# OpenAPI Docs a.k.a Swagger

After deploying the application, Batman got multiple queries from the users on how to use the endpoints. Robyn showed him how to generate OpenAPI specifications for his application.

Out of the box, the following endpoints are setup for you:

- `/docs` The Swagger UI
- `/openapi.json` The JSON Specification

To use a custom openapi configuration, you can:

- Place the `openapi.json` config file in the root directory.
- Or, pass the file path to the `openapi_file_path` parameter in the `Robyn()` constructor. (the parameter gets priority over the file).

However, if you don't want to generate the OpenAPI docs, you can disable it by passing `--disable-openapi` flag while starting the application.

```
python app.py --disable-openapi
```## [How to use?]
(/documentation/en/api_reference/openapi:
to-use)
```

```
* Query Params: The typing for query
params can be added as `def get(r:
Request, query_params:
GetRequestParams)` where
`GetRequestParams` is a subclass of
`QueryParams`
```

```
* Path Params are defaulted to string
```

```
type (ref:  
<https://en.wikipedia.org/wiki/Query_stl
```

```
### Basic App
```

```
from robyn.robyn import  
QueryParams
```

```
from robyn import Robyn, Request
```

```
app = Robyn( file_object=file,  
openapi=OpenAPI( info=OpenAPIInfo(  
title="Sample App", description="This  
is a sample server application.",  
termsOfService="https://example.com/  
terms/", version="1.0.0",  
contact=Contact( name="API Support",  
url="https://www.example.com/suppo  
rt", email="support@example.com", ),  
license=License( name="BSD2.0",  
url="https://opensource.org/license/bs  
d-2-clause", ),
```

```
externalDocs=ExternalDocumentation  
(description="Find more info here",  
url="https://example.com/"),  
components=Components(), ), ), )
```

```
@app.get("/") async def welcome():  
    """welcome endpoint""" return "hi"
```

```
class  
GetRequestParams(QueryParams):  
    appointment_id: str year: int
```

```
@app.get("/api/v1/name",  
openapi_name="Name Route",  
openapi_tags=["Name"]) async def  
get(r: Request, query_params:  
GetRequestParams): """Get Name by  
ID""" return r.query_params
```

```
@app.delete("/users/:name",  
openapi_tags=["Name"]) async def
```

```
delete(r: Request): “““Delete Name by  
ID”“” return r.path_params
```

```
if name == “main”: app.start() ``##
```

How does it work with subrouters?

Subrouters

```
from robyn.robyn import QueryParams
```

```
from robyn import Request, SubRouter
```

```
subrouter: SubRouter =  
SubRouter(prefix="/sub")
```

```
@subrouter.get("/")  
async def subrouter_welcome():  
    """welcome subrouter"""  
    return "hiiii subrouter"
```

```
class  
SubRouterGetRequestParams(QueryParams):  
    _id: int
```

```
value: str
```

```
@subrouter.get("/name")
async def subrouter_get(r: Request,
query_params:
SubRouterGetRequestParams):
    """Get Name by ID"""
    return r.query_params
```

```
@subrouter.delete("/:name")
async def subrouter_delete(r: Request):
    """Delete Name by ID"""
    return r.path_params
```

```
app.include_router(subrouter)
```## [Other Specification Params]
(/documentation/en/api_reference/openapi:
specification-params)
```

We support all the params mentioned in the latest OpenAPI specifications (<https://swagger.io/specification/>). See an example using request & response



bodies below:

### Request & Response Body

```
from robyn.types import
JSONResponse, Body
```

```
class Initial(Body): is_present: bool
letter: Optional[str]
```

```
class FullName(Body): first: str
second: str initial: Initial
```

```
class CreateItemBody(Body): name:
FullName description: str price: float
tax: float
```

```
class CreateResponse(JSONResponse):
success: bool items_changed: int
```

```
@app.post("/") def
create_item(request: Request, body:
```

```
CreateItemBody) -> CreateResponse:
return CreateResponse(success=True,
items_changed=2) ```With the
reference documentation deployed
and running smoothly, Batman had a
powerful new tool at his disposal. The
Robyn framework had provided him
with the flexibility, scalability, and
performance needed to create an
effective crime-fighting application,
giving him a technological edge in his
ongoing battle to protect Gotham City.
```

## Using Pydantic Models

If you have Pydantic installed (`pip install "robyn[pydantic]"` OR `pip install "robyn[all]"`), you can use Pydantic `BaseModel` classes directly as handler parameter annotations. Robyn will

automatically validate the request body **and** generate a rich OpenAPI schema — including property types, required fields, defaults, and \$ref for nested models.

## Pydantic + OpenAPI

```
from pydantic import BaseModel

class UserCreate(BaseModel):
 name: str
 email: str
 age: int
 active: bool = True

@app.post("/users", openapi_tags=
["Users"])
def create_user(user: UserCreate) ->
dict:
 """Create a new user"""
 return {"name": user.name}
```For the full guide on Pydantic
validation, nested models, error
responses, and OpenAPI integration, see
```

the dedicated [Pydantic Integration] (/documentation/en/api_reference/pydantic/ page).

```
## [Documenting routes (status codes,
deprecation, extra responses)]
(/documentation/en/api_reference/openapi-
routes-status-codes-deprecation-extra-
responses)
```

Every route decorator (`@app.get``, `@app.post``, ... and the `SubRouter`` equivalents) accepts a set of flags so you can document – and in some cases change the runtime behaviour of – a route directly from the decorator:

- * `status_code`` – the default success status code. It is reflected in the spec (e.g. `201`` instead of `200``)
- **and**** applied at runtime to plain returns (dict/list/str/bytes). An explicit `Response(...)`` return always keeps its own status.
- * `response_model`` – a type (typically a Pydantic model) used as the success

response schema. When the handler returns a dict, it is validated and re-serialized through the model so the wire response matches the docs.

- * ``responses`` – additional documented responses keyed by status code. Each value may be a plain description string, a ``{"description": ..., "model": SomeType}`` dict, or a full OpenAPI response object.

- * ``deprecated=True`` – marks the operation deprecated (rendered with strikethrough in Swagger UI).

- * ``include_in_schema=False`` – hides the route from the spec entirely (useful for health checks and internal endpoints).

Route documentation flags

```
from robyn.types import  
JSONResponse
```

```
class UserResponse(JSONResponse):  
    id: int name: str
```

```
class ErrorResponse(JSONResponse):  
    message: str
```

```
@app.post( "/users", status_code=201,  
response_model=UserResponse,  
responses={ 404: "User not found",  
422: {"description": "Validation  
error", "model": ErrorResponse}, },  
openapi_tags=["Users"], ) def  
create_user(body: UserResponse) ->  
UserResponse: """Create a user  
(responds with 201)""" return {"id": 1,  
"name": "Bruce"}
```

```
@app.get("/legacy", deprecated=True)  
def legacy(): """Old endpoint kept for  
backwards-compat""" return "use  
/users instead"
```

```
@app.get("/healthz",  
include_in_schema=False) def  
healthz(): return "ok" ``##
```

Authentication & the Swagger “Authorize” button

When you call

`app.configure_authentication(...)`,
Robyn automatically registers a
matching security scheme (BearerAuth
for a BearerGetter) so Swagger UI's
Authorize button works out of the
box. Routes declared with
`auth_required=True` advertise that
requirement in the spec, so they
render with a lock icon and send the
credential when you try them out.

You can also declare schemes
explicitly via

`Components(securitySchemes=...):`

Security schemes

```
from robyn.openapi import OpenAPI,
OpenAPIInfo, Components

app = Robyn(
    __file__,
    openapi=OpenAPI(
        info=OpenAPIInfo(
            components=Components(
                securitySchemes={
                    "BearerAuth":
{"type": "http", "scheme": "bearer",
"bearerFormat": "JWT"},
                }
            )
        )
    ),
)

@app.get("/me", auth_required=True)
def me(request):
    """Requires a Bearer token – shows
a lock in Swagger UI"""
    return "secret"
```


` `` `If no security scheme is configured, Robyn no longer emits an empty `securitySchemes` object, so the ****Authorize**** button stays hidden instead of opening an empty popup.

[A note on auto-generated parameters]
(/documentation/en/api_reference/openapi-note-on-auto-generated-parameters)

Robyn builds the OpenAPI spec by ****inspecting your handler's type annotations**** – not its body. Reading values dynamically via `request.query\_params`, `request.json()`, or `request.path\_params` produces working endpoints, but Robyn has nothing to introspect, so those parameters won't appear in `/docs`.

To document them, annotate the handler with typed params instead:

* Query params: a subclass of

```
`QueryParams` (e.g. `query_params:
MyQueryParams`)
* Request body: a subclass of
`Body`/`JsonBody`, a `TypedDict`, or a
Pydantic `BaseModel`
* Path params: derived automatically
from the route string (`/users/:id`)
```

Stdlib return/field types such as
`datetime`, `date`, `UUID`, `Decimal`,
`Enum`, and `Literal` are mapped to
their proper JSON Schema
`type`/`format`, and container types
like `list[str]` and `Optional[str]`
render correctly.

```
## [What's next?]
(/documentation/en/api_reference/openapi:
next)
```

Batman wondered about whether Robyn
handlers can be dispatched to multiple
processes.

Robyn showed him the way!

```
[Multiprocess Execution]
(/documentation/en/api_reference/multipi
```

```
---
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
## [Pydantic Integration]
(/documentation/en/api_reference/pydant:
integration)
```

Robyn supports [Pydantic] (<https://docs.pydantic.dev/>) v2 as an optional dependency for automatic request body validation and rich OpenAPI schema generation. Validation is ****opt-in per handler**** – it only activates when you annotate a parameter with a Pydantic ``BaseModel``. Handlers without Pydantic annotations are completely unaffected: no parsing, no validation, no overhead. When Pydantic is not installed at all, Robyn never imports it.

```
## [Installation]
(/documentation/en/api_reference/pydant:
```

Install Robyn with Pydantic support
using the optional extra:

```
### Installation
```

Pydantic onlyAll extrasconda

```
pip install "robyn[pydantic]"
```robyn[all] includes Pydantic, Jinja2
templating, and any future optional
features.
```

## **Basic Usage**

Define a Pydantic BaseModel and use it  
as a type annotation on your handler  
parameter. Robyn will automatically  
parse the incoming JSON body,

validate it against the model, and inject the validated instance into your handler.

## Basic Pydantic Validation

### SynchronousAsynchronous

```
from pydantic import BaseModel
from robyn import Robyn
```

```
app = Robyn(__file__)
```

```
class UserCreate(BaseModel):
 name: str
 email: str
 age: int
 active: bool = True
```

```
@app.post("/users")
def create_user(user: UserCreate):
 """Create a new user"""
 return {
```

```
 "name": user.name,
 "email": user.email,
 "age": user.age,
 "active": user.active,
}
```

```
if __name__ == "__main__":
 app.start()
````## [Validation Errors]  
(/documentation/en/api_reference/pydant:  
errors)
```

When the request body fails validation, Robyn automatically returns a ****422 Unprocessable Entity**** response with structured error details. You do not need to write any error handling code.

For example, sending ``{"name": "Alice", "email": "alice@example.com", "age": "not_a_number"}`` would produce:

```
{ "error": "Validation Error", "detail":  
[ { "type": "int_parsing", "loc": ["age"],
```

“msg”: “Input should be a valid integer, unable to parse string as an integer”, “input”: “not_a_number” }] }

```Missing required fields are also caught:

```
{
 "error": "Validation Error",
 "detail": [
 {
 "type": "missing",
 "loc": ["email"],
 "msg": "Field required",
 "input": {"name": "Alice", "age":
30}
 }
]
}
```

```## [Nested Models]  
(/documentation/en/api_reference/pydantic:models)

Pydantic models can reference other models. Robyn handles nested validation automatically.

```
### Nested Models
```

```
from pydantic import BaseModel  
from robyn import Robyn
```

```
app = Robyn(file)
```

```
class Address(BaseModel): street: str  
city: str zip_code: str
```

```
class UserWithAddress(BaseModel):  
name: str email: str address: Address
```

```
@app.post("/users") def  
create_user(data: UserWithAddress):  
    """Create a user with an address"""  
    return {"name": data.name, "city":  
data.address.city} ``If the  
nestedaddressobject is missing or  
malformed, Robyn returns a 422 with the
```


full error path (e.g. [“address”, “city”]).

Using with the Request Object

You can combine Pydantic parameters with the standard `Request` object in the same handler. This gives you access to headers, query params, and other request metadata alongside the validated body.

Pydantic + Request

```
from pydantic import BaseModel
from robyn import Robyn, Request

app = Robyn(__file__)
```

```
class UserCreate(BaseModel):
    name: str
    email: str
    age: int
    active: bool = True
```

```
@app.post("/users")
def create_user(request: Request, user:
UserCreate):
    """Create a user – access both raw
request and validated model"""
    return {
        "method": request.method,
        "name": user.name,
        "email": user.email,
    }
```## [Returning Pydantic Models
Directly]
(/documentation/en/api_reference/pydant:
pydantic-models-directly)
```

You can return a Pydantic model instance (or a list of them) directly from a handler. Robyn will automatically serialize it to JSON with

the correct ``Content-Type`` header – no need to call ``.model_dump()`` manually.

### Returning Models

Single `modelList` of models

```
@app.post("/users") def
create_user(user: UserCreate) ->
UserCreate: """Validate and echo back
the user""" return user`Both forms
produce an application/json response.
The single-model path uses Pydantic's
Rust-based model_dump_json()` for
maximum throughput.
```

## How Validation Is Triggered

Pydantic validation is **annotation-driven, not method-driven**. The router inspects each handler's

signature at registration time; any parameter annotated with a `BaseModel` subclass triggers automatic validation of `request.body` when that route is called. This works with every HTTP method — POST, PUT, PATCH, DELETE, or any other method that carries a body.

## Any HTTP Method

### PUTPATCH

```
@app.put("/users/:id")
def update_user(user: UserCreate):
 return {"updated": True, "name":
user.name}
```## [OpenAPI Integration]
(/documentation/en/api_reference/pydant:
integration)
```

When you use Pydantic models, Robyn automatically generates rich JSON Schema in your OpenAPI specification at `/openapi.json`. This includes:

```
* **Property types** – `string`,  
`integer`, `boolean`, etc.  
* **Required fields** – fields without  
defaults are listed in `required`  
* **Default values** – shown in the  
schema  
* **Nested models** – referenced via  
`$ref` and placed in  
`components/schemas`
```

```
### OpenAPI with Pydantic
```

```
from pydantic import BaseModel  
from robyn import Robyn, Request
```

```
app = Robyn(file)
```

```
class Address(BaseModel): street: str  
city: str zip_code: str
```

```
class UserWithAddress(BaseModel):  
name: str email: str address: Address
```

```
@app.post("/users", openapi_tags=[
    "Users"]) def create_user(request:
    Request, data: UserWithAddress) ->
    dict: """Create a user with a nested
    address""" return {"name":
    data.name, "city": data.address.city}
`The generated/openapi.json` will
contain:
```

```
{
  "paths": {
    "/users": {
      "post": {
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "type": "object",
                "properties": {
                  "name": {"type":
"string", "title": "Name"},
                  "email": {"type":
"string", "title": "Email"},
                  "address": {"$ref":
```

```

"#/components/schemas/Address"}
    },
    "required": ["name",
"email", "address"],
    "title":
"UserWithAddress"
    }
  }
}
}
},
},
"components": {
  "schemas": {
    "Address": {
      "type": "object",
      "properties": {
        "street": {"type": "string",
"title": "Street"},
        "city": {"type": "string",
"title": "City"},
        "zip_code": {"type":
"string", "title": "Zip Code"}
      },
      "required": ["street", "city",

```

```

"zip_code"],
    "title": "Address"
}
}
}
}
}
}
` `` ## [Pydantic vs Body]
(/documentation/en/api_reference/pydantic:
vs-body)

```

Robyn supports two approaches for typed request bodies. Choose the one that fits your needs:

| Feature | `Body` subclass | Pydantic |
|----------------------------|------------------------------|--------------------------------------|
| | `BaseModel` | |
| --- | --- | --- |
| Installation | Built-in | `pip install "robyn[pydantic]"` |
| Validation | No automatic validation | Full validation with detailed errors |
| Error responses | Manual | Automatic |
| 422 with structured errors | | |
| Return serialization | Manual | |
| `dict()` | Auto-serialize model to JSON | |

| | | |
|--|---|---|
| | | |
| | OpenAPI schema | Basic type inference |
| | Full JSON Schema (types, required, defaults, ` \$ref `) | |
| | Nested models | Supported (basic) |
| | Supported (with ` \$ref ` in OpenAPI) | |
| | Performance overhead | None Only when Pydantic is installed and used |

Both approaches work with OpenAPI documentation. If you need validation, use Pydantic. If you just need OpenAPI schema hints without validation, `Body` is sufficient.

[Important Notes]

(/documentation/en/api_reference/pydantic: notes)

*** **Opt-in per handler**** – Validation only runs on handlers where a parameter is annotated with a Pydantic `BaseModel`. All other handlers (using `Body`, `Request`, path params, etc.) behave exactly as before with zero additional overhead.

* **One Pydantic body per handler** – Each handler can have at most one parameter annotated with a Pydantic model. The entire request body is parsed into that single model. If you need multiple model inputs, compose them into a single parent model with nested fields.

* **Request validation only** – Robyn validates *incoming* request bodies against Pydantic models but does not validate *outgoing* responses. When you return a model instance, it is serialized as-is without re-validation. This is a deliberate design choice for performance – if you constructed the model, it's already valid.

[What's next?]

(/documentation/en/api_reference/pydant:next)

Batman wondered about whether Robyn handlers can be dispatched to multiple processes.

Robyn showed him the way!

```
[Multiprocess Execution]
(/documentation/en/api_reference/multi
```

```
---
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
## [Redirection]
(/documentation/en/api_reference/redirec
```

Batman wanted to redirect some endpoints to others. Robyn helped him do so by the following:

```
### Request
```

```
from robyn import Robyn, Response
app = Robyn(file)
```

```
@app.get("/") async def index():
return Response( status_code=307,
```

```
description=““, headers={”Location”:  
“landing”}, )
```

```
@app.get(“/landing”) def landing():  
return “hii!” ``## What’s next?
```

Now, Batman wanted to have the ability to upload files to the server if any new villain appeared. Robyn introduced him to the file upload and some of the form data features.

- File Uploads
-

Request Object

The request object is a dataclass that contains all the information about the request. It is available in the route handler as the first argument.

The request object is created in Rust side but is exposed to Python as a dataclass.

- Attributes:
- `query_params (QueryParams)`: The query parameters of the request. e.g. `/user?id=123 -> {"id": [ "123" ]}`
- `headers (dict[str, str])`: The headers of the request. e.g. `{"Content-Type": "application/json"}`
- `params (dict[str, str])`: The parameters of the request. e.g. `/user/:id -> {"id": "123"}`
- `body (Union[str, bytes])`: The raw body of the request. For JSON payloads, use the `json()` method to parse the body into a dict with proper type preservation.

- `method (str)`: The method of the request. e.g. `GET`, `POST`, `PUT`, `DELETE`
- `ip_addr (Optional[str])`: The IP Address of the client
- `identity (Optional[Identity])`: The identity of the client

Request

`GET`

`/hello_world`

`@dataclass`

`class Request:`

`"""`

`query_params: QueryParams`

`headers: Headers`

`path_params: dict[str, str]`

`body: Union[str, bytes]`

`method: str`

`url: Url`

```
form_data: dict[str, str]
files: dict[str, bytes]
ip_addr: Optional[str]
identity: Optional[Identity]
"""
```

```
```## [Parsing JSON Body]
(/documentation/en/api_reference/request
json-body)
```

The `request.json()` method parses the request body as JSON and returns a Python `dict` with full type preservation:

- \* JSON `null` becomes Python `None`
- \* JSON numbers become Python `int` or `float`
- \* JSON booleans become Python `bool`
- \* JSON strings become Python `str`
- \* JSON arrays become Python `list`
- \* JSON objects become Python `dict`

Nested structures are handled recursively up to a maximum depth of 128 levels.

### Parsing JSON

POST

/example

```
@app.post("/example") async def
handler(request: Request): data =
request.json() # Returns a dict with
preserved types # e.g. {"count": 42,
"active": true, "tags": ["a", "b"]} # ->
{"count": 42, "active": True, "tags":
["a", "b"]} return {"received": data}
`If the body is not valid JSON or is
not a JSON object, a ValueError` will
be raised.
```

## **Extra Path Parameters**

Robyn supports capturing extra path parameters using the `*extra` syntax in



route definitions. This allows you to capture any additional segments in the URL path that come after the defined route.

For example, if you define a route like this:

```
@app.get("/sync/extra/*extra")
def sync_param_extra(request: Request):
 extra =
request.path_params["extra"]
 return extra
```

Any additional path segments after `/sync/extra/`` will be captured in the `extra`` parameter. For instance:

\* A request to `/sync/extra/foo/bar`` would result in `extra = "foo/bar"``

\* A request to `/sync/extra/123/456/789`` would result in `extra = "123/456/789"``

You can access the extra path parameters through

``request.path_params["extra"]`` in your route handler.

This feature is particularly useful when you need to handle dynamic, nested routes or when you want to capture an unknown number of path segments.

## [Easy Access Parameters]  
([/documentation/en/api\\_reference/request-access-parameters](/documentation/en/api_reference/request-access-parameters))

Instead of manually extracting and converting query parameters and path parameters from the request object, you can declare them directly in your function signature with type annotations. Robyn will automatically resolve and coerce them for you.

Any handler parameter that doesn't match a known request component (``Request``, ``QueryParams``, ``Headers``, etc.) is treated as an individual path or query parameter.

**\*\*Basic usage\*\*** — path params and query params with type coercion and defaults.

### Easy Access Params

Typed ParamsMixed with Request

GET

/items/:id?q=hello&page=5

```
@app.get("/items/:id") async def
get_item(id: int, q: str, page: int = 1): #
id is coerced from the path param
string to int # q is taken from ?q=... #
page defaults to 1 if not provided
return {"id": id, "q": q, "page": page}
```

**Optional, List, Bool, and Float  
params** — Robyn handles common  
Python types automatically.

- `Optional[T]` — resolves to `None` when not provided

- `List[T]` — collects repeated query params (e.g. `?tag=a&tag=b`)
- `bool` — accepts `true/false`, `1/0`, `yes/no`, `on/off`
- `float` — standard float coercion

## Advanced Types

### OptionalListBool & Float

GET

/search

```
@app.get("/search")
def search(name: str, age:
Optional[int] = None):
 return {"name": name, "age": age}
GET /search?name=bob ->
{"name": "bob", "age": null}
GET /search?name=bob&age=30 ->
{"name": "bob", "age": 30}
```Error handling``` — if a required
```

parameter is missing or a value cannot be coerced to the declared type, Robyn returns a `400 Bad Request` response automatically.

Validation Errors

GET

/items/:id

```
@app.get("/items/:id") def get_item(id: int, q: str): return {"id": id, "q": q}
```

GET /items/42 -> 400
(missing required 'q')

**GET /items/abc?q=test -
> 400 (cannot coerce
'abc' to int)**

```## Type Aliases`

Robyn ships convenience type aliases that describe the runtime types of the request components, so you can annotate handler code and have it checked by `mypy/pyright`. They are importable from both `robyn` and `robyn.types`.

- `RequestMethod: Literal["GET", "POST", "PUT", "DELETE", "PATCH", "HEAD", "OPTIONS", "CONNECT", "TRACE"]` — the value of `request.method`.
- `RequestBody: str | bytes` — the type of `request.body` (text for UTF-8 payloads, raw bytes otherwise).
- `RequestURL: alias of url` — the type of `request.url`.

Type Aliases

GET

/typed

```
from robyn import Robyn, Request
from robyn import RequestMethod,
RequestBody, RequestURL
```

```
app = Robyn(__file__)
```



```

@app.get("/typed")
def handler(request: Request):
    method: RequestMethod =
request.method
    body: RequestBody = request.body
    url: RequestURL = request.url
    return {"method": method}
```## [What's next?]
(/documentation/en/api_reference/request
next)

```

Now, Batman wanted to understand the configuration of the Robyn server. He was then introduced to the concept of Robyn env files.

```

* [Robyn Env]
(/documentation/en/api_reference/robyn_ε

<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
objects -->

```

```
[Response Return Styles]
(/documentation/en/api_reference/responses/objects#response-return-styles)
```

Robyn automatically converts your handler's return value into a proper HTTP response. Here's a complete reference of every supported return style.

text/plain

```
[String]
(/documentation/en/api_reference/responses/objects#string)
```

The simplest return style. Returns the string as `text/plain` with status 200. The string is encoded as UTF-8 automatically.

```
Response
```

```
GET
```

```
/hello
```

```
from robyn import Robyn
```

```
app = Robyn(file)
```

```
@app.get("/hello") def hello(): return
"Hello, World!" ``## Dictionary or List
```

Returning a `dict` or `list` automatically serializes the value to JSON and sets `Content-Type: application/json` with status 200.

## Response

DictionaryList

GET

/data

```
@app.get("/user")
def get_user():
 return {"name": "Alice", "age": 30}
```

````## [Response Object]`  
(/documentation/en/api_reference/responses/objects#response-object)

The ``Response`` object gives you full control over status code, headers, and body. Use it when you need to customize everything about the response.

The response body can be passed as either ``body`` (the preferred, intuitively-named argument) or ``description`` (kept for backwards compatibility). The two are aliases – pass exactly one. ``headers`` is optional and defaults to an empty ``Headers`` object.

* ``status_code`` (int): The HTTP status code, e.g. ``200``.

* ``headers`` (Headers | dict | None): Response headers. Accepts a ``Headers`` instance, a plain ``dict``, or ``None`` (empty).

* ``body`` (str | bytes): The response body. Alias for ``description``.

```
* `description` (str | bytes): The  
response body (legacy name; prefer  
`body`).
```

```
### Response
```

```
body (preferred)description (legacy)
```

```
GET
```

```
/custom
```

```
from robyn import Robyn, Response,  
Headers
```

```
app = Robyn(file)
```

```
@app.get("/custom") async def  
custom_response(): return Response(  
status_code=200,  
headers=Headers({"X-Custom":  
"value"}), body="OK", ) ``## Bytes
```

Returning a bytes object sets Content-Type: application/octet-stream with status 200. Useful for binary data such as images or file content generated in memory.

Response

GET

/binary

```
@app.get("/binary")
async def binary_data():
    return b"binary data"
``` ## [Pydantic BaseModel]
(/documentation/en/api_reference/responses#pydantic-basemodel)
```

If you return a Pydantic `BaseModel` instance, Robyn serializes it to JSON automatically with `Content-Type: application/json` and status 200.

**\*\*Caveat:\*\*** Pydantic must be installed as an optional dependency (``pip install pydantic``). If it is not installed, the model will not be detected and will fall through to the default string serialization.

### Response

GET

/model

```
from pydantic import BaseModel
from robyn import Robyn
```

```
app = Robyn(file)
```

```
class User(BaseModel):
 name: str
 email: str
 age: int
```

```
@app.get("/model")
def get_model():
 return User(name="Alice",
 email="alice@example.com", age=30)
```

```
``## Tuple (body, headers,
status_code).
```

A 3-element tuple of (body, headers, status\_code) lets you set a custom status code and headers inline. The body element is itself formatted using the same rules (string, dict, bytes, etc.).

**Caveat:** The tuple must have exactly 3 elements. Passing a tuple with a different number of elements raises a `ValueError`.

## Response

Error responseCreated response

POST

/create



```

from robyn import Headers

@app.get("/not-found")
def not_found():
 return (
 {"error": "Resource not
found"},
 Headers({"X-Error": "true"}),
 404,
)
```
\[FileResponse / serve\\_file / serve\\_html\]
\(/documentation/en/api\\_reference/responses/objects#fileresponse--serve\\_file--serve\\_html\)

```

Robyn provides helper functions for serving files. `serve_file` sets `Content-Disposition: attachment` and auto-detects the MIME type. `serve_html` sets `Content-Type: text/html`. You can also construct a `FileResponse` directly for full control.

Response

```
serve\_fileserve\_htmlFileResponse
```

```
GET
```

```
/download
```

```
from robyn import Robyn from  
robyn.responses import serve_file
```

```
app = Robyn(file)
```

```
@app.get("/download") def  
download(): return  
serve_file("report.pdf") ``## html()
```

The `html()` helper wraps a raw HTML string into a Response with Content-Type: `text/html` and status 200. Useful for returning dynamically generated HTML without a template engine.

Response

GET

/page

```
from robyn import Robyn
from robyn.responses import html

app = Robyn(__file__)

@app.get("/page")
def page():
    return html("<h1>Hello, World!</h1>
<p>Welcome to Robyn.</p>")
```## [StreamingResponse]
(/documentation/en/api_reference/responses#streamingresponse)
```

`StreamingResponse` sends chunked responses using a generator (sync or async). The default `media\_type` is `text/event-stream`, but you can set it to any MIME type. The body is streamed chunk by chunk, so the client receives data as it is produced.

```
Response
```

```
Sync generator Async generator
```

```
GET
```

```
/stream
```

```
from robyn import Robyn from
robyn.responses import
StreamingResponse
```

```
app = Robyn(file)
```

```
@app.get("/stream") def stream(): def
generate(): for i in range(5): yield
f"chunk {i}"
```

```
return StreamingResponse(
 content=generate(),
 media_type="text/plain",
)
```

## ``## SSEResponse

SSEResponse is a convenience wrapper around StreamingResponse pre-configured for Server-Sent Events. Use it with the SSEMessage helper to format messages in the SSE protocol. Each message can have optional event, id, and retry fields.

For a deeper guide on Server-Sent Events, see the SSE documentation.

## **Response**

Basic SSEAsync SSE

GET

/events

```

from robyn import Robyn
from robyn.responses import
SSEResponse, SSEMessage
import time

app = Robyn(__file__)

@app.get("/events")
def events():
 def event_stream():
 for i in range(10):
 yield SSEMessage(
 f"Event {i}",
 event="update",
 id=str(i),
)
 time.sleep(1)

 return SSEResponse(event_stream())
````## [What's next?]
(/documentation/en/api_reference/respons
objects#whats-next)

```

Now that you know every way to return data from a Robyn handler, explore file uploads to learn how to receive files

from clients.

```
* [File Uploads]
(/documentation/en/api_reference/file-
uploads)
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
## [Configuring the server through an
environment file]
(/documentation/en/api_reference/robyn_
the-server-through-an-environment-file)
```

Batman wanted to configure the server through an environment file. Changing code continuously induced the risk of error.

```
## [Environment Variables]
(/documentation/en/api_reference/robyn_
variables)
```

- * ``ROBYN_PORT``: Specifies the port on which the Robyn server will listen.
 - + Default: ``8080``
 - + Example: ``ROBYN_PORT=3000``
- * ``ROBYN_HOST``: Specifies the host address for the Robyn server.
 - + Default: ``127.0.0.1``
 - + Example: ``ROBYN_HOST=0.0.0.0``
- * ``ROBYN_BROWSER_OPEN``: Open the browser on successful start.
 - + Default: ``False``
 - + Example: ``ROBYN_BROWSER_OPEN=True``
- * ``ROBYN_DEV_MODE``: Configures the dev mode
 - + Default: ``False``
 - + Example: ``ROBYN_DEV_MODE=True``
- * ``ROBYN_MAX_PAYLOAD_SIZE``: Sets the maximum payload size for HTTP requests and WebSocket messages in bytes.
 - + Default: ``1000000`` bytes
 - + Example:
``ROBYN_MAX_PAYLOAD_SIZE=1000000``

You can have a ``robyn.env`` file to load them automatically in your environment.

These environment variables are typically set in a `robyn.env` file located at the root of the project. The server parses this file at startup to configure itself accordingly.

For more details on the structure and usage of the `robyn.env` file, refer to the documentation snippet:

```
### Sample project directory
```

```
-project/ -robyn.env -index.py ...  
``Sample robyn.env` file:
```

Sample Robyn.env

```
ROBYN_PORT=8080  
ROBYN_HOST=127.0.0.1  
RANDOM_ENV=123  
ROBYN_BROWSER_OPEN=True  
ROBYN_DEV_MODE=True  
ROBYN_MAX_PAYLOAD_SIZE=1000000  
````With the web application deployed  
and running smoothly, Batman had a
```

powerful new tool at his disposal. The Robyn framework had provided him with the flexibility, scalability, and performance needed to create an effective crime-fighting application, giving him a technological edge in his ongoing battle to protect Gotham City.

## [What's next?]

(/documentation/en/api\_reference/robyn\_what's\_next)

Batman - Thanks, Robyn. Now tell me more.

Robyn - Let us learn about the Middlewares and events now!

\* [Middlewares]

(/documentation/en/api\_reference/middlewares)

---

<!-- robyn-documentation source:

[https://robyn.tech/documentation/en/api\\_reference](https://robyn.tech/documentation/en/api_reference)

-->

## # Scaling and Production Deployment

Robyn is designed to scale efficiently from development to production. This guide covers scaling strategies, production deployment patterns, and performance optimization techniques.

## [Understanding Robyn's Scaling Model]  
(/documentation/en/api\_reference/scaling/robyns-scaling-model)

### ### Multi-Process Architecture

Robyn uses a **\*\*shared-nothing multi-process model\*\*** optimized for modern multi-core systems. Each process:

- \* Runs independently with its own Python interpreter and memory space
- \* Has its own Global Interpreter Lock (GIL), eliminating GIL contention
- \* Can handle multiple concurrent requests via worker threads
- \* Scales linearly across CPU cores for

true parallelism

- \* Provides fault isolation - one process crash doesn't affect others

### ### Worker Threads

Within each process, multiple worker threads provide concurrency:

- \* Share the same Python interpreter and memory space
- \* Are subject to the GIL for CPU-bound tasks (use more processes for CPU work)
- \* Excel at I/O-bound operations where threads can release the GIL
- \* Handle database calls, HTTP requests, file operations efficiently
- \* Provide request-level concurrency within the same process

## [Scaling Configuration]  
([/documentation/en/api\\_reference/scaling\\_configuration](/documentation/en/api_reference/scaling_configuration))

### ### Basic Scaling

**\*\*Single Process Development\*\***: Start simple during development with auto-reload enabled.

**### Development Scaling**

# **Development mode (single process, single worker)**

```
python app.py -dev
```

# Development with custom port

```
python app.py --dev --port 3000
```

**``Multi-Core Production:** Scale across all available CPU cores for maximum performance.

## Production Scaling

```
Automatic optimization (recommended)
python app.py --fast
```

```
Manual configuration for 8-core
system
python app.py --processes 8 --workers 2
```

```
I/O heavy workloads
python app.py --processes 4 --workers 4
```

```
CPU heavy workloads
python app.py --processes 8 --workers 1
```### Advanced Configuration
```

****Hardware-Specific Tuning****: Optimize based on your specific hardware and application characteristics.

```
### Advanced Tuning
```


High-traffic web API (4-core system)

```
python app.py --processes 4 --workers  
3 --log-level INFO
```

Data processing service (8-core system)

```
python app.py --processes 8 --workers  
1 --log-level WARNING
```

Mixed workload (balanced approach)

```
python app.py --processes 6 --workers  
2 --log-level INFO
```

Maximum concurrency (16-core system)

```
python app.py --processes 8 --workers  
4 ``### Configuration Guidelines
```

Choosing the Right Configuration:
Use these guidelines to optimize for
your specific use case.

Configuration Guidelines

```
# For CPU-intensive applications  
# Rule: processes = CPU cores, workers  
= 1
```

```
# Example: 8-core machine
python app.py --processes 8 --workers 1
```

```
# For I/O-intensive applications
# Rule: processes = CPU cores / 2,
workers = 2-4
# Example: 8-core machine
python app.py --processes 4 --workers 3
```

```
# For balanced applications
# Rule: processes = CPU cores / 2,
workers = 2
# Example: 8-core machine
python app.py --processes 4 --workers 2
```

```
# Memory considerations
# Each process uses ~50-100MB base
memory
# Monitor with: ps aux | grep python
```## [Production Deployment
Strategies]
(/documentation/en/api_reference/scaling
deployment-strategies)
```

```
Container Deployment
```

**\*\*Docker Containerization\*\***: Deploy Robyn applications using Docker for consistent environments and easy scaling.

### Docker Deployment

CodeCode

# Dockerfile

FROM python:3.11-slim

# Install system dependencies

```
RUN apt-get update && apt-get install -
y
build-essential
&& rm -rf /var/lib/apt/lists/*
```



# Set working directory

WORKDIR /app

# **Copy requirements and install dependencies**

`COPY requirements.txt . RUN pip  
install --no-cache-dir -r  
requirements.txt`

# Copy application code

COPY ..

# Expose port

EXPOSE 8080

# Health check

```
HEALTHCHECK --interval=30s --
timeout=3s --start-period=5s --retries=3
CMD curl -f
http://localhost:8080/health || exit 1
```

# Run application with optimized settings

CMD ["python", "app.py", "--fast", "--host", "0.0.0.0", "--port", "8080"] ``###  
Load Balancer Configuration

**Nginx Load Balancing:** Use Nginx as a reverse proxy and load balancer for multiple Robyn instances.

## Nginx Configuration

```
/etc/nginx/sites-available/robyn-app
upstream robyn_backend {
 least_conn;
 server 127.0.0.1:8080 max_fails=3
```

```
fail_timeout=30s;
 server 127.0.0.1:8081 max_fails=3
fail_timeout=30s;
 server 127.0.0.1:8082 max_fails=3
fail_timeout=30s;
 server 127.0.0.1:8083 max_fails=3
fail_timeout=30s;
}
```

```
server {
 listen 80;
 server_name yourdomain.com;

 # Security headers
 add_header X-Frame-Options DENY;
 add_header X-Content-Type-Options
nosniff;
 add_header X-XSS-Protection "1;
mode=block";

 # Gzip compression
 gzip on;
 gzip_types text/plain
application/json application/javascript
text/css;
```

```
Static files (if any)
location /static/ {
 alias /var/www/static/;
 expires 1y;
 add_header Cache-Control
"public, immutable";
}
```

```
API routes
location / {
 proxy_pass
http://robyn_backend;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP
$remote_addr;
 proxy_set_header X-Forwarded-
For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-
Proto $scheme;
```

```
Timeouts
proxy_connect_timeout 5s;
proxy_send_timeout 60s;
proxy_read_timeout 60s;
```

```
Health checks
```



```
 proxy_next_upstream error
timeout http_500 http_502 http_503;
 }
```

```
 # Health check endpoint
 location /health {
 access_log off;
 proxy_pass
http://robyn_backend;
 }
}
```

```
```### Kubernetes Deployment
```

****Kubernetes Orchestration****: Deploy and scale Robyn applications in Kubernetes clusters.

Kubernetes Manifests

deployment.yaml

```
apiVersion: apps/v1 kind: Deployment
metadata: name: robyn-app labels:
app: robyn-app spec: replicas: 3
selector: matchLabels: app: robyn-app
template: metadata: labels: app:
robyn-app spec: containers: - name:
robyn-app image: your-
registry/robyn-app:latest ports: -
containerPort: 8080 env: - name:
ROBYN_HOST value: "0.0.0.0" - name:
ROBYN_PORT value: "8080" - name:
ROBYN_LOG_LEVEL value: "INFO"
resources: requests: memory: "256Mi"
cpu: "250m" limits: memory: "512Mi"
```

cpu: "500m" livenessProbe: httpGet:
path: /health port: 8080
initialDelaySeconds: 30
periodSeconds: 10 readinessProbe:
httpGet: path: /health port: 8080
initialDelaySeconds: 5 periodSeconds:
5

apiVersion: autoscaling/v2 kind:
HorizontalPodAutoscaler metadata:
name: robyn-hpa spec:
scaleTargetRef: apiVersion: apps/v1
kind: Deployment name: robyn-app
minReplicas: 3 maxReplicas: 10
metrics: - type: Resource resource:
name: cpu target: type: Utilization
averageUtilization: 70 - type: Resource
resource: name: memory target: type:
Utilization averageUtilization: 80 ``##
Monitoring and Observability

Application Metrics

Built-in Monitoring: Add comprehensive monitoring and metrics collection to your Robyn application for production observability.

Advanced Monitoring Setup

```
from robyn import Robyn
import time
import psutil
import logging
import threading
from collections import defaultdict,
deque

app = Robyn(__file__)

# Configure structured logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %
(levelname)s - %(message)s',
```

```

        handlers=[
            logging.StreamHandler(),
            logging.FileHandler('app.log')
        ]
    )
    logger = logging.getLogger(__name__)

# Advanced metrics collection
class MetricsCollector:
    def __init__(self):
        self.request_count = 0
        self.total_response_time = 0
        self.status_codes =
defaultdict(int)
        self.endpoint_stats =
defaultdict(lambda: {"count": 0,
"total_time": 0})
        self.response_times =
deque(maxlen=1000) # Last 1000
requests
        self.lock = threading.Lock()

    def record_request(self, method,
path, status_code, duration):
        with self.lock:
            self.request_count += 1

```

```
        self.total_response_time +=
duration

self.status_codes[status_code] += 1
        endpoint_key = f"{method}
{path}"

self.endpoint_stats[endpoint_key]
["count"] += 1

self.endpoint_stats[endpoint_key]
["total_time"] += duration

self.response_times.append(duration)

    def get_metrics(self):
        with self.lock:
            avg_response_time =
self.total_response_time /
max(self.request_count, 1)
            p95_response_time =
sorted(self.response_times)
[int(len(self.response_times) * 0.95)]
if self.response_times else 0

        return {
```

```
        "requests_total":
self.request_count,
        "avg_response_time":
avg_response_time,
        "p95_response_time":
p95_response_time,
        "status_codes":
dict(self.status_codes),
        "top_endpoints":
dict(sorted(

self.endpoint_stats.items(),
        key=lambda x: x[1]

["count"],

        reverse=True

)[:10])
    }
```

```
metrics = MetricsCollector()
start_time = time.time()
```

```
@app.before_request
def monitor_request_start(request):
    request.start_time = time.time()
    return request
```

```
@app.after_request
def monitor_request_end(request,
response):
    duration = time.time() -
request.start_time
    status_code = getattr(response,
'status_code', 200)

    # Record metrics

metrics.record_request(request.method,
request.url.path, status_code,
duration)

    # Log slow requests
    if duration > 1.0:
        logger.warning(
            f"Slow request:
{request.method} {request.url.path} - "
            f"{duration:.3f}s (status:
{status_code})"
        )

    # Add response headers
    response.headers["X-Response-Time"]
= f"{duration:.3f}s"
```



```
        response.headers["X-Request-ID"] =  
str(time.time_ns())  
        return response
```

```
# Health endpoint with detailed status
```

```
@app.get("/health", const=True)
```

```
def health_check():
```

```
    return {  
        "status": "healthy",  
        "version": "1.0.0",  
        "uptime_seconds": time.time() -  
start_time  
    }
```

```
# Comprehensive metrics endpoint
```

```
@app.get("/metrics")
```

```
def get_metrics():
```

```
    cpu_percent =  
psutil.cpu_percent(interval=1)  
    memory = psutil.virtual_memory()  
    disk = psutil.disk_usage('/')
```

```
    app_metrics = metrics.get_metrics()
```

```
    return {  
        "system": {
```

```

        "cpu_usage_percent":
cpu_percent,
        "memory_usage_percent":
memory.percent,
        "memory_available_mb":
memory.available / 1024 / 1024,
        "disk_usage_percent":
disk.percent,
        "load_average":
psutil.getloadavg()
    },
    "application": app_metrics,
    "timestamp": time.time()
}

```

Readiness endpoint for k8s

```
@app.get("/ready")
```

```
def readiness_check():
```

```
    # Add your readiness checks here
```

```
(DB connection, etc.)
```

```
    return {"ready": True}
```

```
````## [Performance Optimization]
```

```
(/documentation/en/api_reference/scaling
optimization)
```

### Scaling Best Practices

1. **\*\*Start Simple\*\***: Begin with `--fast`` mode and measure performance
2. **\*\*Profile Your Application\*\***: Identify CPU vs I/O bound operations
3. **\*\*Test Different Configurations\*\***: Benchmark various process/worker combinations
4. **\*\*Monitor Resource Usage\*\***: Watch CPU, memory, and I/O utilization
5. **\*\*Scale Horizontally\*\***: Use multiple instances behind a load balancer

### ### Performance Testing

**\*\*Load Testing\*\***: Use tools like wrk, Apache Bench, or Locust to test different configurations and find optimal settings.

### ### Performance Testing

CodePython

# **Install wrk for load testing**

**macOS: brew install  
wrk**

**Ubuntu: sudo apt  
install wrk**

# Test baseline performance

```
wrk -t4 -c100 -d30s
http://localhost:8080/health
```

**Test with different  
configurations**



# Configuration 1: Single process

```
python app.py --processes 1 --workers
1 & wrk -t4 -c100 -d30s
http://localhost:8080/api/endpoint
```

# Configuration 2: Multi-process

```
python app.py --processes 4 --workers
2 & wrk -t4 -c100 -d30s
http://localhost:8080/api/endpoint
```

# Configuration 3: Fast mode

```
python app.py --fast & wrk -t4 -c100 -
d30s
http://localhost:8080/api/endpoint
```

# **Compare results and choose the best configuration**

````### Environment Variables`

Configuration via Environment: Use environment variables for deployment flexibility.

Environment Configuration

CodePython

```
# Production environment variables
export ROBYN_HOST=0.0.0.0
export ROBYN_PORT=8080
export ROBYN_LOG_LEVEL=INFO
export ROBYN_PROCESSES=4
export ROBYN_WORKERS=2

# Database configuration
export
DATABASE_URL=postgresql://user:pass@localhost:5432/robyn
export
REDIS_URL=redis://localhost:6379/0

# Application settings
export SECRET_KEY=your-secret-key
export DEBUG=false
export ENVIRONMENT=production

# Start application with environment
# config
python app.py
```## [What's next?]
(/documentation/en/api_reference/scaling/next)
next)
```

Now, Batman wanted to extend Robyn.

Robyn told him about the advanced features.

```
* [Advanced Features]
(/documentation/en/api_reference/advance
```

```

```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
Server-Sent Events (SSE)
```

After learning about [form data handling]  
(/documentation/en/api\_reference/form\_data)  
Batman realized he needed a way to push real-time updates to his crime monitoring dashboard. Criminals don't wait for Batman to refresh his browser!

He discovered Server-Sent Events (SSE)  
- a perfect solution for one-way communication from server to client over HTTP. SSE allows Batman to stream

live data to his dashboard without the complexity of full bidirectional communication.

"This is exactly what I need for my crime alerts!" Batman exclaimed. "I can push updates to the dashboard instantly when new crimes are detected."

Server-Sent Events are ideal for:

- \* Real-time notifications
- \* Live data feeds
- \* Progress updates
- \* Chat applications (server-to-client only)
- \* Dashboard updates
- \* Log streaming

## [How does it work?]  
(/documentation/en/api\_reference/server\_does-it-work)

Batman can create Server-Sent Events streams by using the ``SSEResponse`` and ``SSEMessage`` classes. He can use both

regular generators and async generators depending on his needs:

\* **Regular generators**: Perfect for simple data streams or when working with synchronous operations

\* **Async generators**: Ideal when Batman needs to perform async operations like database queries or API calls within the stream

### SSE Response

Basic SSE StreamJSON Data StreamAsync Generator Stream

GET

/events

```
from robyn import Robyn,
SSEResponse, SSEMessage import
time
```

```
app = Robyn(file)
```



```
@app.get("/events") def
stream_events(request): def
event_generator(): for i in range(10):
yield SSEMessage(f"Event {i}",
id=str(i)) time.sleep(1)

return SSEResponse(event_generator())
```

## ``## Async Generators

When Batman needs to perform async operations during his SSE streams - like fetching data from databases or making API calls - he uses async generators with `async def` and `await`. This allows him to handle multiple streams concurrently without blocking other operations.

The key difference is using `async def` for the generator function and `await`

for async operations inside the generator:

## **Advanced Async SSE**

### Database StreamAPI Integration Stream

GET

/events/database

```
from robyn import Robyn, SSEResponse,
SSEMessage
import asyncio
import json
import time

app = Robyn(__file__)

@app.get("/events/database")
async def
 stream_database_events(request):
 async def
```

```

database_event_generator():
 for i in range(10):
 # Simulate async database
query
 await asyncio.sleep(0.3)

 # Simulate fetching data
from database
 data = {
 "crime_id": i,
 "location": f"Gotham
District {i}",
 "severity": "high" if i
% 2 == 0 else "low",
 "timestamp":
time.time()
 }

 yield SSEMessage(
 json.dumps(data),
 event="crime_alert",
 id=str(i)
)

 return
SSEResponse(database_event_generator())

```

```
` `` ## [Streaming raw bytes (binary data
& file downloads)]
(/documentation/en/api_reference/server_
raw-bytes-binary-data--file-downloads)
```

Not everything Batman streams is an SSE event. To stream **arbitrary bytes** – a large file, a generated archive, an `application/octet-stream` body – he uses `StreamingResponse` directly and yields chunks, without loading the whole payload into memory. Each chunk may be `bytes` (sent as-is) or `str` (UTF-8 encoded).

A sync generator yielding `bytes` chunks, served as a binary download:

```
Request
```

```
GET
```

```
/download
```

```
from robyn import Robyn,
StreamingResponse, Headers
```

```
app = Robyn(file)
```

```
@app.get("/download") def
download(request): def file_chunks():
with open("large_file.bin", "rb") as f:
while chunk := f.read(8192): yield
chunk # each chunk is bytes
```

```
return StreamingResponse(
 file_chunks(),
 media_type="application/octet-
stream",
 headers=Headers({
 "Content-Type":
"application/octet-stream",
 "Content-Disposition":
"attachment; filename=large_file.bin",
 }),
)
```

``StreamingResponse accepts both sync and async generators. When you use an **async generator**, it runs on the same event loop as your handler, so you can safely `await` async resources — an async database session, an HTTP client — **inside** the generator:

## Request

GET

/export

```
@app.get("/export")
async def export(request):
 async def rows():
 async with AsyncSessionLocal()
as session:
 result = await
session.execute(select(Record)) #
await works here
 for row in
```

```
result.scalars():
 yield f"{row.id},
{row.name}\n".encode()

 return StreamingResponse(rows(),
media_type="text/csv")
````## [What's next?]
(/documentation/en/api_reference/server_
next)
```

Batman has mastered Server-Sent Events and can now stream real-time updates to his crime dashboard. While SSE is perfect for one-way communication from server to client, Batman realizes he needs bidirectional communication for more interactive features like real-time chat with his allies.

Next, he wants to explore how to handle bidirectional communication with [WebSockets]
(/documentation/en/api_reference/websockl
for more interactive features.

If Batman needs to handle unexpected

situations, he'll learn about
[Exception handling]
(/documentation/en/api_reference/except:
to make his applications more robust.

For scaling his crime monitoring system
across multiple processes, Batman will
explore [Scaling the Application]
(/documentation/en/api_reference/scaling

<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->

Once the Gotham Police Department could
authenticate, Batman wanted each
officer's dashboard to remember small
bits of state between requests – the
theme they picked, a CSRF token, the
last case they viewed – without
standing up a database for it. So he
reached for Robyn's sessions.

[Sessions]

(/documentation/en/api_reference/session

Robyn provides signed-cookie sessions. The session is a small dictionary that is stored on the client inside a tamper-proof cookie (an HMAC-SHA256 signature over the data). No server-side store is required, and the signature means a client cannot modify the contents without invalidating it.

****Note:** the session is signed, not encrypted. The data is encoded, so the client can read it – never store secrets (passwords, raw tokens) in the session.**

Enable sessions once on your app with ``configure_sessions``, then read or write the session inside any handler through ``request.session`` – exactly like ``request.identity`` for authentication.

Call ``configure_sessions`` with a ``secret_key``. Keep this key secret and

stable across restarts and across all your worker processes – it is what signs and verifies every session cookie.

Setup

```
from robyn import Robyn
```

```
app = Robyn(file)
```

```
app.configure_sessions(secret_key="keep-me-secret")
```

Inside a handler, `request.session` is a dictionary-like `Session` object. Read it like `adict`, and mutate it like `adict`. Robyn writes the updated session back to the response cookie automatically – but only when the session was actually modified, so read-only requests do not send a `Set-Cookie`.

Request

GET

/visits

```
@app.get("/visits")
def visits(request):
    request.session["count"] =
request.session.get("count", 0) + 1
    return f"You have visited
{request.session['count']} times"
```To log a user out, or otherwise drop
their state, clear the session.
Emptying the session expires the cookie
in the browser.
```

### Request

POST

/logout

```
@app.post("/logout") def
logout(request):
```

`request.session.clear()` return “logged out” ``## Configuration

`configure_sessions` accepts the following keyword arguments to control the cookie:

- `secret_key` (required) — key used to sign the cookie.
- `cookie_name` — name of the session cookie. Defaults to "session".
- `max_age` — session lifetime in seconds, also used as the cookie Max-Age. Defaults to 14 days. Pass `None` for a cookie that lasts only for the browser session.
- `path` — cookie Path. Defaults to "/".
- `domain` — cookie Domain. Defaults to `None`.
- `secure` — only send the cookie over HTTPS. Defaults to `False`; set it to

True in production.

- `http_only` — hide the cookie from JavaScript. Defaults to `True`.
- `same_site` — "Strict", "Lax", or "None". Defaults to "Lax".

## How it works

Under the hood, `configure_sessions` registers a global `before_request` middleware that loads and verifies the cookie into a `session` and attaches it to `request.session`, and a global `after_request` middleware that signs and writes the session back to the response when it has been modified. Because `request.session` is the same object across the `before_request`, `handler`, and `after_request` phases,

your in-handler changes are exactly what gets written back.

If you want explicit control instead of the automatic middleware, you can use the `SessionManager` directly with `manager.load(request)` and `manager.save(session, response)`.

---

## **Static Files and Directory** **Serving**

Robyn provides flexible static file serving powered by Rust's `actix-files` for high performance. You can serve individual files, entire directories, or SPA build folders.

```
app.serve_directory
```

# Serving Directories

Use `app.serve_directory()` to serve an entire directory of static files. This is commonly used for serving frontend build outputs (React, Vue, etc.) or static asset folders.

The method supports three modes:

- **SPA/Build mode:** Set `index_file` to serve a build folder with an index
- **File listing mode:** Set `show_files_listing=True` for browsable directory listings
- **File-only mode:** Default — serves files directly without directory browsing

# Directory Serving

## STATIC

/static

```
import os
from robyn import Robyn

app = Robyn(__file__)

Mode 1: SPA / Build folder (e.g.,
React, Vue)
app.serve_directory(
 route="/",

 directory_path=os.path.join(os.path.dirname(__file__),
 "build"),
 index_file="index.html",
)

Mode 2: Browsable file listing
app.serve_directory(
 route="/files",
 directory_path="./uploads",
 show_files_listing=True,
```



```
)
```

```
Mode 3: Direct file serving (no
listing, no index)
app.serve_directory(
 route="/assets",
 directory_path="./static",
)
```

```
app.start(port=8080)
```### Parameters
```

| Parameter | Type | Default | Description |
|----------------------|--------|----------|---|
| --- | --- | --- | --- |
| `route` | `str` | required | URL path prefix for the directory |
| `directory_path` | `str` | required | Filesystem path to the directory |
| `index_file` | `str` | `None` | Index file to serve for directory requests (e.g., `index.html`) |
| `show_files_listing` | `bool` | | Enable browsable directory listing |

```
## [Serving Individual Files]
(/documentation/en/api_reference/static.
files#serving-individual-files)
```

Use ``serve_file()`` to serve a specific file from a route handler. The function automatically detects the MIME type based on the file extension and sets appropriate headers.

```
### File Serving
```

```
GET
```

```
/download
```

```
from robyn import Robyn, Request,
serve_file
```

```
app = Robyn(file)
```

```
@app.get("/download/report") async
def download_report(request:
Request): return serve_file(
```

```
“./reports/annual.pdf”,  
file_name=“annual-report.pdf” #  
optional, defaults to basename )
```

```
app.start(port=8080) ``## Serving  
HTML Files
```

Use `serve_html()` to serve an HTML file with the correct `text/html` content type.

HTML Serving

GET

/

```
from robyn import Robyn, Request,  
serve_html
```

```
app = Robyn(__file__)
```

```
@app.get("/")
```

```
async def index(request: Request):
    return
    serve_html("./templates/index.html")

app.start(port=8080)
```## [Serving HTML Strings]
(/documentation/en/api_reference/static_files#serving-html-strings)
```

Use the `html()` helper to return an HTML string directly with the correct content type, without reading from a file.

```
HTML String
```

```
GET
```

```
/page
```

```
from robyn import Robyn, Request,
html
```

```
app = Robyn(file)
```



```
@app.get("/page") async def
dynamic_page(request: Request):
return html("
```

# Hello from Robyn

This is a dynamic page.

“)

`app.start(port=8080) ``## Combining Static Files with API Routes`

You can serve static files alongside API routes in the same application. This is common when building a full-stack app with a frontend SPA and a backend API.

# Combined

FULL

/app

```
import os
from robyn import Robyn, Request

app = Robyn(__file__)

Serve the React/Vue build
app.serve_directory(
 route="/",

 directory_path=os.path.join(os.path.dirname(__file__),
 "frontend/build"),
 index_file="index.html",
)

API routes work alongside static
serving
@app.get("/api/health")
def health():
 return {"status": "ok"}
```

```

@app.get("/api/users")
async def get_users(request: Request):
 return [{"id": 1, "name": "Alice"}]

app.start(port=8080)
```
## [What's next?]
(/documentation/en/api_reference/static_files#whats-next)

* [File Uploads]
(/documentation/en/api_reference/file-uploads)
* [Form Data]
(/documentation/en/api_reference/form_data)

---

<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->

## [Templating.]
(/documentation/en/api_reference/templating)

```

Batman wanted to quickly render html pages on the website. He wanted to use

a templating engine to render the html pages. Robyn told him that he can use the Jinja2 templating engine to render the html pages. He can use the `JinjaTemplate` class to render the html pages.

Batman was excited to learn that he could add events as functions as well as decorators.

```
### Request
```

```
GET
```

```
/hello\_world
```

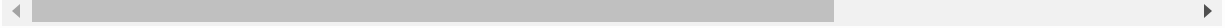
```
from robyn.templating import  
JinjaTemplate
```

```
current_file_path =  
pathlib.Path(file).parent.resolve()  
JINJA_TEMPLATE =
```

```
JinjaTemplate(os.path.join(current_file_path, "templates"))
```

```
@app.get("/template_render") def  
template_render(): context =  
{ "framework": "Robyn",  
  "templating_engine": "Jinja2" }
```

```
template =  
JINJA_TEMPLATE.render_template(template_  
**context)  
return template
```



``test.html file

Request

GET

/hello_world

```
<!DOCTYPE html>  
<html lang="en">
```

```
<head>
  <meta charset="utf-8">
  <title>Results</title>
</head>

<body>
  Hello {{ framework }}! You're using
  {{ templating_engine }}.
</body>
```## [A quick `render` shortcut]
(/documentation/en/api_reference/templating/quick-render-shortcut)
```

Batman didn't always want to construct a `JinjaTemplate` by hand. Robyn told him that for the common case – a `templates` folder next to his application file – he can use the `render` shortcut instead.

`render` resolves the `templates` directory relative to the file that calls it, so there is no need to wire up paths with `pathlib`. It renders with Jinja2 by default; pass a different `templates\_dir` to point at

another folder, or a custom  
`TemplateInterface` via  
`template\_engine` to use a different  
templating engine.

### Request

GET

/template\\_render

from robyn.templating import render

```
@app.get("/template_render") def
template_render(): # Jinja2 is the
default templating engine return
render("index.html",
name="Batman") ``## Supporting
Custom Templating Engines
```

Batman was also super excited to  
know that Robyn allows the support  
of custom templating engines.

To do that, you need to import the  
TemplateInterface from robyn.templating

## Request

GET

/hello\_world

```
from robyn.templating import
TemplateInterface
```

```Then You need to have a  
`render\_template` method inside your
implementation. So, an example would
look like the following:

```
### Request
```

GET

/hello_world

class

JinjaTemplate(TemplateInterface): def

```
init(self, directory, encoding="utf-8",  
followlinks=False): self.env =  
Environment(  
loader=FileSystemLoader(  
searchpath=directory,  
encoding=encoding,  
followlinks=followlinks ) )
```

```
def render_template(self,  
template_name: str, kwargs): return  
self.env.get_template(template_na  
me).render(kwargs) ``## What's  
next?
```

Now, Batman wanted to have the ability to redirect the endpoints.

- Redirection
-

Testing

Batman wanted to test his Robyn application without spinning up a full server on every run. Robyn introduced him to the built-in `TestClient` — a lightweight, in-process test client that executes route handlers directly, making tests fast and deterministic.

Getting Started

The `TestClient` wraps a Robyn app and lets you call routes as if you were making real HTTP requests — but everything happens in-process. No ports, no sockets, no server startup.

Import `TestClient` from `robyn.testing`, pass your app to it, and start making

requests:

Request

GET

/hello

```
from robyn import Robyn
from robyn.testing import TestClient

app = Robyn(__file__)

@app.get("/hello")
def hello(request):
    return "Hello, World!"

client = TestClient(app)

def test_hello():
    response = client.get("/hello")
    assert response.status_code == 200
    assert response.text == "Hello,
World!"
```  
The TestResponse Object
```



Every request method returns a  
`TestResponse` with the following  
properties:

| Property      | Type      | Description                       |
|---------------|-----------|-----------------------------------|
| ---           | ---       | ---                               |
| `status_code` | `int`     | HTTP status code                  |
| `text`        | `str`     | Response body as a decoded string |
| `content`     | `bytes`   | Raw response body                 |
| `headers`     | `Headers` | Response headers                  |
| `ok`          | `bool`    | `True` if status is 2xx           |

`TestResponse` also has a `.json()`  
method that parses the body as JSON.

### Request

GET

/users

```
@app.get("/users") def
get_users(request): return [{"name":
"Batman"}, {"name": "Robin"}]
```

```
def test_json_response(): response =
client.get("/users") assert response.ok
data = response.json() assert len(data)
== 2 assert data[0]["name"] ==
"Batman" ```### HTTP Methods
```

The `TestClient` supports all common HTTP methods. Methods that typically send a body (`POST`, `PUT`, `PATCH`, `DELETE`) accept a `json_data` parameter for convenience.

Use `json_data` to send JSON payloads — the client automatically sets `Content-Type: application/json` and serializes the data:

# Request

POST

/items

```
@app.post("/items")
def create_item(request):
 body = request.json()
 return {"id": 1, "name":
body["name"]}

def test_post_json():
 response = client.post("/items",
json_data={"name": "Batarang"})
 assert response.status_code == 200
 assert response.json()["name"] ==
"Batarang"
```

```You can also send raw string or bytes bodies, custom headers, query parameters, form data, and files:

Request

POST

/upload

```
def test_with_all_options(): response =  
client.post( "/search", body="raw body  
content", headers={"X-Custom":  
"value"}, query_params={"q":  
"batman"}, ) assert response.ok ```###
```

Path Parameters

Routes with path parameters work exactly as they do in production. The `TestClient` matches the route pattern and extracts parameters automatically.

Path parameters are resolved from the URL and passed to your handler through the normal Robyn parameter resolution pipeline:

Request

GET

/users/:user_id

```
@app.get("/users/:user_id")
def get_user(request, user_id: int):
    return {"user_id": user_id}

def test_path_params():
    response = client.get("/users/42")
    assert response.json()["user_id"]
== 42
```### Testing Middleware
```

The `TestClient`` replicates the full request pipeline – before middlewares, the handler, global response headers, and after middlewares – in the same order as the Rust runtime.

Middlewares that modify the request or response are executed just like in production:

### Request

GET

/protected

```
@app.before_request() def
add_request_id(request):
request.headers.set("X-Request-ID",
"test-123") return request
```

```
@app.after_request() def
add_server_header(response):
response.headers.set("X-Server",
"Robyn") return response
```

```
@app.get("/protected") def
protected(request): return
request.headers.get("X-Request-ID")
```

```
def test_middleware_pipeline():
response = client.get("/protected")
```

```
assert response.text == "test-123"
assert response.headers.get("X-
Server") == "Robyn" ```### Using as a
Context Manager
```

TestClient implements the context manager protocol. When used with `with`, the internal event loop is automatically cleaned up:

## Request

GET

/

```
def test_with_context_manager():
 with TestClient(app) as client:
 response = client.get("/hello")
 assert response.ok
 # event loop is closed here
```### Running Tests with pytest
```

Since `TestClient` doesn't start a server, tests run as fast as regular unit tests. Use `pytest` directly – no special plugins or fixtures required.

A typical test file:

```
### Test File
```

```
TEST
```

```
test\_app.py
```

```
import pytest from robyn import  
Robyn from robyn.testing import  
TestClient
```

```
app = Robyn(file)
```

```
@app.get("/") def index(request):  
    return "Home"
```

```
@app.get("/health") def  
health(request): return {"status":
```



```
“ok”}
```

```
@app.post("/echo") def echo(request):  
    return request.json()
```

```
client = TestClient(app)
```

```
def test_index(): assert  
    client.get("/").text == "Home"
```

```
def test_health(): data =  
    client.get("/health").json() assert  
    data["status"] == "ok"
```

```
def test_echo(): payload = {"message":  
    "hello"} response = client.post("/echo",  
    json_data=payload) assert  
    response.json() == payload
```

```
def test_not_found(): response =  
    client.get("/nonexistent") assert
```

response.status_code == 404 ```Run
with:

```
pytest test_app.py -v  
```### Available Methods
```

| Method                                      | Signature       |
|---------------------------------------------|-----------------|
| ---                                         | ---             |
| `client.get(path, **kw)`                    | GET request     |
| `client.post(path, json_data=None, **kw)`   | POST request    |
| `client.put(path, json_data=None, **kw)`    | PUT request     |
| `client.patch(path, json_data=None, **kw)`  | PATCH request   |
| `client.delete(path, json_data=None, **kw)` | DELETE request  |
| `client.head(path, **kw)`                   | HEAD request    |
| `client.options(path, **kw)`                | OPTIONS request |

All methods accept these keyword arguments:

| Argument       | Type   | Description                 |
|----------------|--------|-----------------------------|
| ---            | ---    | ---                         |
| `body`         | `str`  | bytes`   Raw request body   |
| `headers`      | `dict` | Request headers             |
| `query_params` | `dict` | Query string parameters     |
| `form_data`    | `dict` | Form data fields            |
| `files`        | `dict` | File uploads (name → bytes) |

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

## # Timeout Configuration

Robyn supports comprehensive timeout configuration to handle high-concurrency scenarios and prevent resource exhaustion like the "Too many open files" error.

```
[Configuration Options]
(/documentation/en/api_reference/timeout
options)
```

```
Method Parameters
```

Configure timeouts directly in the  
`app.start()` method:

```
from robyn import Robyn
```

```
app = Robyn(file)
```

```
@app.get("/") async def hello(request):
return "Hello, world!"
```

# Configure timeout settings

```
app.start(host="0.0.0.0", port=8080,
client_timeout=30, # Client connection
timeout (seconds)
keep_alive_timeout=20 # Keep-alive
timeout (seconds)) ``###
```

Environment Variables

Override configuration using  
environment variables:

```
Set timeout configurations
export ROBYN_CLIENT_TIMEOUT=45
export ROBYN_KEEP_ALIVE_TIMEOUT=30
```

```
Start your application
python app.py
```## [Configuration Parameters]
(/documentation/en/api_reference/timeout
parameters)
```

| Parameter | Default | Description |
|----------------------------|---------|--|
| Environment Variable | | |
| --- | --- | --- |
| `client_timeout` | 30 | Maximum time (seconds) for client request processing |
| `ROBYN_CLIENT_TIMEOUT` | | |
| `keep_alive_timeout` | 20 | Time (seconds) to keep idle connections alive |
| `ROBYN_KEEP_ALIVE_TIMEOUT` | | |

```
## [Usage Examples]
(/documentation/en/api_reference/timeout
examples)
```

```
### Basic Configuration
```



Minimal timeout configuration

```
app.start(client_timeout=30) ```###  
High-Traffic Production Setup
```

```
# Optimized for high-traffic scenarios  
app.start(  
    host="0.0.0.0",  
    port=8080,  
    client_timeout=60,          # Allow  
longer processing time  
    keep_alive_timeout=15      # Shorter  
keep-alive for faster turnover  
)  
```### Development Setup
```

# Development-friendly settings

```
app.start(client_timeout=300, # Long
timeout for debugging
keep_alive_timeout=60 # Longer keep-
alive for testing) ``### Load Testing
Configuration
```

```
Optimized for load testing with tools
like wrk
app.start(
 client_timeout=10, # Quick
timeouts
 keep_alive_timeout=5 # Fast
connection turnover
)
```



```
` `` ## [Environment Variable Priority]
(/documentation/en/api_reference/timeout
variable-priority)
```

Environment variables take precedence  
over method parameters:



**This will use  
ROBYN\_CLIENT\_TIMEOUT  
UT=60 if set, otherwise  
30**

```
app.start(client_timeout=30) ``##
Troubleshooting
```

## **“Too Many Open Files” Error**

If you encounter file descriptor exhaustion:

- 1. Increase system limits:**

```
ulimit -n 65536
```2. **Optimize timeout
settings:**
```

```
app.start( client_timeout=15, #
Shorter timeouts
keep_alive_timeout=5 # Faster
connection cleanup ) ```3. Use
environment variables for
deployment:
```

```
export ROBYN_CLIENT_TIMEOUT=15
export
ROBYN_KEEP_ALIVE_TIMEOUT=5###
```

Performance Tuning

For high-throughput APIs:

- Lower keep_alive_timeout (5-15s)
- Moderate client_timeout (15-30s)

For long-running operations:

- Higher `client_timeout` (60-300s)
- Standard `keep_alive_timeout` (20-30s)

Best Practices

1. **Always set explicit timeouts** in production
2. **Use environment variables** for deployment-specific configuration
3. **Test timeout settings** with realistic load patterns
4. **Start with conservative values** and tune based on metrics

Migration Guide

From Previous Versions

If upgrading from earlier Robyn versions, the default behavior changes:

Before (infinite timeout):

```
# Previously: no timeout (could cause  
resource exhaustion)  
app.start(host="0.0.0.0", port=8080)  
````**After (sensible defaults):**
```

# **Now: automatic 30s client timeout, 20s keep-alive**

```
app.start(host="0.0.0.0", port=8080) #
Equivalent to: app.start(
host="0.0.0.0", port=8080,
client_timeout=30,
keep_alive_timeout=20)
```

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/api_
-->
```

```
[Using Rust to extend Robyn]
(/documentation/en/api_reference/using_rust-to-extend-robyn)
```

There may be occasions where Batman may be working with a high computation task, or a task that requires a lot of memory. In such cases, he may want to use Rust to implement that task. Robyn introduces a special way to do this. Not only you can use Rust to extend Python code, you can do it while maintaining the hot reloading nature of your codebase. Making it *\*feel\** like an interpreted version in many situations.

The first thing you need to is to create a Rust file. Let's call it ``hello_world.rs``. You can do it using the cli:

```
Request
```

```
GET
```

```
/hello_world
```

python -m robyn --create-rust-file  
hello\_world ```Then you can open the  
file and write your Rust code. For  
example, let's write a function that  
returns a string.

## Request

GET

```
/hello_world
```

```
// hello_world.rs
```

```
// rustimport:pyo3
```

```
use pyo3::prelude::*;
```

```
#[pyfunction]
```

```
fn square(n: i32) -> i32 {
```



```
 n * n
 // this is another comment
}
```

``Every Rust file that you create using the cli will have a special comment at the top of the file. This comment is used by Robyn to know which dependencies to import. In this case, we are importing the `pyo3` crate. You can import as many crates as you want. You can also import crates from crates.io. For example, if you want to use the `rusqlite` crate, you can do it like this:

```
Request
```

```
GET
```

```
/hello_world
```

```
// rustimport:pyo3
```

```
//: //: [dependencies] //: rusqlite =
"0.19.0"
```

```
use pyo3::prelude::*;
```

```
#[pyfunction] fn square(n: i32) -> i32 {
 n * n * n // this is another comment }
```

```
``Then you can import the function in
your Python code and use it.
```

## Request

GET

/hello\_world

```
from hello_world import square
```

```
print(square(5))
```

```
```To run the code, you need to use the  
`--compile-rust-path` flag. This will  
compile the Rust code and run it. You  
can also use the `--dev` flag to watch  
for changes in the Rust code and  
recompile it on the fly.
```

```
### Request
```

GET

/hello_world

python -m robyn --compile-rust-path
“.” --dev ``An example of a Robyn app
with a Rust file that using
therusqlite` crate to connect to a
database and return the number of
rows in a table:
<https://github.com/sansyrox/rusty-sql>

What's next?

Batman was curious to know what
else he could do with Robyn.

Robyn told him to keep an eye on the
GraphQL support.

GraphQL Support

WebSocketsWebSockets

WebSockets

After mastering Server-Sent Events for one-way communication, Batman realized he needed something more powerful. When Commissioner Gordon wanted to chat with him in real-time during crisis situations, Batman needed bidirectional communication.

“SSE is great for pushing updates to my dashboard,” Batman thought, “but I need two-way communication for coordinating with my allies!”

To handle real-time bidirectional communication, Batman learned how to work with WebSockets using Robyn's modern decorator-based API. Under the hood, messages flow through Rust channels for maximum performance — no Python GIL overhead during message dispatch.

Request

Basic EchoWith Callbacks

WebSocket

/web_socket

```
from robyn import Robyn

app = Robyn(__file__)

@app.websocket("/web_socket")
async def handler(websocket):
```

```
        while True:
            msg = await
websocket.receive_text()
            await
websocket.send_text(f"Echo: {msg}")

app.start()
```## [Receiving Messages]
(/documentation/en/api_reference/websocket
messages)
```

The `receive_text()` method blocks until the next message arrives from the client. It is backed by a Rust `tokio::mpsc` channel, so the Python handler genuinely suspends without holding the GIL.

When the client disconnects, `receive_text()` raises `WebSocketDisconnect`. You can either catch it explicitly or let the internal wrapper handle it silently.

```
Receiving Messages
```

Text MessagesJSON Messages

WebSocket

/web\\_socket

```
@app.websocket("/ws") async def
handler(websocket): try: while True:
msg = await websocket.receive_text()
await websocket.send_text(f"Got:
{msg}") except WebSocketDisconnect:
print(f"Client {websocket.id}
disconnected") ``## Sending Messages
```

To send a message to the current client, use `send_text()` or `send_json()`. All send methods are async.

## **Sending Messages**

Send TextSend JSON

# WebSocket

/web\_socket

```
@app.websocket("/ws")
async def handler(websocket):
 while True:
 msg = await
websocket.receive_text()
 await
websocket.send_text(f"Echo: {msg}")
```## [Broadcasting]
(/documentation/en/api_reference/websocket)
```

To send a message to all connected clients on the same WebSocket endpoint, use the `broadcast()` method.

Broadcasting

WebSocket

/chat




```
@app.websocket("/chat") async def  
handler(websocket): while True: msg  
= await websocket.receive_text() #  
Send to all connected clients await  
websocket.broadcast(f"User  
{websocket.id}: {msg}") # Also send a  
confirmation to this client only await  
websocket.send_text("Your message  
was sent") ``## Query Parameters
```

You can access query parameters
from the WebSocket connection URL
via `websocket.query_params`.

Query Params

WebSocket

/ws?

name=gordon&role=commissioner

```

@app.websocket("/ws")
async def handler(websocket):
    name =
websocket.query_params.get("name")
    role =
websocket.query_params.get("role")

    if name == "gordon" and role ==
"commissioner":
        await
websocket.broadcast("Gordon
authorized!")

    while True:
        msg = await
websocket.receive_text()
        await
websocket.send_text(f"Hello {name}:
{msg}")
` `` ## [Easy Access Query Parameters]
(/documentation/en/api_reference/websocket/
access-query-parameters)

```

Instead of manually calling
`websocket.query\_params.get(...)` , you
can declare typed query parameters

directly in your handler, ``on_connect``, and ``on_close`` signatures. Robyn will automatically resolve and coerce them – just like HTTP easy access parameters.

Parameters with defaults are optional. Parameters without defaults are required – if missing, the connection is rejected with an error message.

Easy Access Query Params

HandlerCallbacks

WebSocket

`/ws?room=chat&page=5`

```
@app.websocket("/ws") async def  
handler(websocket, room: str =  
"default", page: int = 1): try: while  
True: msg = await  
websocket.receive_text() await  
websocket.send_text( f"room={room}
```

```
page={page} msg={msg}” ) except  
WebSocketDisconnect: pass ``##
```

Closing Connections

To programmatically close a WebSocket connection from the server side, use `websocket.close()`. This will:

1. Close the WebSocket connection.
2. Remove the client from the WebSocket registry.
3. Cause any pending `receive_text()` to raise `WebSocketDisconnect`.

Close Connection

WebSocket

/ws

```

@app.websocket("/ws")
async def handler(websocket):
    while True:
        msg = await
websocket.receive_text()
        if msg == "quit":
            await websocket.close()
            break
        await
websocket.send_text(f"Got: {msg}")
```
[Connect and Close Callbacks]
(/documentation/en/api_reference/websocket-and-close-callbacks)

```

You can attach optional ``on_connect`` and ``on_close`` callbacks to your WebSocket handler. These are decorators on the handler function itself.

- \* ``on_connect`` is called when a new client connects. Its return value is sent to the client as the first message.

- \* ``on_close`` is called when the connection closes. Its return value is sent to the client as the final

message.

Both callbacks receive a ``websocket`` object with access to ``id`` and ``query_params``. Both are optional.

### Callbacks

WebSocket

/chat

```
@app.websocket("/chat") async def
chat(websocket): while True: msg =
await websocket.receive_text() await
websocket.broadcast(msg)
```

```
@chat.on_connect def
on_connect(websocket): return
f"Welcome, {websocket.id}!"
```

```
@chat.on_close def
on_close(websocket): return
```

## “Goodbye!” ``## WebSocket API Reference

The `websocket` object passed to handlers exposes the following methods and properties:

| Method / Property                             | Description                                                                       |
|-----------------------------------------------|-----------------------------------------------------------------------------------|
| <code>await websocket.receive_text()</code>   | Block until next message; raises <code>WebSocketDisconnect</code> on close        |
| <code>await websocket.receive_bytes()</code>  | Block until next binary message; raises <code>WebSocketDisconnect</code> on close |
| <code>await websocket.receive_json()</code>   | Same as <code>receive_text()</code> but JSON-decoded                              |
| <code>await websocket.send_text(data)</code>  | Send string to this client                                                        |
| <code>await websocket.send_bytes(data)</code> | Send binary data to this client                                                   |

`await websocket.send_json(data)` | Send  
JSON to this client |  
`await websocket.broadcast(data)` | Send  
to all clients on this endpoint |  
`await websocket.close()` | Close the  
connection server-side |  
`websocket.id` | Connection UUID string  
|  
`websocket.query_params` | Query  
parameters from the connection URL  
|

## **What's next?**

As the codebase grew, Batman wanted to onboard the justice league to help him manage the application.

Robyn told him about the different ways he could scale his application,



and how to use views and subrouters to make his code more readable.

- Views and SubRouters
- 

## **Robyn Architecture**

### **Overview**

Robyn's unique architecture combines Python's ease of development with Rust's performance. This hybrid design allows developers to write familiar Python code while benefiting from Rust's speed and memory safety.

# **The Python-Rust Hybrid Design**

## **Two-Layer Architecture**

Robyn operates on two interconnected but distinct layers:

### **Python Layer (Developer Interface):**

- Route definitions and decorators (`@app.get`, `@app.post`, etc.)
- Request parameter injection and validation
- Business logic execution
- Middleware configuration
- Response formatting

### **Rust Layer (Performance Engine):**

- HTTP request parsing and validation
- URL routing and pattern matching
- WebSocket connection management
- Static file serving
- Response serialization
- Memory management

## **Communication Bridge**

The layers communicate through **PyO3**, a Rust crate that enables seamless Python-Rust interoperability:

1. **Function Registration:** Python route handlers are registered with the Rust runtime at startup
2. **Request Flow:** Rust handles incoming HTTP requests and calls

Python handlers via PyO3

3. **Response Processing:** Python responses are converted back to Rust for efficient HTTP serialization

## Why This Design Works

- **Best of Both Worlds:** Python's productivity with Rust's performance
- **Zero-Copy Operations:** Minimal data copying between layers
- **Memory Safety:** Rust prevents common server vulnerabilities
- **Async Integration:** Seamless integration with Python's asyncio

# **Server Process Model**

Robyn is built on a multi-process, multi-threaded model that maximizes hardware utilization:

## **Master Process**

The master process in Robyn is responsible for initializing the server, managing worker processes, and handling signals. It creates a socket and passes it to the worker processes, allowing them to accept connections. The master process is implemented in Python, providing a familiar interface for developers while leveraging Rust's performance for core operations.

```

216:257:robyn/__init__.py
 def start(self, host: str =
"127.0.0.1", port: int = 8080,
_check_port: bool = True):
 """
 Starts the server

 :param host str: represents the
host at which the server is listening
 :param port int: represents the
port number at which the server is
listening
 :param _check_port bool:
represents if the port should be
checked if it is already in use
 """

 host = os.getenv("ROBYN_HOST",
host)
 port =
int(os.getenv("ROBYN_PORT", port))
 open_browser =
bool(os.getenv("ROBYN_BROWSER_OPEN",
self.config.open_browser))

 if _check_port:

```

```
 while
self.is_port_in_use(port):
 logger.error("Port %s
is already in use. Please use a
different port.", port)
 try:
 port =
int(input("Enter a different port: "))
 except Exception:

logger.error("Invalid port number.
Please enter a valid port number.")
 continue

 logger.info("Robyn version:
%s", __version__)
 logger.info("Starting server at
http://%s:%s", host, port)

 mp.allow_connection_pickling()

 run_processes(
 host,
 port,
 self.directories,
 self.request_headers,
```

```

 self.router.get_routes(),
self.middleware_router.get_global_middleware(),
self.middleware_router.get_route_middleware(),
self.web_socket_router.get_routes(),
 self.event_handlers,
 self.config.workers,
 self.config.processes,
 self.response_headers,
 open_browser,
)
```## [Worker Processes]
(/documentation/en/architecture#worker-
processes)

```

Robyn uses multiple worker processes to handle incoming requests. Each worker process is capable of managing multiple threads, allowing for efficient concurrent processing. The number of worker processes can be configured using the `--processes` flag, with a default of 1.


```
66:116:robyn/processpool.py def
init_processpool( directories:
List[Directory], request_headers:
Headers, routes: List[Route],
global_middlewares:
List[GlobalMiddleware],
route_middlewares:
List[RouteMiddleware], web_sockets:
Dict[str, WebSocket], event_handlers:
Dict[Events, FunctionInfo], socket:
SocketHeld, workers: int, processes:
int, response_headers: Headers, ) ->
List[Process]: process_pool = [] if
sys.platform.startswith("win32") or
processes == 1: spawn_process(
directories, request_headers, routes,
global_middlewares,
route_middlewares, web_sockets,
event_handlers, socket, workers,
response_headers, )
```

```

        return process_pool

for _ in range(processes):
    copied_socket = socket.try_clone()
    process = Process(
        target=spawn_process,
        args=(
            directories,
            request_headers,
            routes,
            global_middlewares,
            route_middlewares,
            web_sockets,
            event_handlers,
            copied_socket,
            workers,
            response_headers,
        ),
    )
    process.start()
    process_pool.append(process)

return process_pool

```

``## Worker Threads

Within each worker process, Robyn utilizes multiple threads to handle requests concurrently. The number of worker threads can be configured using the `--workers` flag. By default, Robyn uses a single worker thread per process.

Request Processing Flow

Understanding how requests flow through Robyn's hybrid architecture:

1. Request Arrival

HTTP Request → Rust HTTP Parser → Fast Validation

```### 2. Routing and Matching

Validated Request → Rust Router (matchit crate) → Route Resolution

### ```### 3. Parameter Extraction

Matched Route → Rust Parameter Parser →  
Path/Query/Header Extraction

### ```### 4. Python Handler Execution

Extracted Parameters → PyO3 Bridge  
→ Python Handler → Response ```### 5.  
Response Processing

Python Response → Rust Serializer →  
HTTP Response → Client

```## [Rust Integration Deep Dive]  
(/documentation/en/architecture#rust-
integration-deep-dive)

Robyn's Rust integration is powered by
the ****Tokio**** async runtime and several
high-performance crates:

Core Components

* ****Tokio Runtime****: Handles async I/O
and task scheduling

* ****Actix Web****: Provides HTTP server

functionality
* **PyO3**: Enables Python-Rust
communication
* **matchit**: Ultra-fast URL routing
with radix tree implementation
* **Serde**: JSON
serialization/deserialization

Performance Benefits

1. **Zero-allocation routing** using
compiled radix trees
2. **Memory-efficient HTTP parsing**
with minimal allocations
3. **Async task scheduling** without
GIL interference
4. **Direct memory access** for static
file serving

```
76:107:src/server.rs pub fn start( &mut  
self, py: Python, socket: &PyCell,  
workers: usize, ) -> PyResult<()> {  
pyo3_log::init();
```

```
    if STARTED
        .compare_exchange(false, true,
SeqCst, Relaxed)
        .is_err()
    {
        debug!("Robyn is already
running...");
        return Ok(());
    }
```

```
    let raw_socket =
socket.try_borrow_mut()?.get_socket();
```

```
    let router = self.router.clone();
    let const_router =
self.const_router.clone();
    let middleware_router =
self.middleware_router.clone();
    let websocket_router =
self.websocket_router.clone();
    let global_request_headers =
self.global_request_headers.clone();
    let global_response_headers =
self.global_response_headers.clone();
    let directories =
self.directories.clone();
```

```
    let asyncio =  
py.import("asyncio")?;  
    let event_loop =  
asyncio.call_method0("new_event_loop")?;  
  
asyncio.call_method1("set_event_loop",  
(event_loop, ))?;  
  
    let startup_handler =  
self.startup_handler.clone();  
    let shutdown_handler =  
self.shutdown_handler.clone();
```

``## Const Requests Optimization

Robyn's "Const Requests" feature provides significant performance improvements for static endpoints:

How Const Requests Work

- 1. Route Registration:** Routes marked with `const=True` are

- identified at startup
2. **Response Caching:** The first response is cached in Rust memory
 3. **Direct Serving:** Subsequent requests bypass Python entirely
 4. **Zero Overhead:** Responses are served directly from Rust with minimal CPU usage

Performance Impact

- **10x faster response times** compared to regular Python handlers
- **Minimal memory usage** with efficient caching
- **No Python GIL contention** for cached responses
- **Ideal for:** Health checks, API metadata, configuration endpoints

Example Usage

```
from robyn import Robyn

app = Robyn(__file__)

# Regular route - executes Python on
# every request
@app.get("/dynamic")
def dynamic_endpoint():
    return {"timestamp": time.time()}
# Changes every request

# Const route - cached in Rust after
# first request
@app.get("/health", const=True)
def health_check():
    return {"status": "healthy",
            "version": "1.0.0"} # Static response

# Perfect for API metadata
@app.get("/api/info", const=True)
def api_info():
    return {
        "name": "My API",
        "version": "2.1.0",
```

```
        "endpoints": ["/users",
"/posts", "/health"]
    }
```

```### When to Use Const Routes

- \* \*\*Health/status endpoints\*\* that return consistent data
- \* \*\*API documentation\*\* or metadata endpoints
- \* \*\*Configuration endpoints\*\* with static values
- \* \*\*Version information\*\* endpoints

## [Scaling Configuration Guide]  
(/documentation/en/architecture#scaling-configuration-guide)

### Understanding Processes vs Workers

**Processes:**

- \* Independent Python interpreters
- \* Share no memory (shared-nothing architecture)
- \* Each process has its own GIL
- \* Best for CPU-bound applications

- \* Recommended: 1 process per CPU core

**\*\*Workers\*\*** (within each process):

- \* Threads sharing the same Python interpreter

- \* Affected by Python's GIL

- \* Better for I/O-bound operations

- \* Recommended: 2-4 workers per process

### ### Configuration Strategies

#### #### CPU-Intensive Applications

**Favor more processes,  
fewer workers**

```
python app.py --processes=8 --
workers=1
```

# Example: Image processing, data analysis, calculations

```
``#### I/O-Intensive Applications
```

```
Favor fewer processes, more workers
python app.py --processes=2 --workers=8
```

```
Example: Database queries, API calls,
file operations
```

```
```#### Balanced Applications
```

General-purpose configuration

```
python app.py --processes=4 --  
workers=2
```

Most web applications fit this pattern

```### Hardware-Based  
Recommendations

For a system with **N CPU cores**:

| Application Type | Processes | Workers |
|------------------|-----------|---------|
| CPU-bound        | N         | 1       |
| I/O-bound        | N/2       | 4       |
| Balanced         | N/2       | 2       |
| High-traffic     | N         | 2       |

# Performance Testing

Always benchmark your specific application:

```
Test different configurations
python app.py --processes=1 --workers=1
Baseline
python app.py --processes=2 --workers=2
Moderate scaling
python app.py --processes=4 --workers=1
CPU-focused
python app.py --processes=2 --workers=4
I/O-focused
python app.py --fast
Auto-optimized
```## [Scaling Considerations]
(/documentation/en/architecture#scaling-considerations)
```

* Robyn's multi-process model allows it to scale across multiple CPU cores effectively.

* The combination of Python and Rust allows for both ease of development and high performance.

- * Const Requests feature can significantly improve performance for routes with constant output.
- * When scaling, consider both the number of processes and workers to find the optimal configuration for your hardware and application needs.

[Development Mode]
(/documentation/en/architecture#development mode)

Robyn provides a development mode that can be activated using the `--dev` flag. This mode is designed for ease of development and includes features like hot reloading. Note that in development mode, multi-process and multi-worker configurations are disabled to ensure consistent behavior during development.`

```
92:101:robyn/argument_parser.py if  
self.dev and (self.processes != 1 or  
self.workers != 1): raise Exception(“–
```

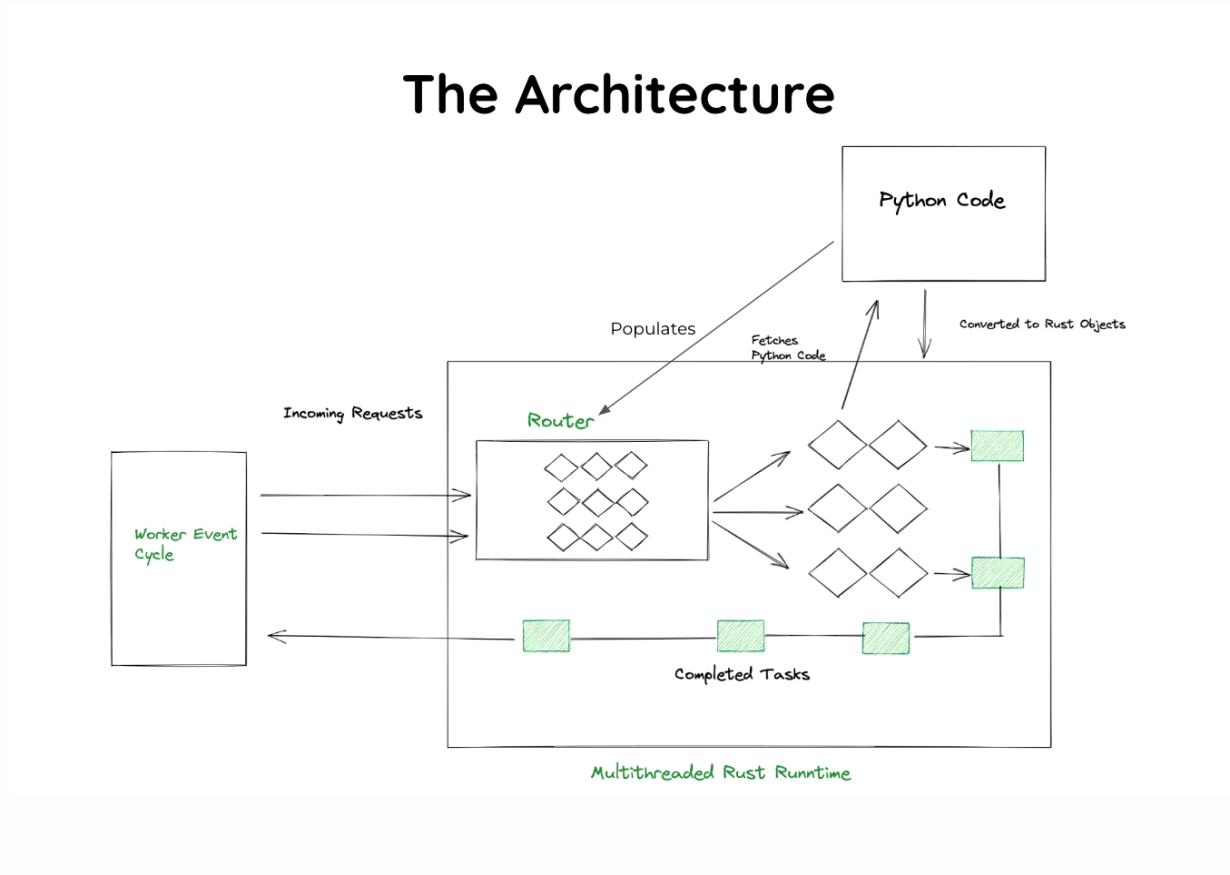
processes and `-workers` shouldn't be used with `-dev`")

```
    if self.dev and args.log_level is
None:
        self.log_level = "DEBUG"

    elif args.log_level is None:
        self.log_level = "INFO"
    else:
        self.log_level = args.log_level
```

“By understanding these design principles and adjusting the configuration accordingly, developers can leverage Robyn's unique architecture to build high-performance web applications that efficiently utilize system resources.

Design Diagram



Talks

- EuroPython 2022
- GeoPython 2022
- PyCon US 2022
- PyCon Sweden 2021

Blogs

- Hello, Robyn!

Next Steps

Batman was now interes

- Hosting
-

Real Life Web Apps with Robyn

Batman was tasked with building a web application to manage the crime data in Gotham City. The application would allow the Gotham police department to store and retrieve data on criminal activities, suspects, and their locations. He decided to use the Robyn web framework to build this application efficiently and quickly.

You can find the source code for this application [here](#).

Installing Robyn

The first step was to install Robyn. Batman created a virtual environment and installed Robyn using pip.

```
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install robyn
```## [Creating a Robyn Application]
(/documentation/en/example_app#creating-a-robyn-application)
```

Batman wanted to create a Robyn app and was about to create an `src/app.py` before he was told that Robyn comes with a CLI tool to create a new application. He ran the following command to create a new Robyn application.

```
$ python -m robyn --create ``This,
would result in the following output.
```

```
$ python3 -m robyn --create
? Directory Path: .
? Need Docker? (Y/N) Y
? Please select project type
(Mongo/Postgres/Sqlalchemy/Prisma):
> No DB
 Sqlite
 Postgres
 MongoDB
 SqlAlchemy
 Prisma
```and the following directory
structure.
```

Batman was asked a set of questions to configure the application. He chose to use the default values for most of the questions.

And he was done! The Robyn CLI created a new application with the following structure.

```
|— src | |— app.py |— Dockerfile
```Modeling Routes
```

---

# **Authentication and Authorization**

To restrict access to the crime data, Batman added authentication and authorization to the application. He decided to use JWT (JSON Web Token) for authentication. He created a new table for users and added an endpoint for user registration.

## **User Model**

Batman added a new User model to represent the users who can access the application.



## Example request with basic auth

```
models.py
from sqlalchemy import Column, Integer,
String, Boolean

class User(Base):
 __tablename__ = "users"

 id = Column(Integer,
primary_key=True, index=True)
 username = Column(String,
unique=True, index=True)
 hashed_password = Column(String)

 def __repr__(self):
 return "<User(id={id},
username={username}, hashed_password=
{hashed_password})>".format(
 id=self.id,
 username=self.username,
 hashed_password=self.hashed_password,
)
```Then in crud.py, he added a new
```

method to create a user.

Example request with basic auth

crud.py

**also need to do pip
install passlib[bcrypt]**

```
from sqlalchemy.orm import Session  
from .models import User
```

```
from passlib.context import  
CryptContext
```

```
pwd_context =  
CryptContext(schemes=["bcrypt"],  
deprectated="auto")
```

```
def get_password_hash(password: str)  
-> str: return  
pwd_context.hash(password)
```

```
def get_user(db: Session, user_id: int):  
    return db.query(User).filter(User.id ==  
    user_id).first()
```

```
def create_user(db: Session, user:  
    User): hashed_password =  
    get_password_hash(user.password)  
    db_user =  
    User(username=user.username,  
    hashed_password=hashed_password)  
    db.add(db_user) db.commit()  
    db.refresh(db_user) return db_user  
``## Authentication Utilities
```

Batman created utility functions to handle authentication, including hashing passwords and verifying passwords.

Example request with bearer token

```
# crud.py
# also need to do pip install
passlib[bcrypt]
# pip install "python-
jose[cryptography]"
from passlib.context import
CryptContext
from jose import JWTError, jwt

pwd_context = CryptContext(schemes=
["bcrypt"], deprecated="auto")

def verify_password(plain_password,
hashed_password):
    return
pwd_context.verify(plain_password,
hashed_password)

def get_password_hash(password):
    return pwd_context.hash(password)

ALGORITHM = "HS256"
SECRET_KEY = "your_secret_key"
```

```
def create_access_token(data: dict,
expires_delta: timedelta = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() +
expires_delta
    else:
        expire = datetime.utcnow() +
timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode,
SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

```
def decode_access_token(token: str):
    return jwt.decode(token,
SECRET_KEY, algorithms=[ALGORITHM])
```

```
def authenticate_user(db: Session,
username: str, password: str):
    user = get_user_by_username(db,
username)
    if user is None:
        return False
    if not verify_password(password,
```

```
user.hashed_password):  
    return False
```

```
        created_token =  
create_access_token(data={"sub":  
user.username})  
    return created_token  
````## [User Registration Endpoint]  
(/documentation/en/example_app/authentic
registration-endpoint)
```

Batman added a new endpoint to allow users to register.

### Setting up Routes

```
from . import crud
```

```
@app.post("/users/register") async def
register_user(request): user =
request.json() with SessionLocal() as
db: created_user =
crud.create_user(db, user) return
created_user
```



```
@app.post("/users/login") async def
login_user(request): user =
request.json() with SessionLocal() as
db: token =
crud.authenticate_user(db, **user)

if token is None:
 raise
HTTPException(status_code=401,
detail="Invalid credentials")

return {"access_token": token}
```

``Next: Authentication Middlewares

---

## Authentication and Authorization Middleware

Batman added middleware to the Robyn application to verify the JWT

tokens and to restrict access to certain endpoints based on the user's role.

## **Setting up Authentication Middlewares**

```
from robyn.authentication import
AuthenticationHandler, BearerGetter,
Identity
```

```
class
BasicAuthHandler(AuthenticationHandler)
 def authenticate(self, request:
Request):
 token =
self.token_getter.get_token(request)

 try:
 payload =
crud.decode_access_token(token)
 username = payload["sub"]
 except Exception:
 return
```

```

 with SessionLocal() as db:
 user =
crud.get_user_by_username(db,
username=username)

 return Identity(claims={"user":
f"{ user }"})

app.configure_authentication(BasicAuthH

```

```

@app.get("/users/me",
auth_required=True)
async def get_current_user(request):
 user =
request.identity.claims["user"]
 return user

```

````With the web application in place, the Gotham City Police Department could now efficiently manage crime data and track criminal activities in real-time. Batman had successfully used the Robyn web framework to build a real-world application to help fight crime in Gotham City.

```
[Checkout the real time notifications]
(/documentation/en/example_app/real_time
```

```
---
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/exar
-->
```

```
## [Deploying the Application]
(/documentation/en/example_app/deploymer
the-application)
```

After thoroughly testing the web application and ensuring that all features were working as expected, Batman decided it was time to deploy it to a production server. He chose to use a robust and scalable platform, ensuring that his application would be available and performant at all times.

```
### Deploying the Application
```

python app.py --processes=n --workers=m ```With the web application deployed and running smoothly, Batman had a powerful new tool at his disposal. The Robyn framework had provided him with the flexibility, scalability, and performance needed to create an effective crime-fighting application, giving him a technological edge in his ongoing battle to protect Gotham City.

OpenAPI Docs

Crime Data Model and Database Connection

Batman designed a data model to represent crime data, including

information about the crime, suspect, and location. He decided to use a SQLite database to store the data and used an ORM (Object Relational Mapping) library to interact with the database.

```
# models.py
from sqlalchemy import create_engine,
Column, Integer, String, DateTime,
Float
from sqlalchemy.orm import
declarative_base, sessionmaker

DATABASE_URL =
"sqlite:///./gotham_crime_data.db"

engine = create_engine(DATABASE_URL)
SessionLocal =
sessionmaker(autocommit=False,
autoflush=False, bind=engine)

Base = declarative_base()
```

```
class Crime(Base):
    __tablename__ = "crimes"

    id = Column(Integer,
primary_key=True, index=True)
    type = Column(String, index=True)
    description = Column(String)
    location = Column(String)
    suspect_name = Column(String)
    date_time = Column(DateTime)
    latitude = Column(Float)
    longitude = Column(Float)
````## [Setting up the Robyn
Application]
(/documentation/en/example_app/modeling_
up-the-robyn-application)
```

Batman set up a Robyn application and configured it to use the database session to access the SQLite database.

Based on the Database model, Batman created a few helper methods to interact with the database. These methods would be used by the endpoints to perform CRUD operations on the

```
database.
```

```
crud.py
```



# crud.py

```
from sqlalchemy.orm import Session
from .models import Crime
```

```
def get_crime(db: Session, crime_id:
int): return
db.query(Crime).filter(Crime.id ==
crime_id).first()
```

```
def get_crimes(db: Session, skip: int =
0, limit: int = 100): return
db.query(Crime).offset(skip).limit(limi
t).all()
```

```
def create_crime(db: Session, crime):
db_crime = Crime(**crime)
```

```
db.add(db_crime) db.commit()
db.refresh(db_crime) return db_crime
```

```
def update_crime(db: Session,
crime_id: int, crime): db_crime =
get_crime(db, crime_id) if db_crime is
None: return None for key, value in
crime.items(): setattr(db_crime, key,
value) db.commit()
db.refresh(db_crime) return db_crime
```

```
def delete_crime(db: Session,
crime_id: int): db_crime =
get_crime(db, crime_id) if db_crime is
None: return False
db.delete(db_crime) db.commit()
return True ``## Crime Data
Endpoints
```

Batman created various endpoints to manage crime data. These endpoints allowed the Gotham City Police

Department to add, update, and retrieve crime data.

**A note on `def` VS `async def`.** The CRUD helpers above use **synchronous** SQLAlchemy, so the handlers below are deliberately plain `def` functions. Robyn runs synchronous handlers in a worker thread, which means a blocking database call there never stalls the event loop. If you would rather write `async def` handlers, pair them with an **async** database driver (for example SQLAlchemy's `asyncio` extension via `create_async_engine` and `await session.execute(...)`) so the database I/O is genuinely non-blocking. The one combination to avoid is a blocking, synchronous

DB call inside an `async def` handler  
— that ties up the event loop for  
every other request.

## Setting up Routes

```
__main__.py
from robyn import Robyn
from robyn.robyn import Request,
Response
from sqlalchemy.orm import Session

app = Robyn(__file__)

@app.post("/crimes")
def add_crime(request):
 with SessionLocal() as db:
 crime = request.json()
 insertion =
crud.create_crime(db, crime)

 if insertion is None:
 raise Exception("Crime not
added")
```

```
 return {
 "description": "Crime added
successfully",
 "status_code": 200,
 }
```

```
@app.get("/crimes")
def get_crimes(request):
 with SessionLocal() as db:
 skip =
int(request.query_params.get("skip",
"0"))
 limit =
int(request.query_params.get("limit",
"100"))
 crimes = crud.get_crimes(db,
skip=skip, limit=limit)

 return crimes
```

```
@app.get("/crimes/:crime_id",
auth_required=True)
def get_crime(request):
 crime_id =
int(request.path_params.get("crime_id"))
 with SessionLocal() as db:
```

```
 crime = crud.get_crime(db,
crime_id=crime_id)
```

```
 if crime is None:
 raise Exception("Crime not
found")
```

```
 return crime
```

```
@app.put("/crimes/:crime_id")
```

```
def update_crime(request):
```

```
 crime = request.json()
```

```
 crime_id =
```

```
int(request.path_params.get("crime_id"))
```

```
 with SessionLocal() as db:
```

```
 updated_crime =
```

```
crud.update_crime(db,
```

```
crime_id=crime_id, crime=crime)
```

```
 if updated_crime is None:
```

```
 raise Exception("Crime not
found")
```

```
 return updated_crime
```

```
@app.delete("/crimes/:crime_id")
```

```
def delete_crime(request):
```

```
 crime_id =
```

```
int(request.path_params.get("crime_id"))
 with SessionLocal() as db:
 success = crud.delete_crime(db,
crime_id=crime_id)
 if not success:
 raise Exception("Crime not
found")
 return {"message": "Crime deleted
successfully"}
```[Next: Authentication]
(/documentation/en/example_app/authentic
```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/exar
-->
```

```
## [Monitoring and Logging]
(/documentation/en/example_app/monitorin
and-logging)
```

To keep an eye on the performance of his application and troubleshoot issues, Batman wanted a proper access log – one line per request showing the

method, path, status code, and how long the request took.

Robyn builds on Python's standard ``logging`` module, so you can assemble a request/access log from two middlewares: a ``before_request`` hook that starts a timer, and an ``after_request`` hook that emits the log line once the response is ready. The ``after_request`` hook can receive **both** the ``request`` and the ``response``, which is exactly what you need to log the path and the status code together. And because Robyn shares a ``contextvars`` context across the ``before_request``, handler, and ``after_request`` hooks, a ``ContextVar`` is a tidy place to stash the start time.

```
### Request / access logging
```

```
import logging
import time
from contextvars import ContextVar
```


from robyn import Request, Response

**A dedicated logger for
access logs. It inherits
Robyn's logging
configuration,**

**so keep the level at
INFO or below
(running with --log-
level WARN would hide**

these lines).

```
logging.basicConfig(level=logging.INFO)
access_logger = logging.getLogger("robyn.access")
```

```
_request_start: ContextVar[float] = ContextVar("request_start")
```

```
@app.before_request()
def start_timer(request: Request):
    _request_start.set(time.perf_counter())
    return request
```

```
@app.after_request()
def log_request(request: Request, response: Response):
    start =
```

```
_request_start.get(None) duration_ms
= (time.perf_counter() - start) * 1000 if
start is not None else 0.0
access_logger.info( "%s %s -> %s
(%.2fms)", request.method,
request.url.path,
response.status_code, duration_ms, )
return response ```With those two
hooks registered globally, every
request produces a line such as:
```

```
INFO:robyn.access:GET /crimes -> 200
(1.83ms)
```

```A couple of things worth knowing:

\* The two-argument  
`after\_request(request, response)`  
signature is what gives you access to  
the request alongside the response. If  
you only need the response, a single-  
argument `after\_request(response)`  
works too – Robyn picks the calling  
convention based on your function's  
parameters.

- \* ``after_request`` hooks always run, even when a ``before_request`` hook short-circuits the request, so your access log captures every response.
- \* Prefer ``%s``-style logging arguments (as above) over f-strings so the message is only formatted when the log level is actually enabled.

With monitoring and logging in place, Batman could now easily detect issues and analyze the performance of his web application, ensuring that it was always running optimally and ready to assist him in his fight against crime.

```
[Deploying the application]
(/documentation/en/example_app/deploymer
```

```

```

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/exar
-->
```

```
[OpenAPI Docs a.k.a Swagger]
```

(/documentation/en/example\_app/openapi#docs-aka-swagger)

After deploying the application, Batman got multiple queries from the users on how to use the endpoints. Robyn showed him how to generate OpenAPI specifications for his application.

Out of the box, the following endpoints are setup for you:

- \* `/docs`` The Swagger UI
- \* `/openapi.json`` The JSON Specification

However, if you don't want to generate the OpenAPI docs, you can disable it by passing `--disable-openapi`` flag while starting the application.

To use a custom openapi configuration, you can:

- \* Place the `openapi.json`` config file in the root directory.

\* Or, pass the file path to the  
`openapi\_file\_path` parameter in the  
`Robyn()` constructor. (the parameter  
gets priority over the file).

python app.py --disable-openapi ``##  
How to use?

- Query Params: The typing for query params can be added as `def get(r: Request, query_params: GetRequestParams)` where `GetRequestParams` is a subclass of `QueryParams`
- Path Params are defaulted to string type (ref: [https://en.wikipedia.org/wiki/Query\\_string](https://en.wikipedia.org/wiki/Query_string))



# Basic App

```
from robyn.robyn import QueryParams

from robyn import Robyn, Request

app = Robyn(
 file_object=__file__,
 openapi=OpenAPI(
 info=OpenAPIInfo(
 title="Sample App",
 description="This is a
sample server application.",

termsOfService="https://example.com/terms",
 version="1.0.0",
 contact=Contact(
 name="API Support",

url="https://www.example.com/support",

email="support@example.com",
),
 license=License(
 name="BSD2.0",
```

```
url="https://opensource.org/license/bsd-2-clause",
),
```

```
externalDocs=ExternalDocumentation(description="more info here",
url="https://example.com/"),
 components=Components(),
),
),
)
```

```
@app.get("/")
async def welcome():
 """welcome endpoint"""
 return "hi"
```

```
class GetRequestParams(QueryParams):
 appointment_id: str
 year: int
```

```
@app.get("/api/v1/name",
openapi_name="Name Route",
```

```
openapi_tags=["Name"])
async def get(r: Request, query_params:
GetRequestParams):
 """Get Name by ID"""
 return r.query_params
```

```
@app.delete("/users/:name",
openapi_tags=["Name"])
async def delete(r: Request):
 """Delete Name by ID"""
 return r.path_params
```

```
if __name__ == "__main__":
 app.start()
```## [How does it work with
subrouters?]
(/documentation/en/example_app/openapi#l
does-it-work-with-subrouters)
```

```
### Subrouters
```

```
from robyn.robyn import
QueryParams
```

```
from robyn import Request,  
SubRouter
```

```
subrouter: SubRouter =  
SubRouter(prefix="/sub")
```

```
@subrouter.get("/") async def  
subrouter_welcome(): """welcome  
subrouter""" return "hiiiiii subrouter"
```

```
class  
SubRouterGetRequestParams(QueryP  
arams): _id: int value: str
```

```
@subrouter.get("/name") async def  
subrouter_get(r: Request,  
query_params:  
SubRouterGetRequestParams): """Get  
Name by ID""" return r.query_params
```

```
@subrouter.delete("/:name") async def  
subrouter_delete(r: Request):
```

```
“““Delete Name by ID””” return  
r.path_params
```

```
app.include_router(subrouter) ``##  
Other Specification Params
```

We support all the params mentioned in the latest OpenAPI specifications (<https://swagger.io/specification/>). See an example using request & response bodies below:

Request & Response Body

```
from robyn.types import JSONResponse,  
Body
```

```
class Initial(Body):  
    is_present: bool  
    letter: Optional[str]
```

```
class FullName(Body):  
    first: str
```

```
second: str
initial: Initial
```

```
class CreateItemBody(Body):
    name: FullName
    description: str
    price: float
    tax: float
```

```
class CreateResponse(JSONResponse):
    success: bool
    items_changed: int
```

```
@app.post("/")
def create_item(request: Request, body:
CreateItemBody) -> CreateResponse:
    return CreateResponse(success=True,
items_changed=2)
```

```With the reference documentation deployed and running smoothly, Batman had a powerful new tool at his disposal. The Robyn framework had provided him with the flexibility,

scalability, and performance needed to create an effective crime-fighting application, giving him a technological edge in his ongoing battle to protect Gotham City.

[Templates]

(/documentation/en/example\_app/templates

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/exar
-->
```

```
[Real time notifications]
(/documentation/en/example_app/real_time
time-notifications)
```

Batman decided to implement real-time notifications for police officers using WebSockets. This would allow officers to receive instant updates on criminal activities, as well as alerts when a new crime is reported.

```
Setting up Real-time Notifications
```

```
from robyn import
WebSocketDisconnect
```

```
@app.websocket("/notifications")
async def notify_handler(websocket):
try: while True: message = await
websocket.receive_text() await
websocket.send_text(f"Received:
{message}") except
WebSocketDisconnect: pass
```

```
@notify_handler.on_connect def
notify_connect(websocket): return
"Connected to notifications"
```

```
@notify_handler.on_close def
notify_close(websocket): return
"Disconnected from notifications"
``## Advanced Search and Filtering
```



To make it easier for the police officers to search for specific crimes or criminals, Batman added advanced search and filtering options to the application. He implemented a new endpoint that allows users to search based on various criteria like crime type, date, location, and status.

## **Advanced Search and Filtering**

```
@app.get("/crimes/search")
async def search_crimes(request):
 crime_type =
request.query_params.get("crime_type")
 date =
request.query_params.get("date")
 location =
request.query_params.get("location")
 status =
request.query_params.get("status")

 crimes = crud.search_crimes(db,
crime_type=crime_type, date=date,
```

```
location=location, status=status)
 return crimes
```

```With the new features in place, the Gotham City Police Department was able to use the web application more effectively to track criminal activities and deploy resources efficiently. Batman's work on the Robyn web framework had a significant impact on Gotham City's crime-fighting efforts, making it a safer place for its citizens.

Although Batman had achieved great success with the current implementation, he knew that there would always be room for improvement and new features to add. But for now, he could take a moment to appreciate his work and focus on his primary duty - protecting Gotham City as the Dark Knight.

[Checkout the monitoring and logging]
(/documentation/en/example_app/monitoring)

```
<!-- robyn-documentation source:  
https://robyn.tech/documentation/en/exar  
-->
```

```
## [Code Organization with SubRouters]  
(/documentation/en/example_app/subrouter  
organization-with-subrouters)
```

As the application grew, Batman needed a way to organize his routes better. He decided to use Robyn's SubRouter feature to group related routes together.

```
from robyn import SubRouter
```

Create a subrouter for crime-related routes

```
crime_router =  
SubRouter(prefix="/crimes")
```

```
@crime_router.get("/list") def  
list_crimes(): return {"crimes":  
get_all_crimes()}
```

```
@crime_router.post("/report") def  
report_crime(request): crime_data =  
request.json() return {"id":  
create_crime_report(crime_data)}
```

Create a subrouter for suspect-related routes

```
suspect_router =  
SubRouter(prefix="/suspects")
```

```
@suspect_router.get("/list") def  
list_suspects(): return {"suspects":  
get_all_suspects()}
```

```
@suspect_router.get("/:id") def  
get_suspect(request, path_params):  
suspect_id = path_params.id return  
{"suspect":  
get_suspect_by_id(suspect_id)}
```

Include the subrouters in the main app

```
app.include_router(crime_router)
app.include_router(suspect_router)
```

``SubRouters help organize related routes under a common prefix, making the code more maintainable and easier to understand. In this example:

- All crime-related routes are under `/crimes`
- All suspect-related routes are under `/suspects`

This organization makes it clear which routes handle what functionality and keeps related code together.

Templates

After implementing the backend, Batman decided to add a frontend to his application. He wanted to create a simple web page that would allow him to view the data he had collected. He also wanted to be able to add new data to the database and edit existing data.

This is when he was introduced to templates!

Templates are a powerful feature of the Robyn framework that allow you to create dynamic web pages using HTML and Python. They are a great way to add a frontend to your application without having to learn a new language or framework.

Robyn supports Jinja2 templates by default but provides an easy way to add other templating engines as well.

Creating a Template

To create a template, you need to create a file with the `.html` extension in the a directory, usually it is convention to use the `templates` directory. For example, if you wanted to create a template called `index.html`,

you would create a file called `index.html` in the `templates` directory.

So the folder structure would look like this:

```
|— app.py
|— templates
|   |— index.html
|— Dockerfile
|— requirements.txt
```

```### Rendering a Template

Once you have created a template, you can render it by using the ``render_template`` function. This function takes the name of the template as its first argument and a dictionary of variables as its second argument.

For example, if you wanted to render the ``index.html`` template, you would use the following code:

### Rendering a Template

```
import os
import pathlib
from robyn.templating import JinjaTemplate
```

```
current_file_path =
pathlib.Path(file).parent.resolve()
jinja_template =
JinjaTemplate(os.path.join(current_file_path, "templates"))
```

```
@app.get("/frontend") async def
get_frontend(request): context =
{ "framework": "Robyn",
 "templating_engine": "Jinja2" }
return jinja_template.render_template("index.html", **context)
```

app.include\_router(frontend) ``Now Batman very happy that the application had come to completion. However, he was not satisfied with the current state of the application. He

felt the code was all crammed in a single file and asked Robyn if there was a way to split the codebase in other parts.

This is Robyn introduced him to the concept of routers and views.

## SubRouters and Views

---

# **Performance comparison across different frameworks**

**Read this before you scroll  
down**

Before delving into the details, it is imperative to note that this comparison doesn't aim to discredit any developers or the frameworks listed below. Mentioning the names of the frameworks is solely for

elucidating a clear comparison. My profound inclination towards the Python web ecosystem has been significantly influenced by all these frameworks, and my intention is not to cause offense to anyone by listing them here.

Moreover, these tests were conducted on my development machine, and thus, the figures portrayed below are not absolute. The numbers only serve to indicate the relative performance of these frameworks under the specific testing conditions.

The oha tool was utilized to test 10,000 requests on the following frameworks, yielding the subsequent results:

1. Flask(Gunicorn)

Total: 5.5254 secs  
Slowest: 0.0784 secs  
Fastest: 0.0028 secs  
Average: 0.0275 secs  
Requests/sec: 1809.8082  
```1. FastAPI(Uvicorn)

Total: 4.1314 secs Slowest: 0.0733 secs
Fastest: 0.0027 secs Average: 0.0206
secs Requests/sec: 2420.4851 ```1.
Django(Gunicorn)

Total: 13.5070 secs
Slowest: 0.3635 secs
Fastest: 0.0249 secs
Average: 0.0674 secs
Requests/sec: 740.3558
```1. Robyn(Doesn't need a \\*SGI)

Total: 1.8324 secs Slowest: 0.0269 secs  
Fastest: 0.0024 secs Average: 0.0091  
secs Requests/sec: 5457.2339 ```1.  
Robyn (5 workers)

```
Total: 1.5592 secs
Slowest: 0.0211 secs
Fastest: 0.0017 secs
Average: 0.0078 secs
Requests/sec: 6413.6480
```

```Robyn is able to serve the 10k requests in 1.8 seconds followed by Flask and FastAPI, which take around 5 seconds(using 5 workers on a dual-core machine). Finally, Django takes around 13.5070 seconds.

```
## [Verbose Logs]
(/documentation/en/framework_performance
logs)
```

```
Flask(Gunicorn)
```

```
Summary: Success rate: 1.0000 Total:
5.5254 secs Slowest: 0.0784 secs
Fastest: 0.0028 secs Average: 0.0275
secs Requests/sec: 1809.8082
```

Total data: 126.95 KiB Size/request: 13 B Size/sec: 22.98 KiB

[illegible]

Latency distribution: 10% in 0.0178
secs 25% in 0.0223 secs 50% in 0.0266
secs 75% in 0.0317 secs 90% in 0.0378
secs 95% in 0.0419 secs 99% in 0.0551
secs

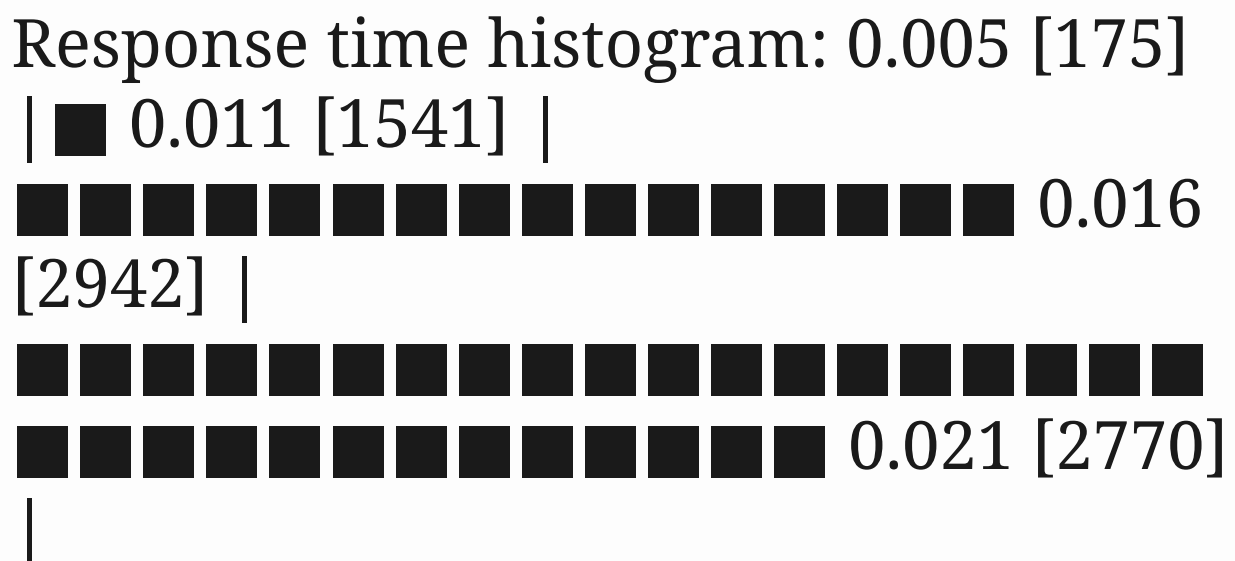
Details (average, fastest, slowest):
DNS+dialup: 0.0071 secs, 0.0001 secs,

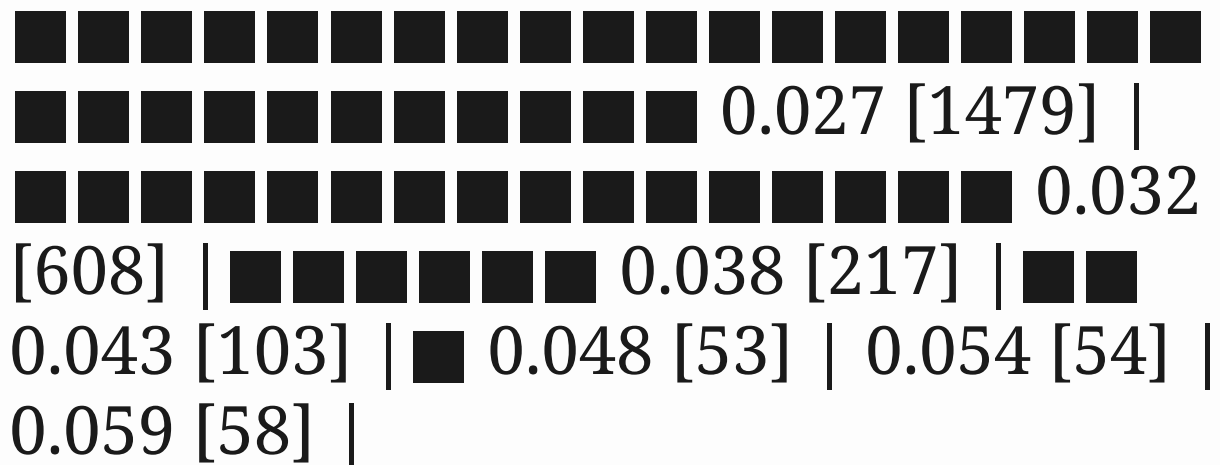
0.0443 secs DNS-lookup: 0.0000 secs,
0.0000 secs, 0.0010 secs

Status code distribution: [200] 10000
responses

Summary: Success rate: 1.0000 Total:
4.1314 secs Slowest: 0.0733 secs
Fastest: 0.0027 secs Average: 0.0206
secs Requests/sec: 2420.4851

Total data: 166.02 KiB Size/request: 17
B Size/sec: 40.18 KiB





Latency distribution: 10% in 0.0120 secs 25% in 0.0151 secs 50% in 0.0194 secs 75% in 0.0243 secs 90% in 0.0300 secs 95% in 0.0348 secs 99% in 0.0522 secs

Details (average, fastest, slowest):
DNS+dialup: 0.0088 secs, 0.0073 secs, 0.0103 secs
DNS-lookup: 0.0001 secs, 0.0000 secs, 0.0008 secs

Status code distribution: [200] 10000 responses

Summary: Success rate: 1.0000 Total:
1.8324 secs Slowest: 0.0269 secs
Fastest: 0.0024 secs Average: 0.0091
secs Requests/sec: 5457.2339

Total data: 117.19 KiB Size/request: 12 B Size/sec: 63.95 KiB

[illegible]

Latency distribution: 10% in 0.0061
secs 25% in 0.0073 secs 50% in 0.0087
secs 75% in 0.0105 secs 90% in 0.0129
secs 95% in 0.0143 secs 99% in 0.0171
secs

Details (average, fastest, slowest):
DNS+dialup: 0.0049 secs, 0.0035 secs,
0.0065 secs DNS-lookup: 0.0001 secs,
0.0000 secs, 0.0010 secs

Status code distribution: [200] 10000
responses

Summary: Success rate: 1.0000 Total:
13.5070 secs Slowest: 0.3635 secs
Fastest: 0.0249 secs Average: 0.0674
secs Requests/sec: 740.3558

Total data: 102.01 MiB Size/request:
10.45 KiB Size/sec: 7.55 MiB

Response time histogram: 0.016 [283]
 | ■ 0.032 [2616] |
 ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
 0.048 [4587] |
 ■
 ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ 0.064 [1829]
 | ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ 0.081 [362] |
 ■ ■ 0.097 [98] | 0.113 [105] | 0.129
 [20] | 0.145 [7] | 0.161 [28] | 0.177 [65]
 |

Latency distribution: 10% in 0.0493
secs 25% in 0.0559 secs 50% in 0.0638
secs 75% in 0.0733 secs 90% in 0.0840
secs 95% in 0.0948 secs 99% in 0.1543
secs

Details (average, fastest, slowest):
DNS+dialup: 0.0097 secs, 0.0001 secs,
0.0444 secs DNS-lookup: 0.0000 secs,
0.0000 secs, 0.0007 secs

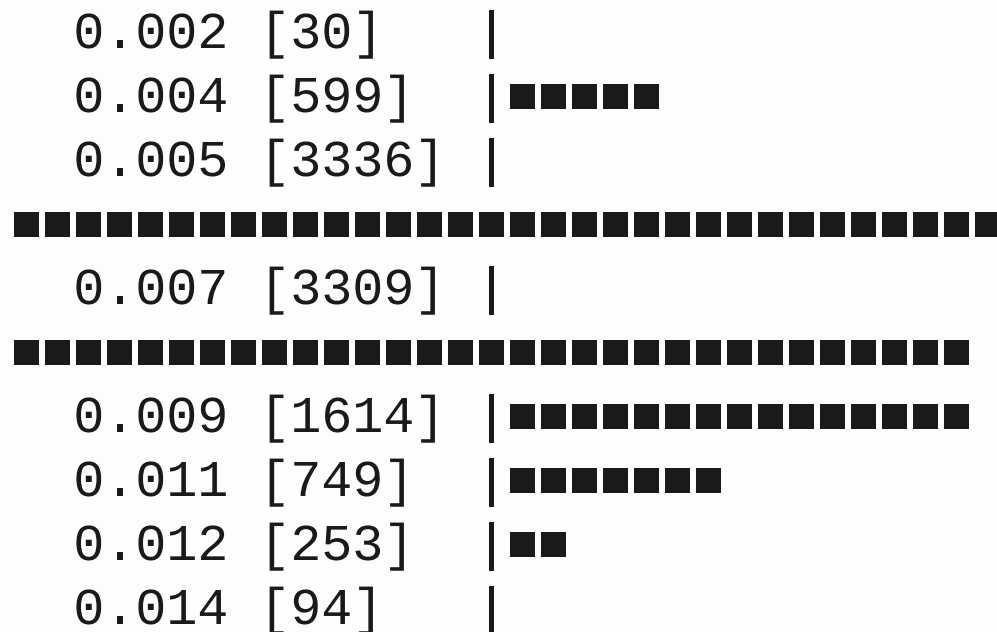
Status code distribution: [200] 10000 responses ``Robyn(with 5 workers)

Summary:

Success rate: 1.0000
Total: 1.5592 secs
Slowest: 0.0211 secs
Fastest: 0.0017 secs
Average: 0.0078 secs
Requests/sec: 6413.6480

Total data: 126.95 KiB
Size/request: 13 B
Size/sec: 81.42 KiB

Response time histogram:



```
0.016 [14] |
0.018 [1]  |
0.019 [1]  |
```

Latency distribution:

```
10% in 0.0055 secs
25% in 0.0063 secs
50% in 0.0074 secs
75% in 0.0089 secs
90% in 0.0107 secs
95% in 0.0117 secs
99% in 0.0142 secs
```

Details (average, fastest, slowest):

```
DNS+dialup:    0.0022 secs, 0.0013
secs, 0.0028 secs
DNS-lookup:    0.0000 secs, 0.0000
secs, 0.0001 secs
```

Status code distribution:

```
[200] 10000 responses
```

The process of hosting a Robyn app on various cloud providers.

Railway

We will be deploying the app on Railway.

A GitHub account is needed as a mandatory prerequisite.

We will deploy a sample “Hello World,” demonstrating a simple GET route and serving an HTML file.

Directory structure:

```
app folder/  
  main.py  
  requirements.txt  
  index.html
```

```Note - Railway looks for a main.py as an entry point instead of app.py. The build process will fail if there is no main.py file.



```
main.py
```

```
from robyn import Robyn, serve_html
```

```
app = Robyn(file)
```

```
@app.get("/hello") async def
h(request): print(request) return
"Hello, world!"
```

```
@app.get("/") async def
get_page(request): return
serve_html("./index.html")
```

```
if name == "main":
app.start(url="0.0.0.0", port=PORT)
```### index.html
```

```
<h1> Hello World, this is Robyn  
framework! <h1>  
```## [Exposing Ports]  
(/documentation/en/hosting#exposing-
```

ports)

The Railway documentation says the following about the listening to ports:

> The easiest way to get up and running is to have your application listen on `0.0.0.0:$PORT`, where `PORT` is a Railway-provided environment variable.

So, passing the host as ``0.0.0.0`` to ``app.start()`` as an argument is necessary.

We need to create a Railway account to deploy this app on Railway. We can do so by going on the ``Railway HomePage``.

Press the "Login" button and select "login with a GitHub account."

``

Then, we press the "New Project" button and select "Deploy from GitHub repo".

```

```

Then we select the repo we want to deploy. And click "Deploy Now".

```


```

Now, we click on our project's card.

Select "Variables" and press the "New Variable" button to set the environments variables.



Then, we go to the "Settings" tab and click on "Generate Domain."

We can generate a temporary domain under the "Domains" tab.



We can go to our domain ``<domain>/hello` and confirm that the message "Hello World" is displayed.`

## [Next Steps]  
(/documentation/en/hosting#next-steps)

\* [Future Roadmap]  
(/documentation/en/api\_reference/future-roadmap)

---

```
<!-- robyn-documentation source:
https://robyn.tech/documentation/en/plugins
-->
```

```
[Plugins]
(/documentation/en/plugins#plugins)
```

Robyn is a versatile and extensible web framework that allows anyone to make plugins over the top of Robyn. Plugins in Robyn allow you to enhance and customize the framework's functionality to suit your specific needs. Here are some noteworthy plugins that can supercharge your Robyn-based projects:

```
Rate Limit Plugin
```

\* Description: This plugin enables you to implement rate limiting for your Robyn application's routes. It helps prevent abuse, and brute-force attacks

and ensures fair usage of your resources.

- \* GitHub repository: [robyn-rate-limits]

([https://github.com/IdoKendo/robyn\\_rate\\_](https://github.com/IdoKendo/robyn_rate_limits)

- \* Installation:

```
`python -m pip install robyn-rate-limits`
```

- \* Usage:

```
from robyn import Robyn, Request
from robyn_rate_limits import
InMemoryStore from
robyn_rate_limits import RateLimiter
```

```
app = Robyn(file) limiter =
RateLimiter(store=InMemoryStore,
calls_limit=3, limit_ttl=100)
```

```
@app.before_request() def
middleware(request: Request): return
limiter.handle_request(app, request)
```

```
@app.get("/") def h(): return "Hello,
World!"
```

app.start(port=8080) ``In this example, robyn-rate-limits is used to enforce a rate limit of 3 requests per 100-seconds window for specific routes. If a client exceeds this limit, they will receive a "Too many requests" message.

The plugin integrates seamlessly with the Robyn web framework, enhancing the security and stability of your application by preventing excessive requests from a single client.

## **What's next?**

After exploring the plugins, Batman wanted to explore the community. So,

Robyn pointed him to

- Future Roadmap